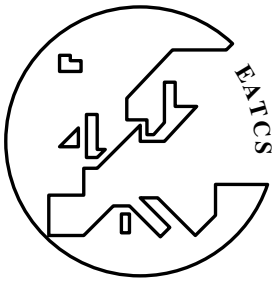
ISSN 0252–9742

Bulletin

of the

European Association for Theoretical Computer Science

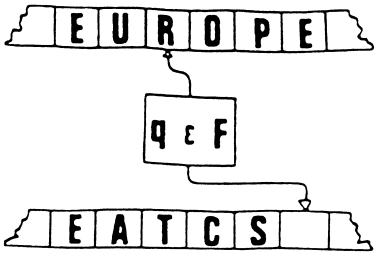
EATCS



Number 149 June 2026

EATCS

General Assembly Members



President:	Giuseppe F. Italiano	Italy
Vice Presidents:	Antoine Amarilli	France
	Inge Li Gørtz	Denmark
Treasurer:	Gabriele Fici	Italy
Bulletin Editor:	Stefan Schmid	Germany

Antoine Amarilli	France	Giuseppe F. Italiano	Italy
Alkida Balliu	Italy	Kasper Green Larsen	Denmark
Ivona Bezakova	USA	Yannic Maus	Austria
Thomas Colcombet	France	Anca Muscholl	France
Anne Driemel	Germany	Yasamin Nazari	Netherlands
Funda Ergün	USA	Charles Paperman	France
Javier Esparza	Germany	Eva Rotenberg	Denmark
Gabriele Fici	Italy	Jukka Suomela	Finland
Pawel Gawrychowski	Poland	Szymon Torunczyk	Poland
Inge Li Goertz	Denmark	Bianca Truthe	Germany

Past Presidents:

Maurice Nivat	(1972–1977)	Mike Paterson	(1977–1979)
Arto Salomaa	(1979–1985)	Grzegorz Rozenberg	(1985–1994)
Wilfred Brauer	(1994–1997)	Josep Díaz	(1997–2002)
Mogens Nielsen	(2002–2006)	Giorgio Ausiello	(2006–2009)
Burkhard Monien	(2009–2012)	Luca Aceto	(2012–2016)
Paul Spirakis	(2016–2020)	Artur Czumaj	(2020–2024)

Secretary Office:	Dmitry Chistikov	UK
	Efi Chita	Greece

EATCS General Assembly Members

email addresses

Antoine Amarilli . . . . . a3nm@a3nm.net
Alkida Balliu . . . . . alkida.balliu@gssi.it
Ivona Bezakova . . . . . ib@cs.rit.edu
Thomas Colcombet . . . . . Thomas.Colcombet@irif.fr
Anne Driemel . . . . . driemel@cs.uni-bonn.de
Funda Ergün . . . . . chair.sigact@sigact.acm.org
Javier Esparza . . . . . esparza@in.tum.de
Gabriele Fici . . . . . gabriele.fici@unipa.it
Pawel Gawrychowski . . . . . awry@cs.uni.wroc.pl
Inge Li Goertz . . . . . inge@dtu.dk
Giuseppe F. Italiano . . . . . gitaliano@luiss.it
Kasper Green Larsen . . . . . larsen@cs.au.dk
Yannic Maus . . . . . yannic.maus@tugraz.at
Anca Muscholl . . . . . anca@labri.fr
Yasamin Nazari . . . . . y.nazari@vu.nl
Charles Paperman . . . . . charles.paperman@univ-lille.fr
Eva Rotenberg . . . . . eva@rotenberg.dk
Stefan Schmid . . . . . stefan.schmid@tu-berlin.de
Jukka Suomela . . . . . jukka.suomela@aalto.fi
Szymon Torunczyk . . . . . szymtor@mimuw.edu.pl
Bianca Truthe . . . . . bianca.truthe@informatik.uni-giessen.de

Bulletin Editor: Stefan Schmid, Berlin, Germany
Cartoons: DADARA, Amsterdam, The Netherlands

The bulletin is entirely typeset by pdfTEX and ConTEXt in TXfonts.

All contributions are to be sent electronically to

bulletin@eatcs.org

and must be prepared in LaTeX 2ε using the class `beatcs.cls` (a version of the standard LaTeX 2ε `article` class). All sources, including figures, and a reference PDF version must be bundled in a ZIP file.

Pictures are accepted in EPS, JPG, PNG, TIFF, MOV or, preferably, in PDF. Photographic reports from conferences must be arranged in ZIP files layed out according to the format described at the Bulletin's web site. Please, consult `http://www.eatcs.org/bulletin/howToSubmit.html`.

We regret we are unfortunately not able to accept submissions in other formats, or indeed submission not *strictly* adhering to the page and font layout set out in `beatcs.cls`. We shall also not be able to include contributions not typeset at camera-ready quality.

The details can be found at `http://www.eatcs.org/bulletin`, including class files, their documentation, and guidelines to deal with things such as pictures and overfull boxes. When in doubt, email `bulletin@eatcs.org`.

Deadlines for submissions of reports are January, May and September 15th, respectively for the February, June and October issues. Editorial decisions about submitted technical contributions will normally be made in 6/8 weeks. Accepted papers will appear in print as soon as possible thereafter.

The Editor welcomes proposals for surveys, tutorials, and thematic issues of the Bulletin dedicated to currently hot topics, as well as suggestions for new regular sections.

Table of Contents

EATCS MATTERS
Letter from the President 3
Letter from the Editor 7

EATCS COLUMNS

The Interview Column, *by C. Avin, S. Schmid*
Know the Person behind the Papers Today:
Nancy Lynch, ... 13

Viewpoint Column *by A. Muscholl, S. Schmid*
Adding "Verified" Badges to Our Paper, One
Lemma at a Time, *by M. Függer, T. Nowak* 23

Viewpoint Column *by A. Muscholl, S. Schmid*
Disordered Reflections, *by M. Raynal* 35

TCS on the Web, *by S. Neumann*
Decentralized Thoughts: Blogging Consensus,
Blockchains, and Cryptography, *by S. Neumann* 43

The Logic in Computer Science Column, *by A. Dawar*
Assisted computations: Inner view, *by Y. Gurevich* 51

The Education Column , *by D. Komm,T. Zeume*
"Good In Theory: How Do LLMs Perform In a
Theory Course?", *by M. Stocker, D. Komm,* 77

The Concurrency Column, *by M. Miculan, N. Yoshida*
Decision Problems for Global Protocol
Specifications, *by E. Li* 95

The Concurrency Column, *by M. Miculan, N. Yoshida*
Typing Scalas-Yoshida Benchmarks, *by*
F. Barbanera, M. Dezani-Ciancaglini, U. de Liguoro 137

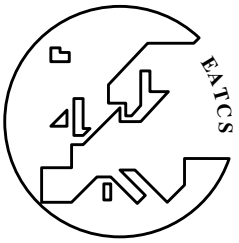
Conference Reports Column, *by L. Feuilloley*
Conference report on STACS 2026, *by A. Amarilli, N. Bousquet, J. Cauvi, U. Giocanti* 151

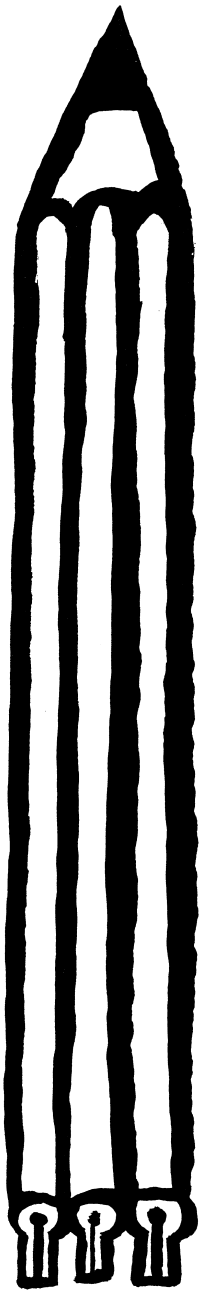
News and Conference Reports

Report from EATCS Japan Chapter, *by Y. Nakashima* 159

EATCS LEAFLET 164

EATCS Matters





Dear EATCS members,

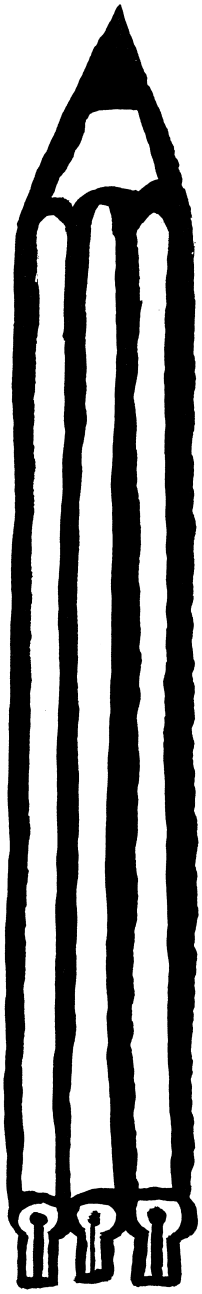
As the Summer of 2026 begins, I would like to send my warm greetings and best wishes to all of you. I hope you are doing well and enjoying exciting research in theoretical computer science.

The June issue of the Bulletin is released just in time for our flagship event, ICALP 2026. This year's edition will be particularly special, as ICALP is co-located with PODC and SPAA. As usual, our annual General Assembly will take place during ICALP, and I very much hope that many of you will join us at Royal Holloway, University of London, from July 7 to 10.

Matthew Hague and his dedicated team at Royal Holloway have worked extremely hard to organize what promises to be an outstanding edition of ICALP. I am confident that the conference will offer an excellent scientific program, stimulating discussions, and many opportunities to reconnect with colleagues and friends.

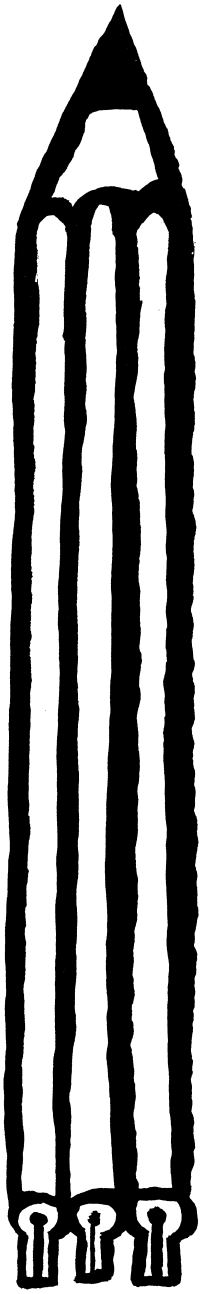
In addition to the invited and contributed talks, ICALP 2026 will also be an opportunity to celebrate some of the remarkable achievements of our community. We are delighted to announce the recipients of this year's prestigious awards:

- *The 2026 Gödel Prize will be awarded to Ilias Diakonikolas, Gautam Kamath, Daniel Kane, Jerry Li, Ankur Moitra and Alistair Stewart, for their paper "Robust Estimators in High Dimensions without the Computational Intractability", SIAM Journal on Computing, 48(2): 742-864, 2019.*



- *The EATCS Award 2026 will be presented to the distinguished Yishay Mansour for many fundamental seminal results. His contributions in the areas of machine learning, reinforcement learning, algorithmic game theory and communication networks are outstanding. He consistently continues to produce groundbreaking results. His work, old and new, has had a profound impact and will continue to influence us for many years to come.*
- *The 2026 Alonzo Church Award for Outstanding Contributions to Logic and Computation will be presented to Pablo Barceló, Leonid Libkin, Wim Martens, Juan Reutter, Miguel Romero, Moshe Vardi and Domagoj Vrgoč for laying foundations of logical languages for querying property graphs that found adoption in the SQL and GQL international standards within a decade.*
- *The Presburger Award 2026 will honor the remarkable contributions Vincent Cohen-Addad and Gautam Kamath.*
- *The EATCS Distinguished Dissertation Award will recognize the outstanding doctoral research of Noah Golowich, Ce Jin, and Hanlin Ren.*

  Furthermore, we will have the distinct pleasure of inducting the EATCS Fellows Class of 2026, celebrating individuals who have made profound and lasting impacts on our field:
- *Hagit Attiya, Technion, Israel, for "landmark results to many areas in the theory and design of distributed*



systems, and continuously played a leading role in shaping the scientific discourse on distributed systems."

- *Joël Ouaknine, Max Planck Institute for Software Systems, Germany, for "fundamental contributions to the algorithmic analysis of dynamical systems and related formalisms."*

- *Prakash Panangaden, McGill University (Emeritus Professor) and Edinburgh University (Honorary Professor), for "seminal contributions to the semantics and logic of probabilistic computation, leading to extensive applications in systems modelling, verification, and machine learning."*

Congratulations to all the awardees! It will be a great pleasure and privilege to celebrate their remarkable achievements and lasting contributions together at ICALP 2026. I would also like to express my sincere gratitude to the members of the award and fellows committees for their thoughtful and dedicated work in selecting this outstanding group of honorees.

I also warmly invite you to attend the EATCS General Assembly, which will take place during ICALP 2026. This is a vital opportunity to gain detailed insights into the EATCS's activities over the past year and our exciting plans for the year ahead. Your presence and engagement are invaluable as we shape the future of our association.

I very much look forward to seeing many of you at Royal Holloway, not only to



share the scientific excitement of ICALP 2026, but also to engage in meaningful discussions during the General Assembly on how we can further strengthen the impact of EATCS within the broader computer science community.

As always, please know that my inbox is open. Your comments, suggestions, and constructive feedback are warmly welcome and deeply valued. EATCS is a community-driven organization, and your ideas play an essential role in helping us improve our activities and better serve the field of theoretical computer science. Every suggestion will receive careful consideration.

I sincerely look forward to connecting with many of you at ICALP 2026, or hearing from you through your thoughtful messages.

Giuseppe F. Italiano
Luiss University, Rome
President of EATCS
president@eatcs.org

June 2026



Dear EATCS member!

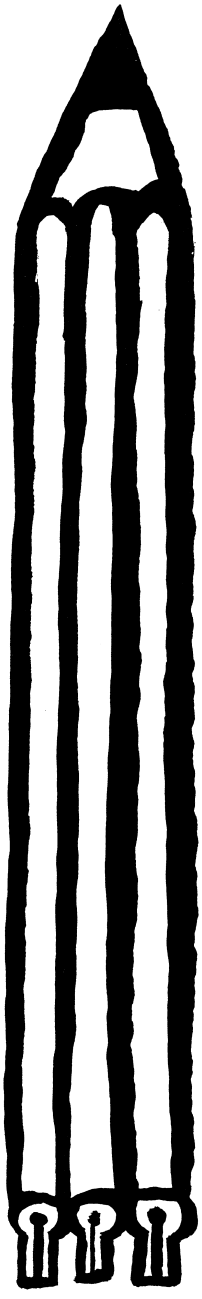
I hope you are ready for the Summer! This issue of the EATCS Bulletin comes with plenty of reading materials.

In the Interview Column, we had the pleasure of speaking with Nancy Lynch who reflects on her scientific journey, the development of the theory of distributed systems, and the people and ideas that influenced her remarkable career.

In the Viewpoint Column, Matthias Függer and Thomas Nowak discuss their experience using AI coding agents and Lean to formally verify results from a theoretical computer science paper, discussing the opportunities and challenges that such tools may bring to our community. The column also features a thought-provoking essay by Michel Raynal, who reflects on the role of research and teaching in universities, the nature of informatics, and the broader mission of the teacher-researcher.

In the TCS on the Web Column, Stefan Neumann interviews several contributors of Decentralized Thoughts, the influential blog on distributed computing, blockchains, and cryptography. The discussion highlights how blogs can complement traditional papers by providing intuition, educational material, and connections across research communities.

In the Logic in Computer Science Column, Yuri Gurevich presents new developments in the theory of Abstract State Machines. Building on earlier work on algorithms and computation, he develops an "inner view" of assisted computation, where algorithms



interact with external environments and oracles.

In the Education Column, Moritz Stocker and Dennis Komm investigate how modern large language models perform on undergraduate Theory of Computing exams. Their blind-grading experiment compares GPT-generated solutions against student submissions and provides insights into the rapidly evolving capabilities of AI systems.

The Concurrency Column features two contributions on the specification and verification of concurrent systems. Elaine Li surveys recent advances on implementability, synthesis, and subtyping for global communication protocols. And Franco Barbanera, Mariangiola Dezani-Ciancaglini, and Ugo de' Liguoro demonstrate the expressive power of modern session-type frameworks through benchmark protocols.

The Conference Column reports on STACS 2026 in Grenoble, highlighting both the scientific program and the surrounding community.

Finally, the Bulletin includes a report from the Japanese Chapter of the EATCS, covering the 24th EATCS-JP/LA Workshop on Theoretical Computer Science and celebrating this year's presentation award winners.

Enjoy the new issue of the Bulletin!

Stefan Schmid, Berlin
June 2026

Institutional Sponsors

CTI, Computer Technology Institute & Press "Diophantus"
Patras, Greece

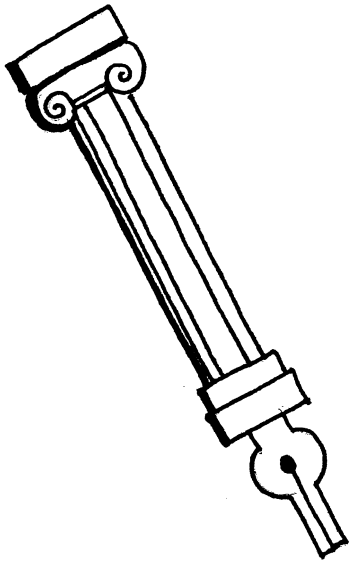
CWI, Centum Wiskunde & Informatica
Amsterdam, The Netherlands

MADALGO, Center for Massive Data Algorithmics
Aarhus, Denmark

Microsoft Research Cambridge
Cambridge, United Kingdom

Springer-Verlag
Heidelberg, Germany

EATCS
Columns



The Interview Column

BY

Chen Avin and Stefan Schmid

Ben Gurion University, Israel and TU Berlin, Germany
{chenavin,schmiste}@gmail.com

KNOW THE PERSON BEHIND THE PAPERS

Today: Nancy Lynch

Bio: *Nancy Lynch has been a Professor in the EECS Department at MIT since 1982. She heads the Theory of Distributed Systems research group in MIT's Computer Science and Artificial Intelligence Lab. She received her B.S. from Brooklyn College in 1968 and her PhD from MIT in 1972, both in Math. Prior to joining MIT, she served on the Math faculties at Tufts and the University of Southern California, and on the CS faculty at Georgia Tech.*

Lynch has written and co-written hundreds of research articles about distributed algorithms and impossibility results, and about formal modeling and verification of distributed systems. Her best-known contribution is the 1982 "FLP" impossibility result for distributed consensus in the presence of process failures, with Fischer and Paterson, followed by a paper with Dwork and Stockmeyer on algorithms for reaching consensus under restricted failure assumptions. Other contributions include the I/O Automata system modeling frameworks, and more recent results on wireless network algorithms and biological distributed algorithms.

Lynch is the author of the textbook "Distributed Algorithms" and co-author of "Atomic Transactions" and "The Theory of Timed I/O Automata". She is an ACM Fellow, a Fellow of the American Academy of Arts and Sciences, and a member of both the National Academies of Sciences and Engineering. She has been awarded many prestigious prizes for her research contributions. She has supervised over 100 graduate students and postdocs, many of whom have themselves become leading researchers.

Can you give a quick summary of your main research contributions?

Nancy: Basically, starting in the late 1970s, working at Georgia Tech and MIT, I helped to develop the field of distributed computing theory. This included algorithms, impossibility results, and formal models and methods. You can read a detailed overview of much of this work in my recent paper "Building a Theory of Distributed Systems: Work by Nancy Lynch and Collaborators" [6].

We ask all interviewees to share a photo with us. Can you please tell us a little about the photo you shared?

Nancy: I found two. The first was taken early in my career, soon after I moved from Georgia Tech to MIT. So that would probably be around 1982. It shows me at the blackboard, explaining some type of argument about distributed consensus. The second was taken late in my career, during a 2019 visit to the Sorbonne, where I was honored with a "Doctorat Honoris Causa" degree.



Who were your main research mentors and how did they influence you?

Nancy: Albert Meyer, my thesis advisor at MIT, taught me how to do research, and in particular, taught me extremely high standards for selecting problems and carrying out technical work. Mike Fischer was my main collaborator during the late 1970s and 1980s, in my early work on distributed computing theory. I appreciate his great ideas and his great judgment about problems and approaches. Both Albert and Mike provided me with contacts in the field of theoretical computer science, helping me to get appointed to program committees, etc. Phil Enslow at Georgia Tech taught me about distributed systems, and suggested papers for me to read, especially those that might lead in theoretical directions. He also provided me with funding contacts.

Can you please tell us something about you that probably most of the readers of your papers don't know?

Nancy: I finished my PhD at the young age of 24, working in complexity theory. I didn't want to keep working in that area, and was not ready yet to choose a new area to commit to for a long time. So I explored a few things for several years.



Finally, at Georgia Tech, with Phil Enslow's influence, I got excited about something that seemed worth serious investment: theory for the new area of distributed systems.

Besides working, I spent a lot of time engaging in outdoor activities with my husband and children. This was mostly hiking and skiing in New Hampshire's White Mountains. We climbed all 48 4000-foot mountains in New Hampshire, which gave us entry to the Appalachian Mountain Club's "4000-Footer Club". We still go to the White Mountains frequently, now also with our grandchildren.

Is there a paper which influenced you particularly, and which you recommend for other community members to read?

Nancy: I was influenced by a 1975 report by Paul Johnson and Robert Thomas on "The Maintenance of Duplicate Databases." I found a version on-line [3]. Also by another paper around that same time by Armin Cremers and Tom Hibbard giving a lower bound for the space requirements needed for solving mutual exclusion—this was the first lower bound I had encountered in the area of distributed computing. Unfortunately, I can't find an on-line version of this—I knew it only as a University of Southern California technical report. The early papers

by Lamport are worth reading—particularly the famous "Time, Clocks, and the Ordering of Events in a Distributed System" [5].

Is there a paper of your own that you would like to recommend for the readers to study? What is the story behind this paper?

Nancy: The Fischer, Lynch, Paterson paper on "Impossibility of Consensus in a Distributed System", commonly called "FLP", is my best-known and best-cited paper [2]. It's only 9 pages long, and easy to read. The "story" behind the paper is well summarized in [6]. Briefly, Mike Fischer and I knew that this was a problem of interest in the distributed systems community. We knew that we could formalize it in theoretical, algorithmic terms, and study it theoretically. Mike Paterson joined us in working out the proof. Actually, we went back and forth, trying to produce an algorithm and an impossibility result.

The FLP paper led to many related and follow-on papers. My most important follow-on paper was "Consensus in the Presence of Partial Synchrony", with Cynthia Dwork and Larry Stockmeyer [1]. This showed practical ways around the inherent limitation expressed by the FLP impossibility result.

Other worthwhile papers are my two I/O Automata papers with Mark Tuttle [7, 8]. The I/O Automata model is a useful, general abstract model for asynchronous distributed systems. It has good facilities for composition and abstraction. However, it wasn't general enough to model all interesting distributed systems. So Dilsun Kaynar, Roberto Segala, Frits Vaandrager, and I designed Timed I/O Automata [4], which included timing and continuous behavior. Now I don't think that we can find one general model that will be suitable for modeling all distributed systems. For example, in modeling biological systems (such as insect colonies or brain networks), we may want to make a variety of different assumptions about timing, continuous vs. discrete behavior, probability, composition, and abstraction.

You could also read the paper I mentioned above [6], which covers most of my research career.

When (or where) is your most productive working time (or place)?

Nancy: I work best at home, in the morning between around 8AM and noon. I never could work well in the evening, and the afternoons are OK but somewhat less productive.

I like to focus on difficult things for a few hours each day and then do something easier. For good advice on working productively, you might want to check out Cal Newport's books or website. (Cal was my PhD student.)

How do you choose what to work on? And what kind of impact are you hoping your work will have? Did this change over the course of your career?

Nancy: I basically work on what interests me. I try to choose problems whose solution could be important for science (usually computer science but recently biology). I work on problems for which I can see a path forward, that is, for which I think I have the right theoretical tools to make progress on them. I like to explore new application areas.

What do you do when you get stuck with a research problem? How do you deal with failures?

Nancy: Well, I flounder around, trying everything I can think of. I talk to other researchers about the problem and difficulties. If I still feel stuck, I put the problem aside and plan to come back to it when a new idea appears. Sometimes that never happens. Sometimes I decide it's not the right problem to work on.

Is there a nice anecdote from your career you like to share with our readers?

Nancy: I was a member of the famous "All-Women Program Committee" for STOC 1989. Christos Papadimitriou was the committee chair, and he decided to try to form a program committee consisting entirely of women. In those days, it was not so obvious that that was feasible, because there were not that many women working in theoretical computer science. There was some actual controversy about this effort, with arguments in Sigact News about whether such a committee would be sufficiently qualified to evaluate everyone's papers.

The members of the committee, in addition to Christos and myself, were Fan Chung, Cynthia Dwork, Faith Fich, Shafi Goldwasser, Debbie Joseph, Maria Klawe, Vijaya Ramachandran, Eva Tardos, Avi Wigderson, and Frances Yao. (Avi Wigderson was very enthusiastic about the project, and wanted to be included.) A very illustrious group of researchers.

Well, the committee did a great job, and the conference turned out great. I think it drew very favorable attention to the women in the field. There was one thing, though. I have never attended such an intense PC meeting! It seemed like everyone had read all of the submitted papers, and had very, very strong opinions about them. Discussions weren't cut off. The result was that the (one-day, in-person) meeting went on for more than 12 hours, running well past a normal dinner hour. It was exhausting!

Do you have any advice for young researchers? In what should they invest time, what should they avoid?

Nancy: My approach is to look for problems of practical significance for some important discipline, and then work on them using good theory.

What are the most important features you look for when searching for graduate students?

Nancy: My students have generally found me, so they already have read some of my papers and have some knowledge and interest in theory of distributed systems. Other than that, they should be very smart and hard-working, but that pretty much describes all MIT students. They should be friendly, cooperative, helpful to other students, so they contribute to a nice research group.

Do you see a main challenge or opportunity for theoretical computer scientists for the near future?

Nancy: Personally, I favor good theory applied to topics of practical importance, such as biological systems. For example, studying insect colony behavior can lead to theoretical study of swarms of robots and other coordinated activities. I am also interested in algorithms that explain how brains accomplish various tasks. Theoreticians should study these at a high level of abstraction, while still trying to capture essential properties of the behavior of actual brains.

Learning is another very important topic, though there is already quite of bit of theoretical study related to learning. Of this work, I prefer the study of learning algorithms that are like those used in actual brains, rather than those that arise in Artificial Neural Networks.

Please complete the following sentences?

- *Being a researcher...*

  Being a researcher is a great job. You have the freedom to think of new ideas and work them out. You get to work with students, postdocs, and other colleagues on research problems. You can also write books, travel and give talks, plan workshops and conferences...you have so many interesting opportunities.

- *... is key to being a happy academic.*

  The key to being a happy academic is not to get overwhelmed by all the interesting opportunities. Focus on one or two at a time. Also, as an academic, you are constantly presented with requests to do stuff. Be sure to say no to enough of these.

- *My first research discovery...*

  My first result was the main result in my thesis on abstract complexity theory. It showed the existence of problems that don't help in solving certain other problems. The argument was a complicated diagonalization construction. In distributed computing theory, my first results were algorithms and lower bounds on the space requirements for solving mutual exclusion, in a paper with Jim Burns, Mike Fischer, and others.

- *Theoretical computer science in 100 years from now...*

  I have no idea what it will be like. I have no idea what computer science will be like in 100 years. Neither does anyone else.

References

[1] C. Dwork, N. Lynch, and L. Stockmeyer. Consensus in the presence of partial synchrony. *J. ACM*, 35(2):288–323, Apr. 1988.

[2] M. J. Fischer, N. A. Lynch, and M. S. Paterson. Impossibility of distributed consensus with one faulty process. *J. ACM*, 32(2):374–382, Apr. 1985.

[3] P. Johnson and R. Thomas. The maintenance of duplicate databases, 1975.

[4] D. K. Kaynar, N. A. Lynch, R. Segala, and F. W. Vaandrager. *The Theory of Timed I/O Automata, Second Edition*. Synthesis Lectures on Distributed Computing Theory. Morgan & Claypool Publishers, 2010.

[5] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Commun. ACM*, 21(7):558–565, July 1978.

[6] N. Lynch. Building a theory of distributed systems: Work by Nancy Lynch and collaborators, 2025.

[7] N. A. Lynch and M. R. Tuttle. Hierarchical correctness proofs for distributed algorithms. In F. B. Schneider, editor, *Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing, Vancouver, British Columbia, Canada, August 10-12, 1987*, pages 137–151. ACM, 1987.

[8] N. A. Lynch and M. R. Tuttle. An introduction to Input/Output Automata. *CWI-Quarterly*, 2(3):219–246, September 1989. Centrum voor Wiskunde en Informatica, Amsterdam, The Netherlands. Technical Memo MIT/LCS/TM-373, Laboratory for Computer Science, Massachusetts Institute of Technology, Cambridge, MA 02139, November 1988.

Viewpoint Column

by

Anca Muscholl and Stefan Schmid

Bordeaux University, France
and TU Berlin, Germany
anca@labri.fr and stefan.schmid@tu-berlin.de

BEATCS no 149

Adding "Verified" Badges to Our Paper, One Lemma at a Time

Matthias Függer and Thomas Nowak

LMF, CNRS, ENS Paris-Saclay, Université Paris-Saclay

Abstract

We describe our experience using AI coding agents to formally verify proofs from a theoretical computer science paper in Lean 4. Using a two-agent workflow where one agent generates proofs and another checks their faithfulness to the paper, we formalized several of the paper's results, including the computational model. We did not know Lean before starting. We reflect on what worked, what did not, what infrastructure is missing, and where this might lead for TCS and for mathematics more broadly.

1 Introduction

Formal verification of mathematical proofs has a long history. The goal of these undertakings was to express valid proof steps through explicit symbolic rules, making mathematical reasoning precise and easier to verify systematically. Early examples include Frege's *Begriffsschrift* (1879) [7], Hilbert's formal axiomatic program (1928) [16], and *Principia Mathematica* by Whitehead and Russell (1910) [22]. These developments eventually led to foundational work connecting symbolic rules with computability and proof theory by Gödel [13], Church [2], Turing [20], and Gentzen [12]. One of the earliest computer-assisted formal verification projects was de Bruijn's Automath system in the late 1960s [5], and the idea that computers should check our proofs is nearly as old as the idea that computers should help us find them. The four-color theorem was proved with computer assistance in 1976 and formally verified in Coq (now Rocq) by Gonthier and Werner in 2005 [14]. Hales's proof of the Kepler conjecture, after years of controversy over its computer-assisted parts, was formally verified in the Flyspeck project in 2014 [15].

Yet after more than half a century, the vast majority of theorems in mathematics and theoretical computer science remain verified only by human peer review. The reason is not that formal verification does not work. It is that it has cost too much.

Formalizing a single nontrivial theorem in a proof assistant like Lean [6], Rocq [1], or Isabelle [18] has traditionally required a specialist.

This is changing. AI coding agents can now interact with proof assistants autonomously: they write proof terms, compile them, read the error messages, and iterate. The human cost of formalization is dropping, and this is starting to reshape how mathematical knowledge is recorded and trusted, in TCS and beyond.

This column is an account of our own experiment in this direction. We recently had AI agents formally verify some of the proofs in a paper on distributed computing [8]. The paper studies averaging algorithms under oblivious message adversaries, introduces a notion of asymptotic subspace consensus, and gives a characterization of solvability. Several of the results now carry green "Verified in Lean" badges in the PDF, each linking to a compiled Lean 4 proof. Neither of us knew Lean before starting.

2 The Two-Agent Workflow

The story begins with a social media post. Jukka Suomela, whose work in distributed computing many readers of this Bulletin will know, posted about a workflow he uses: ask a chatbot to solve a math problem, then ask it to formalize the proof in Lean 4. The chatbot discovers mistakes in its own reasoning through the formalization attempt, fixes them, and tries again. Suomela pointed out that, in principle, he does not even need to look at the Lean code or know what Lean is. The formalization attempt acts as a kind of rubber duck, forcing the model into rigor.

We tried this with ChatGPT, and the iterative self-correction worked as described. But we were suspicious: was the chatbot actually producing valid Lean code? ChatGPT confirmed that it does not have access to the Lean compiler. So we installed Lean and tried to build the code. It did not build.

What followed was tedious but instructive. We went back and forth with ChatGPT, pasting compiler errors, and it identified issues each time. This was slow, because we were acting as a human compile-paste loop. The obvious next step was to use an AI coding agent that could run the Lean compiler itself.

We launched Codex, OpenAI's coding agent, in a Lean project directory and asked it to prove one of the lemmas from the paper. Codex can on its own explore relevant parts in the tex source, run shell commands, compile its own code, read the errors, and iterate. After a few minutes, it produced a Lean proof that compiled. But the Lean statement did not match the English statement of the lemma. Codex had introduced additional hypotheses not present in the paper.

This is a natural failure mode, and in hindsight an obvious one: an agent optimizing for "make it compile" will strengthen hypotheses or weaken conclusions

until Lean accepts the proof. The resulting theorem may be trivially true and bear no relation to the intended claim.

The fix was to use a second agent to check the first one's work. We set up Claude Code, Anthropic's coding agent, to review Codex's output against the paper's LaTeX source. Claude reads both the Lean file and the corresponding section of the paper, and checks for additional hypotheses not present in the paper, weakened conclusions, `sorry` placeholders that mark incomplete proofs or missing arguments, and mismatches between the Lean definitions and the paper's notation. When Claude finds issues, it writes a detailed report specifying what is wrong and how the Lean code deviates from the paper. Codex reads this report and tries again.

This two-agent loop runs with minimal human intervention. In practice, it required several iterations per lemma. Codex would often stop early with an incorrect formalization, and Claude would catch the discrepancy. Sometimes the issue was subtle: a universal quantifier over all vectors replaced by a quantifier over unit vectors, or a strict inequality weakened to a non-strict one. Sometimes it was gross: an entire hypothesis added that made the theorem trivial. The checking agent caught both kinds.

The workflow has a clear separation of concerns. Codex's job is to produce Lean code that compiles. Claude's job is to ensure that what compiles is faithful to the paper. The human's job is to write the paper, to decide which results to formalize, and to verify that the Lean definitions capture the intended concepts.

The agent instructions, which specify the protocol, are available on GitHub [11], and a short demo video of the workflow is also online [9].

3 What We Learned

Our paper involves processes that update their state by taking weighted averages of received values, subject to a communication pattern controlled by an adversary. The main results concern convergence properties and impossibility results characterized by graph-theoretic properties of the communication patterns.

We started with a warm-up: a basic geometric fact about the distance from a point to an open halfspace defined by a hyperplane. This was straightforward, and Codex handled it in a single pass. It gave us confidence that the workflow could produce real results.

The hardest part of the entire formalization was not a proof but the definitions. Formalizing the computational model in Lean, the processes, the rounds, the message adversary, the averaging updates, required making dozens of modeling choices that have no obvious right answer. How should the states be represented? What type should the communication graph have? These choices are invisible in the paper, where we write "each process updates its state by averaging the values it

received" and move on. In Lean, every implicit convention must be made explicit, and this took more iterations, and more human oversight, than any of the proofs.

Once the model was in place, several convergence lemmas went through without major issues. These are geometric arguments involving convex combinations, projections, and distances to halfspaces. Lean's mathematical library Mathlib [3] had good coverage of the relevant linear algebra, and Codex could find the right lemmas and chain them together. We also formalized an impossibility theorem, a result showing that certain convergence guarantees cannot be achieved under certain adversary classes. This is a combinatorial argument about the structure of communication graphs, and it required a different style of Lean proof from the geometric lemmas. The Lean code is nontrivial, and the agents produced all of it, with human oversight on the modeling choices.

Our paper also contains results on Steiner symmetrization of convex bodies, which we did not formalize. Our objective was to verify some of the paper's results, not all of them, and we stopped before tackling the symmetrization arguments.

The benefits of the workflow extend beyond the badges that end up in the final PDF. On a separate paper of ours on chemical reaction networks, we attempted the same two-agent loop and did not produce a formalization we were happy with, so we submitted the paper without badges. The attempt was still worthwhile: along the way, the formalization uncovered errors in the manuscript and suggested how to fix them. Even an unfinished formalization can serve as a careful proofreader.

The cost was modest. The entire formalization took days, not months, and required no prior Lean expertise from either of us. The monetary cost amounted to tens of euros. For comparison, a human Lean expert would likely have done a better and more reusable job, but would have taken longer and cost more. The agents gave us a quick, if somewhat brittle, path to compiled proofs.

To make formal verification visible in the paper, we developed a small LaTeX package called `verified-badges` [10]. It provides commands that add a clickable badge next to a theorem header, linking directly to the proof source. The package supports eleven proof assistants and is released under CC0. For example, the warm-up halfspace lemma mentioned at the start of this section appears in our paper as

Lemma (VERIFIED Lean)**.** *Let point $q \in \mathbb{R}^d$ and normal vector $v \in \mathbb{R}^d \setminus \{0\}$ define the half-space $H = \{h \in \mathbb{R}^d \mid \langle h - q, v\rangle < 0\}$. Then* $\operatorname{dist}(\{z\}, H) = \langle z - q, v\rangle / \|v\|$ *for all points $z \in \mathbb{R}^d \setminus H$.*

and the package likewise produces VERIFIED Rocq for Rocq, VERIFIED Isabelle for Isabelle, and so on. A blue variant, such as FORMALIZED Lean, marks statements whose formalization is complete but whose proof uses `sorry`. The same idea has been used independently in the Rocq community: Leray and Winterhalter's recent paper [17] marks each

formalized lemma and theorem with a small Rocq icon next to its header. A green badge indicates that the statement and proof have been translated into a formal language and that the result type-checks. This is a different claim than an English proof alone, even if it is not an absolute guarantee: the formalization gap remains.

4 The Missing Libraries

The problem of formalizing the right statements is not specific to our paper, or to distributed computing, or even to computer science. Mathematics rests on informal consensus about what definitions mean. When we write "let G be a graph," we rely on a shared understanding that has been stable for a century. A proof assistant does not have that shared understanding unless someone has encoded it.

For core mathematics, Mathlib now provides a large and well-maintained library: groups, rings, topological spaces, measure theory, and much more. It contains over two million lines of Lean code and is growing fast. When we needed a fact about inner products or convex combinations, Mathlib had it, and Codex could find it. This is why the geometric parts of our formalization went smoothly: someone had already done the hard work of defining the right abstractions.

But beyond core mathematics, the coverage is patchy. Physics, economics, and theoretical computer science build on mathematical models that, when formalized at all, are formalized once for a specific result rather than being assembled into widely-adopted, Mathlib-scale curated libraries. Pieces exist but they are scattered: Mathlib has Turing machines; Isabelle's Archive of Formal Proofs hosts a recent formalization of Nash equilibria for finite games, of the Cook–Levin theorem, of monotone-circuit lower bounds, and of the Chandy–Lamport snapshot algorithm; and Rocq has the deepest coverage of computational systems, with verified compilers, separation-logic frameworks, and cryptographic frameworks among others. But there is no standard shared library for, say, the LOCAL or CONGEST models, for population protocols, or for Lagrangian mechanics that a researcher can pick up and build on. In our case, we performed ad-hoc formalizations: each definition was written for this specific paper. This works once, but the resulting Lean code may not be reusable.

CSLib [4] is a recent initiative aiming to build a Lean library for computer science foundations: computational models, complexity analysis tools, verified algorithms and data structures, and deductive verification techniques. It targets, for computer science, the role Mathlib plays for pure mathematics.

Research communities are well-placed to drive this work themselves. Each TCS subcommunity has some standard models that its members already largely share, even if formalizing them surfaces choices that papers usually paper over. A shared formalization of those models, with families of interoperable variants

where conventions differ, is a natural service for a conference or a special-interest group to offer its authors, alongside proceedings and artifact evaluation. The hard part is not the technical work but the governance: who maintains the library, who arbitrates incompatible encodings, and how this labor earns credit in the usual academic accounting.

The same principle applies to any field with a mathematical backbone. What is needed is broad coverage: the standard models of computation, the standard models of physics, the standard models of economic theory, all formalized in a common framework so that results can interoperate. A small group of researchers, possibly aided by AI agents, could formalize the standard models from foundational textbooks and maintain them as an open library. This would be a contribution not just to formal verification but to the foundations of each field: the process of formalizing a definition forces you to resolve every ambiguity, and the resulting library becomes a reference that is precise in a way that no textbook can be. It would also have a pedagogical dimension, since students could trace any claim back to the axioms mechanically, turning the library into a teaching tool as well as a reference.

5 Looking Forward

Today, the proofs in our papers are checked by peer reviewers who read them, think about them, and try to convince themselves that they are correct. This process is valuable but imperfect. Published proofs are sometimes found to be wrong years or decades after the fact, and human proof-checking does not scale to the complexity of modern mathematics. Voevodsky, who discovered an error in a 1989 paper of his own, is a well-known case: he became one of the most prominent advocates for formal verification and devoted much of the latter part of his career to developing univalent foundations and the UniMath library [21].

Formal verification offers a complementary check. It does not replace peer review, since reviewers also evaluate significance, clarity, novelty, and the appropriateness of the model. But it can provide a machine-checked guarantee that the mathematical reasoning is correct, conditional on the formalization being faithful. Until recently, the cost of formalization made this impractical for most papers. AI agents have changed this.

Consider arXiv. It is already a public pool of papers with mathematical claims, and authors or referees can opt to put any of them through a formalization attempt. The `paper-badger` tool [19] is one concrete instance: given an arXiv identifier, it extracts the theorem statements and asks an AI agent to formalize them in Lean and Mathlib. Used as an opt-in diagnostic by authors, it will not catch every error—the formalization gap is real, and theorem statements often depend on macros, hidden

conventions, and definitions spread across the paper, so false negatives will be common. But the mere attempt at formalization would surface ambiguities in definitions and statements that English prose hides, and would catch some errors that currently slip through.

Or consider peer review. A conference or journal could assign, alongside its human reviewers, an AI reviewer whose sole job is to formalize the claimed results and check the proofs. This reviewer would not evaluate novelty, significance, or writing quality. It would answer one question: does the proof, as written, imply the claimed theorem in a formal sense? A report from such a reviewer would not be a verdict but a diagnostic. It might say: "Lemma 3 compiles as stated, but the proof of Theorem 1 uses a stronger induction hypothesis than the one stated in the paper." Human reviewers could then focus their energy on the parts of reviewing that require judgment, the parts that machines cannot do, rather than spending hours line-checking algebraic manipulations.

Beyond individual papers, the corpus of published mathematics is a large system running on trust: each paper assumes the correctness of the papers it cites, and the chain of trust extends back to the axioms. Occasionally a link breaks and the consequences propagate forward. A formally verified mathematical library would replace this chain of trust with a chain of proof. Mathlib already covers large parts of undergraduate and graduate mathematics. It is growing fast, and AI agents are accelerating the pace. It is not hard to imagine a future where the standard theorems of mathematics exist in a formal library, and new results are expected to build on that library rather than reprove things from scratch in English. This would be a shift comparable to the introduction of rigorous proof itself in the nineteenth century.

As AI-assisted formalization becomes routine, a finer taxonomy of mathematical knowledge will become visible. Some results are *formalized*: their statements and proofs exist in a proof assistant and type-check. Others are *easily formalizable*: their proofs are clear and detailed enough that an AI agent can formalize them with minimal human guidance, even if no one has bothered to do so yet. And then there are results that are *not obviously formalizable*.

This stratification has always existed, but formalization has been too expensive for it to be empirically observable at scale. As the cost drops, the boundaries will become visible, and they will tell us something about our own mathematical practice. A result that resists formalization is not necessarily wrong, but it is a result whose proof rests on conventions and intuitions that have not been made explicit. This is useful information. It points to places where our understanding is less complete than we thought, where the informal consensus may be hiding ambiguity or error. In the long run, the most valuable contribution of AI-assisted formalization may not be the proofs it verifies but the gaps it reveals.

This is not science fiction. The individual components exist today. What is

missing is infrastructure: the curated libraries of formal definitions, the integration with submission systems, and the community norms that would make this feel natural rather than imposed.

Natural-language proofs will continue to be valuable. The agents need the English proof as a guide, and readers benefit from understanding the proof, not just knowing that it is correct. A Lean term certifies that the reasoning holds, but it does not, by itself, convey how the author thought about the problem. The English proof carries the intuitions and analogies that let a reader spot the next variant, the next generalization, the next theorem. As the cost of formal verification drops, it can become a routine check rather than a special project, and natural-language proofs can keep doing what they are best at.

References

[1] Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development: Coq'Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. Springer, Heidelberg, 2004. doi:10.1007/978-3-662-07964-5.

[2] Alonzo Church. An unsolvable problem of elementary number theory. *American Journal of Mathematics*, 58(2):345–363, 1936. doi:10.2307/2371045.

[3] The Mathlib Community. Mathlib: a unified library of mathematics formalized in Lean 4. https://leanprover-community.github.io/mathlib4_docs/.

[4] CSLib. A focused effort on formalizing computer science in Lean, 2026. https://www.cslib.io/.

[5] N. G. de Bruijn. The mathematical language AUTOMATH, its usage, and some of its extensions. In M. Laudet, D. Lacombe, L. Nolin, and M. Schützenberger, editors, *Proceedings of the Symposium on Automatic Demonstration*, volume 125 of *Lecture Notes in Mathematics*, pages 29–61, Heidelberg, 1970. Springer. doi:10.1007/BFb0060623.

[6] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Proceedings of the 28th International Conference on Automated Deduction (CADE 2021)*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635, Cham, 2021. Springer. doi:10.1007/978-3-030-79876-5_37.

[7] Gottlob Frege. *Begriffsschrift, eine der arithmetischen nachgebildete Formelsprache des reinen Denkens*. Verlag von Louis Nebert, Halle, 1879.

[8] Matthias Függer and Thomas Nowak. Asymptotic subspace consensus in dynamic networks, 2026. arXiv:2602.19121 [cs.DC]. To appear in the Proceedings of the 5th Symposium on Algorithmic Foundations of Dynamic Networks (SAND 2026). doi:10.48550/arXiv.2602.19121.

[9] Matthias Függer and Thomas Nowak. A short demo of the two-agent formalization workflow, 2026. Video. https://youtu.be/rE5_Zc2AMwM.

[10] Matthias Függer and Thomas Nowak. Verified badges: a LaTeX package for proof-assistant badges, 2026. https://github.com/BioDisCo/verified-badges.

[11] Matthias Függer and Thomas Nowak. Vibe formalizing: an AI-assisted formal verification workflow, 2026. https://github.com/BioDisCo/vibe-formalizing.

[12] Gerhard Gentzen. Untersuchungen über das logische Schließen. I. *Mathematische Zeitschrift*, 39(1):176–210, 1935. doi:10.1007/BF01201353.

[13] Kurt Gödel. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik*, 38(1):173–198, 1931. doi:10.1007/BF01700692.

[14] Georges Gonthier. Formal proof—the four-color theorem. *Notices of the American Mathematical Society*, 55(11):1382–1393, 2008. https://www.ams.org/notices/200811/tx081101382p.pdf.

[15] Thomas Hales, Mark Adams, Gertrud Bauer, Tat Dat Dang, John Harrison, Le Truong Hoang, Cezary Kaliszyk, Victor Magron, Sean McLaughlin, Tat Thang Nguyen, Quang Truong Nguyen, Tobias Nipkow, Steven Obua, Joseph Pleso, Jason Rute, Alexey Solovyev, Thi Hoai An Ta, Nam Trung Tran, Thi Diep Trieu, Josef Urban, Ky Vu, and Roland Zumkeller. A formal proof of the Kepler conjecture. *Forum of Mathematics, Pi*, 5:e2, 2017. doi:10.1017/fmp.2017.1.

[16] David Hilbert. Die Grundlagen der Mathematik. *Abhandlungen aus dem Mathematischen Seminar der Hamburgischen Universität*, 6:65–85, 1928. doi:10.1007/BF02940602.

[17] Yann Leray and Théo Winterhalter. Encode the Cake and Eat It Too: Controlling Computation in Type Theory, Locally. *Proceedings of the ACM on Programming Languages*, 10(POPL):62:1–62:30, 2026. doi:10.1145/3776704.

[18] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer, Heidelberg, 2002. doi:10.1007/3-540-45949-9.

[19] Thomas Nowak. paper-badger: automated Lean 4 formalization and verification badges for arXiv papers, 2026. https://github.com/BioDisCo/paper-badger.

[20] Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1):230–265, 1936. doi:10.1112/plms/s2-42.1.230.

[21] Vladimir Voevodsky. The origins and motivations of univalent foundations, 2014. The Institute Letter, Institute for Advanced Study, Summer 2014. https://www.ias.edu/ideas/2014/voevodsky-origins.

[22] Alfred North Whitehead and Bertrand Russell. *Principia Mathematica*, volume 1. Cambridge University Press, Cambridge, 1910.

Disordered Reflections

Michel Raynal

Univ Rennes (IRISA, Inria, CNRS) France

Only that which has no history can be defined.
Friedrich Nietzsche.

Ideas that are in vogue often end up like dead leaves.
Anonymous.

The profusion of things hid the scarcity of ideas and the erosion of beliefs. (in *Les Années*).
Annie Erneaux (Nobel Prize 2022)

At a certain point in my career[1], I asked myself the very natural question, namely "What is my job as a university professor and researcher?" This question quickly branched out into more specific questions such as *What is teaching in a university?* and *What is research in a university?* As a professor and researcher in informatics, another question rose just as naturally:*What is informatics?* After a long period of reflection I define Informatics as the science which has algorithms at its core.[2].

This question directly implies another one, namely *What a university informatics curriculum should include?* These questions may seem naive, perhaps denoting a lack of maturity (or a late maturity...) of the author, but so whatever! Without answering all these questions, the following text offers some attempts to provide a few answers.

First, let us note that “teacher-researcher” is a compound word which, although not appearing in dictionaries, is more than just a juxtaposition of words. It also defines a state of mind that makes its two components difficult to separate. It is unfortunate that the profession of teacher-researcher is not associated with a specific indivisible word.

[1]This article finds its origin in an article that appeared in 2016 in volume 9 of the *Société Informatique de France* (SIF).

[2]My research area in on distributed algorithms, that I define as the *science of (formalized) cooperation among a set of independent computing entities*. While they address different contexts, distributed computing and parallel computing ares ometimes confused. Parallelism addresses efficiency of computations. Roughly speaking its aim is to look for data independence in order to process simultaneously batches of independent data (any problem solved by a parallel algorithm could be solved (of course very inefficiently) by a sequential algorithm.

Research and Universities

Research is one of the *raisons d'être of universites*, and this is independent from time and place. Research at universities is the gasoline of teaching. This constitutes the fundamental difference of teaching in universities and secondary schools. Research is an *adventure*, personal and collective, of an intellectual nature. By the word *adventure* I means that research is not defined by a predefined roadmap. Its goal is to deepen our understanding of the world around us and of the artifacts we create to acquire this knowledge. Moreover research is also a quest of a certain universality. We must never forget that words *universe*, *universality* and *university* have the same lexical root!

To achieve its goals research relies on curiosity, determination, wisdom, personal knowledge of its actors, as well as on serendipity (in the sense of an unexpected encounter between prior knowledge, state of mind, and a specific context). A large part of research consists of incremental advances[3]. Those advances form fertile ground from which breakthroughs emerge and then, taken together, enable paradigm shifts (In the sense given by Thomas S. Kuhn in his book *The Structure of Scientific Revolutions*). One of the difficulties of research is to provide this fertile ground without getting bogged down in it. Indeed, it was not by constantly improving the technology of candles that electricity was discovered, understood, and mastered. Research requires time and patience.

In many countries university professor positions are permanent positions (tenure) and this is not coincidence. It dates back to earliest Chinese mandarins: the permanence of their position guaranteed them the time for reflection rather than impulsive and hasty action necessary to maintain and govern the empire. The permanence of the position is a necessary condition for establishing a long-term perspective, without being (too) constrained by the hiccups and vagaries of the short term. "*Time to market*'' must not be the diktat, nor the criterion defining all research themes, nor the analytical grid presiding over their evaluation.

The sentence "*No one enters here unless he is a geometer*" engraved on the pediment of Plato's Academy sums up a state of mind specific to scientific research. There is certainly a continuum from research to its directly visible impact on society, and therefore from teacher-researcher to engineers. (I use here the word "engineer" in the industrial sense. There are, of course, "research engineers"[4] whose work is essential to research, and whose societal impact is not directly visible in products.) For the teacher-researcher (and unlike the engineer), applications are more important for the new questions they raise than for their direct impact on the economy.

The duality of teacher-researcher/engineer is similar to the duality of historian/journalist. The work of the former consists, in their chosen period of study, of analyzing and connecting events, and putting them into perspective, with the aim of providing us with a reasoned, comprehensive, and coherent view of a given period (past or contemporary). Depending on the schools of thought to which historians belong, it is possible to have different analyses and presentations. For citizens, this plurality is one of the guarantors of democracy. The work of the journalist is linked to current events and therefore to present facts. It consists essentially of informing us (and also sometimes taking a critical look at the facts presented) [5].

[3] Incremental is used here in the sense of not constituting a rupture, and does not imply a lack of creativity.

[4] Again, there is no specific word to designate a particular activity.

[5] It is striking that some periods with an enormous historical impact were very short-lived (e.g., –from a western

Like a historian, the work of a teacher-researcher in science consists of exploring and deepening scientific fields in such a way that not only do knowledge advances, but the fundamental and necessary aspects are also taught to students. Trained with the appropriate concepts and methods, each generation of students brings companies with new ideas, new methods, and new knowledge, such that after ten generations of students, these successive waves bring about real industrial revolutions in companies. This process of percolation constitutes the real and visible impact of universities on society, an impact that proves itself every day.

What is a Good Article?

PhD students sometimes ask the question: what is a good paper? What a question! Early in my career (when I was younger), my answer was largely syntactic. Among good articles, I counted those that had received awards granted only to papers published at least x years earlier (for instance, $x = 10$ for the Dijkstra Prize). Some of these articles have stood the test of time (a nontrivial achievement) and have often had a profound impact on their scientific communities.

Over time, however, my perspective shifted from syntax to semantics. Today, from my point of view, a good article is one that makes me slightly jealous that I did not write it myself (or I was not a co-author) of it! Of course, jealous is meant here in a friendly sense. Such an article is well written and develops ideas that we find interesting, sometimes even profound. This is the hallmark of good articles: readers make them their own, assimilate them, and pass on their essence to students.

An Anecdote on the Primacy of Results

Invited as a guest speaker in a conference a few years ago I chose to give a presentation on the *patterns* found in distributed algorithms. As an introductory example, before turning to distributed computing, I presented the well-known pattern in sequential computing proposed by William George Horner (1786–1837) for efficiently evaluating polynomials.

While browsing the web (thanks Wikipedia!), I discovered that this method had previously been proposed by the Italian philosopher and mathematician Paolo Ruffini (1765-1822). A more thorough bibliographic research led me to discover that Horner's method had already been used by the Chinese Zhu Shijie (1270-1330), who used the name *fan fa* to refer to it in his book titled *The Jade Mirror of the Four Unknowns* (1203). I also discovered that this book included a presentation of the so-called "Pascal's triangle" (1623-1662).

Books on the history of science provide many other examples. As a scientist humorously said, the value of a result could be measured by the number of times it has been rediscovered! Maybe we should teach not only science but the history of the science we are teaching. This will make students aware that science is not a mysterious artifact but a "product" of human beings.

---

point of view- the age of Pericles, the Italian Renaissance, or the French Revolution), all of which may be viewed as paradigm shifts.

And Innovation in all of This!

The widespread use of the word *innovation* in recent times seems to have turned it into a mantra, supposedly capable of warding off nearly all the problems generated by the junk economy.

Politicians who allocate research funding must justify to voters that public money is being well spent, which is entirely legitimate. Unfortunately, this often leads to a vision of research restricted to its most visible and immediately accessible aspects, those conveniently labeled *innovations*. And media amplification completes the picture. Unfortunately, this means that their vision of research is very often limited to its most visible and accessible (?) aspect to the general public[6] This is then amplified by media coverage, and that's all!

As university professors know from experience, the reality is of course much more subtle. There are many innovations that never leave the research laboratories. Furthermore, not all research necessarily leads to innovation that is visible to the general public, and some research results only find applications a long time after they have been discovered (lasers and touchscreen glass are interesting examples of this). Innovation most often comes from a (well-trained!) engineer or researcher who combines scientific expertise with a keen interest in working with technological tools.

> *A science created solely for the purpose of applications is impossible. Truths are only fruitful if they are linked together. If we focus only on those from which we expect an immediate result, the intermediate links will be missing and there will be no chain*. Henri Poincaré, in *Science and Hypothesis* (1902).

Teaching at a University

Like yin and yang, university research and teaching are inseparable. Research is the realm of uncertainty. We tackle a problem without knowing what the outcome will be, and once published, a result becomes part of the past, carefully preserved in DBLP and in one's CV [7]. To caricature, for a teacher-researcher, the most important problem is almost always the one she or he is trying to solve.

Teaching, by contrast, largely belongs to the realm of certainties. We teach established knowledge, adjusting it slightly from year to year depending on curricula and on what we ourselves have absorbed from recent research. Naturally, this sense of certainty erodes as one moves from undergraduate studies to the research-oriented master's level.

When we consider the teaching/research duality, teaching is almost always on the side of certainties. We teach known things, which we modify partially from one year to the next depending on the curriculum concerned and what we have ourselves assimilated from the research results relating to this curriculum. Of course, the notion of certainty deteriorates and eventually fades away as one progresses from the first years to higher education. [8].

[6] Today, words such as *blockchain* and *AI* often play this role. It is interesting to observe that the notion of "chain of blocks" used to build a total order on transactions was introduced in the early eighties in a failure-free context (this was named *distributed state machine replication)*. Moreover it is also worth noticing that the initial target application of blockchains was money transfer, an application that does not need a total order on all the money transfers.

[7] Unfortunately, so-called negative results are too often considered second-class citizens, despite their illuminating value.

[8] This is a far cry from multiple-choice questions...

Thinking about teaching and having a vision for it are things that cannot be improvised and are at the heart of the teaching and research profession. Teaching consists of stimulating students' intellectual curiosity and giving them the desire to learn (which may seem obvious but is far from easy...). The student's desire to learn is much more important than the teaching method used (regardless of any innovations associated with it).

Teaching requires reflection and vision, it cannot be improvised. As Leslie Lamport has pointed out, teaching cannot be reduced to an accumulation of facts or case studies. Henri Lebesgue (1875–1941) described university teaching as "thinking aloud in front of students." Teaching students how to reason, and how to adopt a scientific—and therefore critical—mindset, lies at the heart of higher education. Schoolchildren are not taught to read and write using washing-machine manuals (which are very used tools), but through the works of great authors. One might argue that this is inefficient, since few will become novelists. Yet reading beautiful texts enriches imagination and instills a sense of structure and flexibility—properties inherent to any language[9]. These experiences define asymptotes that elevate our own writing and thinking. Ultimately, teaching consists of stimulating students' intellectual curiosity and fostering their desire to learn —an objective that may seem obvious, but is far from easy. This desire is far more important than any particular teaching method, regardless of how innovative it may appear.

> "*For me, the first challenge for computing science is to discover how to maintain order in a finite, but very large, discrete universe that is intricately intertwined. And a second, but not less important challenge is how to mould what you have achieved in solving the first problem, into a teachable discipline: it does not suffice to hone your own intellect (that will join you in your grave), you must teach others how to hone theirs. The more you concentrate on these two challenges, the clearer you will see that they are only two sides of the same coin: teaching yourself is discovering what is teachable.*" Edsger W. Dijkstra, *My Hopes of Computing Science*, EWD 709 (1979).

[9] Never forget that *languages* are the most useful tools used by humans and even some animals.

Conclusion

Although immediacy, speed, and competition are seen in today's society as (market) values, it is important to note that we do not think any faster today than our ancestors were thinking twenty-five centuries ago, at a time where history, democracy (in a particular form), mathematics), algorithms, etc. were "invented". In a form or another there were universities a thousand years ago (e.g. Platon Academia and Confucius temples in China) and there will still be universities in a thousand years. We are aware that we belong to an old, large and great family. We feel at home in almost any university in the world, and we feel solidarity with our colleagues present and those who came before us, regardless of their beliefs and particular cultures. We are with them in a spirit of exchange and knowledge, not economic competitiveness.

A Vocabulary Remark

I always use the word *Informatics* and try not to use the word *Computer science*. This is due to the following observation made by many colleagues, namely informatics is not the science of computers, exactly as biology is not the science of microscopes and astronomy is not the science of telescopes. We do not have to confuse the tool and the target.

The case of *Computing science* is different. As advocated by Gottfried Wilhelm Leibniz, each concept must have a *one word* name and a definition (as done in any dictionary). So Informatics (name) is the science of algorithms (definition) and the word computing has to be understood as an algorithm at work.

ps.I want to warmly thank George Giakkoupis and Stefan Schmid for their constructive comments on a draft version of this article.

TCS ON THE WEB

BY

STEFAN NEUMANN

TU Wien
Erzherzog-Johann-Platz 1
1040 Vienna, Austria
stefan.neumann@tuwien.ac.at
https://neumannstefan.com

Decentralized Thoughts[1] is a group blog on distributed computing, blockchains, and cryptography. Ittai Abraham started it in May 2019, and it has since grown into a broad collection of course notes, research-oriented posts, and overview articles. The site also contains a "Start Here" page[2] with material for a graduate course on Blockchains, Distributed Computing, and Cryptography, a course page[3], and a video collection[4].

In this conversation, we do not only talk about the blog itself. We also discuss how a good blog can convey intuition and collect scattered knowledge, something that is much harder to do in a paper. Decentralized Thoughts is especially interesting because it sits between several communities—distributed computing, cryptography, systems, and blockchains—and also between theory and practice.

Today I am very happy to talk to four of the authors of Decentralized Thoughts: Ittai Abraham[5], Kartik Nayak[6], Ling Ren[7], and Alin Tomescu[8].

[1] https://decentralizedthoughts.github.io
[2] https://decentralizedthoughts.github.io/start-here/
[3] https://decentralizedthoughts.github.io/course/
[4] https://decentralizedthoughts.github.io/videos/
[5] https://a16zcrypto.com/team/ittai-abraham/
[6] https://users.cs.duke.edu/~kartik/
[7] https://sites.google.com/view/renling
[8] https://alinush.github.io

DECENTRALIZED THOUGHTS: BLOGGING CONSENSUS, BLOCKCHAINS, AND CRYPTOGRAPHY

A Conversation with Ittai Abraham, Kartik Nayak, Ling Ren, and Alin Tomescu

Ittai started Decentralized Thoughts in May 2019 on his own. After that, the blog soon started having multi-author posts on diverse topics. How has the blog changed over time and what are your motivations for blogging?

Ittai: I had been thinking about starting a blog for a long time, but I did not really have the courage to do it. This goes back to my time at Microsoft Research, where I was involved in pushing for what later became the *Windows on Theory* group blog[1]. Every year I would think, "now I will start a blog," and then I would not. What finally pushed me in 2019 was the feeling that, as the saying goes, someone was wrong on the internet. There were too many public discussions about blockchains that did not make sense to me from the point of view of distributed computing. So I started writing. That helped me overcome the hesitation of doing something outside the formal publication format.

The multi-author part was natural to me because I had the group-blog model in mind. We are in the same community, we collaborate, and people can contribute when they have something useful to explain. I sometimes think of it as a kind of proof of work: if you want to be part of the blog, you write posts. Some people contribute a lot for a while and then less; others join later. The blog changes with the people who put work into it.

Kartik: For me, part of the motivation was that consensus and blockchains are difficult to enter from any single community. The classic textbooks are very good, but they frame the area in a particular way. If you come from cryptography, PODC-style distributed computing, systems, or blockchains, people use different language and focus on different questions. When I started working in the area, many of my first insights came from conversations with people like Ling, Ittai, Dahlia, and Sasha, and from building my own mind map of the literature. The blog gradually became a way to put such a mind map in writing.

[1] `https://windowsontheory.org`

Ling: The field of consensus and blockchains is in dire need of good educational resources. The classic textbooks on distributed computing are amazing, but they cover a much broader field. And over the years, the terminology has evolved, and the focus areas have shifted. For someone who wants to learn consensus and blockchains today, the classic textbooks are no longer fully adequate. Decentralized Thoughts has become one of the leading resources for that purpose.

Alin: What I like about the format is that it lets us write down what researchers often know only by reading between the lines. There are many things people explain to each other in conversations, but they are not written down in papers. A blog post gives you a place to spell out that understanding directly.

Your site contains not only a blog but also a large amount of content, including a graduate course, cheat sheets, and videos. Who do you write for, and how do you hope people use the material?

Ittai: There is no single audience. It is a group blog, so different people write for different reasons. Sometimes I write course notes for students. Sometimes I write because there is a widely held view on the internet that I think is wrong or incomplete. Sometimes I want to talk about the philosophy of blockchains, and sometimes I want to explain a new paper. The posts that get the strongest feedback are often not the ones for five specialists, but the ones that give a broader, higher-level understanding of an area.

Alin: For me, the best reason to write is that something is interesting and under-explained. In cryptography, there may be many papers on a topic, but still no clear explanation of what is actually known, which assumptions matter, or how one would use the primitive securely. The first posts I wrote with Ittai were like that: the papers existed, but a newcomer still could not easily understand the whole story. A blog post is a chance to write down that missing understanding. Also, it is simply fun to write.

Kartik: A useful blog post can give the reader a mind map. In consensus, a protocol is a way to control a huge space of possible executions. Papers usually explain the contribution and the difference from prior work. A blog post can go bottom-up: what are the rules, why are they there, what goes wrong if we remove one of them, and how does this protocol relate to nearby ones? That kind of explanation is often what a reader needs in order to actually understand the area.

Ling: I hope the material helps people enter the field. It can be useful for a student, a researcher from a neighboring area, or a practitioner who wants to understand the assumptions and results that matter. We try to use terminology that unifies the classic consensus literature and recent advances in blockchains, and that makes it a practical entry point.

Several of you contrasted blog posts with papers. What can a blog do that a paper usually cannot?

Ittai: A paper is written under very specific incentives. In practice, the incentive is often to get accepted by three reviewers. That is not the same as teaching the reader what is important. A paper is not always written to explain what really happened or what the reader should remember. A blog post has a different goal: it can simply try to improve understanding. It can say, "here is the idea, here is why it matters, here is how it relates to other ideas."

A blog can also be updated. If you later find a better proof, a better explanation, or a useful external resource, you can edit the post and add links. That is much harder with a paper, and even a book update is a big event. A blog post can remain useful for years precisely because it can be improved.

Alin: Blogs are good at collecting scattered knowledge. For a primitive such as ECDSA[2], there may be many papers, but not one place that explains the scheme, public-key recovery, batch verification, malleability, why Bitcoin used it, why Ethereum used it, and what the practical tradeoffs are. A blog can be a focused survey of one idea or one primitive. It does not have to claim novelty; it can focus on clarity.

Kartik: That is important because academic writing often does the opposite. To get a paper accepted, you emphasize the difference from previous work. But when you want to understand an area, it is equally important to know when two things are really the same idea in different languages. A blog post can make those similarities explicit.

The blog also bridges theory and practice. How do you think about that gap?

Alin: On the cryptography side, I see a big gap between the research that is done in academia and what is actually done in practice. Academic papers and practical implementations have different concerns. In a paper, the Fiat–Shamir transform can look almost trivial: hash the transcript. In a real system, this is much more difficult. I see a big gap in terms of concerns and in terms of appreciation for how practical these ideas are when implemented—what is actually possible to do in two months in a team and what is impossible to do. There is no academic interest in how to write secure cryptography—to actually do it in practice where users' funds may be lost if you get the details wrong. I think the blog posts are a great opportunity to discuss some of that.

Ling: On this topic, I'd like to mention a recent article by Moshe Vardi that resonates with me a lot.[3] He argues that theoretical computer science should take

[2] https://en.wikipedia.org/wiki/Elliptic_Curve_Digital_Signature_Algorithm

[3] https://cacm.acm.org/opinion/what-is-theoretical-computer-science/

more inspiration from physics than from mathematics. In physics, there are beautiful theories, but only those that explain the real world are promoted, while those that do not are ultimately discarded. Much of the success of computer science rests on its relevance to practical systems. We should promote theories that better connect to the real world.

Kartik: In distributed computing, a protocol is trying to shepherd an enormous number of possible executions into a set of good states. It does that by imposing rules. When I read a paper, it is often not clear what each rule is doing, or what breaks if the rule is removed. But that is exactly the understanding one needs in practice. A blog post can spend the time to explain these failure modes and the intuition behind the rules.

Ittai: Practitioners often need the lay of the land. They may not want every technical detail of one paper; they want to know what the space looks like, which assumptions exist, which tools are simple, and where to look next. A blog can give a more neutral overview than a paper whose goal is to sell one particular contribution.

The blog often connects classic distributed computing with modern blockchains. Has the blockchain perspective changed which theoretical questions seem important?

Ling: Yes. Bitcoin proposed an ingenious and highly unconventional solution to consensus. It introduced features that traditional models overlooked: permissionless participation, participants becoming inactive and rejoining, and systems where one may not even know how many other participants are active. These features have opened many new theoretical directions in consensus.

Ittai: For me, blockchains also highlight questions around incentives, economics, and censorship resistance. These topics sit at the boundary of computing, game theory, and economics. I have been saying for years that I want more of that content on the blog, and I keep failing to do it. But I do think it is very important, and censorship resistance is already a good example of a question at that intersection.

Kartik: The blockchain perspective also forces us to compare models and assumptions more carefully. Results that look similar may be stated in different languages by different communities. Conversely, a small model difference may be the crucial issue. The blog helps us make those distinctions visible.

Alin: From the cryptography side, modern blockchain systems also expose the practical meaning of assumptions. A primitive is not only a theorem; it is something that must survive implementation choices, adversarial incentives, and real users. That changes which explanations are useful.

Has writing for Decentralized Thoughts changed your own research process?

Ittai: Writing for a wider audience forces clarity. I sometimes think of Timothy Gowers's essay[4] about the two cultures of mathematics: one more algebraic, building from axioms, and one more combinatorial, with islands of clever constructions. Writing posts has helped me move toward a more structured, almost algebraic view of distributed computing: a landscape of models, protocols, and lower bounds that are connected to each other. Sometimes seeing those connections suggests new theoretical questions.

Kartik: I would not say that writing one post immediately leads to one paper. But it changes how you understand the material. When you try to explain the mind map clearly, you notice missing pieces and start asking questions differently.

Alin: I have seen a blog post directly inspire research. In a project on publicly verifiable secret sharing, we needed a batched zero-knowledge range proof and were stuck. I remembered a short blog post by William Borgeaud about a simple range proof. It was not batched and not zero-knowledge, but after reading it again it became clear that the idea could be batched and adapted to our setting. That few-paragraph explanation helped us find the right direction.

What are you currently most excited about, in your own research and for the future of Decentralized Thoughts?

Kartik: For the blog, it is still on my agenda to write some chapter posts. We have many posts, but a new reader may not know in which order to read them. I would like to see more chapter posts or index posts: pages that gather several posts and turn them into a coherent path through a topic. That would also help with the courses we teach.

Ittai: Recently I spent time editing older posts on purpose, rather than only writing new ones, because some of the older posts are used a lot. That is one advantage of the blog. It can be archival, like a post on selfish mining that remains useful for years, and it can also be timely, almost like social media. When Simplex suddenly became important, for example, we could write about it quickly and relate it to Tendermint while the topic was active.

I would also like to see more material on incentives, economics, and censorship resistance. And I have a hidden goal of connecting more cryptography material to Decentralized Thoughts. I think in the future, people will start learning cryptography with post-quantum primitives. So I think there is an opportunity here to educate people about cryptography again.

Alin: I am very optimistic about high-quality technical writing online. It is searchable and open. If you search for a somewhat obscure topic, a clear blog post is

[4]Gowers, William Timothy. "The two cultures of mathematics." Mathematics: Frontiers and Perspectives 65 (2000): 65.

often much easier to find than the relevant PDF. In the LLM age, avoiding paywalls and writing knowledge in a form that is easy to index may become even more important.

Ling: The insights from the recent blockchain boom give us a lot of new research directions in distributed computing. As I mentioned before, Bitcoin has many interesting and attractive new features. But at the same time, it also has significant downsides, including long latency and energy inefficiency. Achieving the new features of Bitcoin without incurring its downsides has been one of my main research interests in recent years.

Finally, is there anything else you want our readers to know?

Alin: I would encourage young researchers to write. Start a blog, write about what you are learning, or explain your own research. Writing things down is a great way to see whether you really understand them and where the gaps are. It also creates connections with people who care about the same ideas.

Kartik: When you write a blog, the feedback is not always public. Often there are no comments and little visible reaction, but then someone writes or says that they know your work because they read a post. That is another good reason for researchers, especially young researchers, to make their ideas accessible.

Ittai: We are happy to receive high-quality guest posts. In that sense, Kartik and I often act as lightweight editors: someone with expertise sends a post, we help shape it, and the community gets a useful explanation. More broadly, attention is one of the scarce resources in research. Blog posts help focus attention on ideas, and researchers should practice communicating their ideas to a wider audience.

Ling: As much as I enjoy writing on Decentralized Thoughts, I do not think it can fully replace a good textbook. A blog can do much of the work of orientation, but a carefully written textbook is needed for a rigorous and complete understanding of the field. Writing a textbook is a daunting task, but I think I will try.

Thank you for this interview, Ittai, Kartik, Ling, and Alin!

THE LOGIC IN COMPUTER SCIENCE COLUMN

BY

ANUJ DAWAR

Department of Computer Science and Technology
University of Cambridge, Cambridge, CB3 0FD, UK
anuj.dawar@cl.cam.ac.uk

In this issue, we welcome back Yuri Gurevich, who is well known to the readers of this column. He is here with a contribution that is a further development in the realm of Abstract State Machines. As the regular readers of this column know very well, these are a formalization of the notion of an algorithm that can work at any level of abstraction. While the original formulation was for sequential deterministic algorithms, more recent developments by Yuri and his collaborators have included formalizations of nondeterministic and quantum algorithms. In the present version, an algorithm is seen as interacting with an external environment. This environment can be the source of nondeterminism or randomness. In the article presented here, Yuri develops the fomalization of the *inner* view of a deterministic algorithm which sees the input from the environment through oracles. I am excited to share this with you.

BEATCS no 149

ASSISTED COMPUTATIONS: INNER VIEW

Yuri Gurevich
University of Michigan, Ann Arbor, MI, USA

If people do not believe that mathematics is simple,
it is only because they do not realize how complicated life is.
— John von Neumann

1 Introduction

Starting in the 1930s, if not earlier, logicians were interested in "effective methods" of computation, which they eventually renamed "algorithms." By the 1960s, the intuitive notion of algorithm had been substantially clarified — see, for example, [12, §1.1]. To distinguish this notion from later generalizations, we call such algorithms *basic*.

The original Church-Turing thesis was about basic algorithms, which are deterministic and work in sequential time, i.e. proceed step by step. For a long time, the attempts to derive the thesis from first principles met with limited success. According to [7], called "the original study" below, there were two missing ingredients. One was a formalization of abstract state. The emergence of numerous levels of abstraction in software facilitated this formalization. The other ingredient was the bounded exploration principle formalizing Kolmogorov's observation that algorithms compute in steps of bounded complexity [11].

The main theorem of the original study was that three basic principles—sequential time, abstract state, and bounded exploration—imply that every basic algorithm is step-by-step simulated by a basic abstract state machine (ASM). The ASM model of computation was introduced earlier to provide an operational specification of software (see [6] for a standard exposition). The original Church-Turing thesis follows from the three principles plus the assumption that initial states contain only the minimum information necessary to compute a function [4].

This work was supported in part by the U.S. Army Research Office under Grant W911NF-20-1-0297.

I am indebted to Andreas Blass for careful reading of numerous drafts and for many insightful suggestions.

A natural question is whether this progress could be extended to nondeterminism. The answer is yes, and the core idea appears already in the original study [7, §9.1]. Algorithms are inherently deterministic[1] and need an environment to perform non-algorithmic tasks like flipping a coin. This idea was explored in depth in [2, 3]. Later we applied the idea to quantum computing [10], which was not covered in [2, 3].

Eventually we realized that it is beneficial for the algorithm to interact with partners that assist it (and may in turn be assisted by it). Mathematically, these partners/assistants are oracles. This, together with a few additional insights, substantially simplifies and strengthens the analysis of [2, 3]; in particular, it yields an explicit treatment of quantum circuit computations.

This perspective naturally leads to two complementary views of computation: the *inner view* (as seen by the algorithm itself) and the *outer view* (as seen by an external observer). The outer view is richer. An external observer sees how the interaction with partners develops. The outer view analysis was sketched in [8]. A journal version is in preparation [10].

The inner view is much simpler but clarifies many aspects nevertheless. Here we present the inner view, which is closer to the original study. In this view, oracle-managed functions are like other functions managed by the environment (as opposed to program variables managed by the algorithm).

2 Example algorithms

Example 1. A version of Euclid's algorithm for computing the greatest common divisor of two positive integers.

```
a:= Input1; b:= Input2;
until b=0 do (a := b || b := a mod b);
print "GCD(Input1,Input2)="a
```

Here Input1 and Input2 are positive integers provided by the user. The "until h do R" is equivalent to "while $\neg h$ do R" but avoids negating the termination (a.k.a. halting) condition. The sign || denotes parallel composition, which in this case is simultaneous assignment.

Example 2. A well-known ruler-and-compass algorithm known as the tangent algorithm. In a fixed Euclidean plane we are given a circle A, its center p, and a point q outside of A. The algorithm constructs a tangent to A through q.

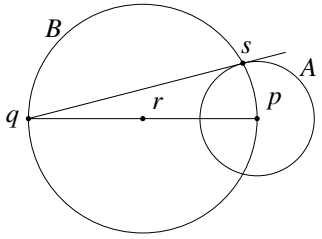
[1] Recall Yogi Berra's instruction: "When you come to a fork in the road, take it."

(1) Draw the midpoint r between p and q.

(2) Draw the r-centered circle B through q.

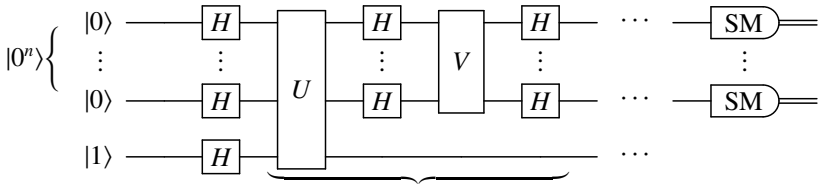
(3) Choose a point s where A and B intersect.

(4) Draw a line through q and s.



The tangent algorithm has no code for drawing or choosing. It delegates all that to the environment. It just orchestrates the process and tells the environment what to do and in what order.

Example 3. Grover's search algorithm [5], one of the most famous quantum circuit algorithms, is given by the following circuit diagram:



Our description of the algorithm involves quantum terminology; if it is unfamiliar to you, don't worry. The algorithmic part should be clear.

The circuit has $n + 1$ wires numbered from top to bottom. Time flows left to right. A state of the algorithm has two separate parts known as the classical and quantum states. The initial quantum state is $|0\rangle \otimes \cdots \otimes |0\rangle \otimes |1\rangle$.

The gates H, U, and V perform unitary transformations of the quantum state. Each SM gate performs a standard measurement and produces a classical bit: 0 or 1. Double lines indicate production of classical data.

3 Algorithm-apt structures

This is an auxiliary and somewhat technical section; but the notion of algorithm-apt structures is key in our analysis of computation.

Informally, a state of an algorithm is a complete description of the result of the computation performed so far. The subsequent computation of the algorithm is determined by the state and the environment.

Reality check 3.1. This is an idealization of course which is largely valid but frays on the margin. In real-world system engineering, in addition to syntax and semantics, there is also pragmatics. Resources are limited, and shortcuts between different levels of abstraction may be beneficial to efficiency. Think about terminal states for example. In many implementations, the algorithm doesn't "know" it has finished; it simply stops receiving CPU cycles. ◁

Formally, states may be viewed as structures in the sense of mathematical logic. The rich experience of mathematics and computer science strongly suggests that any static mathematical setting can be faithfully represented by a structure.

Q: It is often said that a state of a program is determined by the values of programming variables. Do you agree with that?

A: Conventional programming variables may not determine the state. For example, the recursion stack of a recursive program contains additional state information. Hence our more general notion of programming variables. ◁

We consider specialized vocabularies and structures. As far as vocabularies are concerned, we stipulate the following.

`Relations` In logic, vocabularies include relation symbols. In programming, a relation is a Boolean-valued function, and that is how we treat relations. But we mark the function symbols intended to be relations; in that sense we have relations in our vocabularies.

`Function labels` We use several kinds of basic functions in our structures, and it is convenient to give names to those kinds, used as labels for the corresponding function symbols.

`Types` We assign types to function symbols, albeit in a rudimentary way sufficient for our purposes.

Definition 3.2. An *algorithm-apt* vocabulary V is a finite set of *function symbols*, subject to the following requirements.

(1) Function symbols may be marked as *relation*.

(2) Every function symbol has one of the following five labels:
program variable, logic, type, plain, oracle.

(3) Each function symbol f is assigned a *function type*:

$$f : T_1 \times \cdots \times T_r \to T_0$$

where r is the *arity* of f, all T_i's are type symbols, symbols $T_1, \ldots, T_r$ are the *argument types* of f, and T_0 the *value type*. If f is a relation then T_0 is `Bool`. If f is nullary, we say that f is of type T_0.

(4) V contains several particular symbols.

- The type symbols `Any`, `Bool`, and `Integer`; all three are relations.
- The equality sign, which is a relation of type $\texttt{Any} \times \texttt{Any} \to \texttt{Bool}$, and the propositional connectives $\wedge$, $\vee$, $\to$ (binary), $\neg$ (unary), and `true`, `false` (nullary) whose argument and value types are `Bool`. All these are logic symbols.
- The symbols for the standard arithmetical operations $0, 1 :$ `Integer` and $+, -, \times, \div :$ $\texttt{Integer} \times \texttt{Integer} \to \texttt{Integer}$. ◁

Terms in a vocabulary V are defined recursively. If $f : T_1 \times \cdots \times T_r \to T_0$ and $t_1, \ldots, t_r$ are terms of type $T_1, \ldots, T_r$ respectively, then $f(t_1, \ldots, t_r)$ is a term of type T_0. Here the case $r = 0$ is the basis of recursion.

Recall that a relation R on a set S is represented by — and may be identified with — the set $\{s \in S : R(s) \text{ is true}\}$.

Definition 3.3. An *algorithm-apt structure* X of an algorithm-apt vocabulary V consists of the following components.

(1) A nonempty set $|X|$, the *base set* of X.

(2) The interpretations f_X of the V symbols, the *basic functions* of X. If $f : T_1 \times \cdots \times T_r \to T_0$ then f_X is a partial function from $(T_1)_X \times \cdots \times (T_r)_X$ to $(T_0)_X$. If f is a type symbol, it is convenient to view f_X as a subset of $|X|$, an *element type*, or just *type*.

It is required that conditions S1–S3 below are satisfied. ◁

The partiality of basic functions is necessary to deal with oracles. Given a query, an oracle may or may not respond. Also, the reply may be inappropriate and thus illegal.

If $f_X(\bar{x}) = \texttt{true}_X$, we say that $f_X(\bar{x})$ *holds* in X, symbolically $X \models f_X(\bar{x})$.

S1 $\texttt{true}_X$ and $\texttt{false}_X$ are distinct. The equality sign is interpreted as identity: If x_1, x_2 are elements of $|X|$, then $(x_1 =_X x_2)$ produces $\texttt{true}_X$ if x_1, x_2 are the same element, and $\texttt{false}_X$ otherwise.

S2 $\texttt{Any}_X = |X|$. $\texttt{Bool}_X = \{\texttt{true}_X, \texttt{false}_X\}$, $\texttt{Integer}_X$ is (a copy of) the integers, the propositional and arithmetical symbols are interpreted as expected.

S3 If $f : T_1 \times T_2 \times \cdots \times T_r \to T_0$ then f_X is a partial function from $(T_1)_X \times (T_2)_X \times \cdots \times (T_r)_X$ to $(T_0)_X$.

If T is a type then the elements of T_X are of type T. We do not assume that types are mutually exclusive, so an element may have several types.

Concerning the five labels, the intention is that programming variables are managed by the algorithm, whereas all other basic functions are managed by (possibly different parts of) the environment:

Basic functions	**Access mode for algorithms**	**Determinism required?**	**Examples**
Prog vars	Read, write	Yes	Variable arrays
Logic	Read	Yes	Prop connectives
Type	Read	Yes	`Bool`, `Integer`
Plain	Read	Yes	Arithmetic
Oracle	Read	No	User input

4 Sequential Time Postulate

We start with recalling the syntax and semantics of (well formed) terms, called *expressions* in programming.

Definition 4.1. Let V be a vocabulary and X a V structure. By mutual recursion, we define

(i) terms t of vocabulary V,
(ii) the (value) type of term t,
(iii) whether term t is defined in X, and
(iv) if yes, the value $[\![t]\!]_X$ of t in X.

If f is a function symbol in V of type $T_1 \times \cdots \times T_r \to T_0$ and $t_1, \ldots, t_r$ are terms of the types $T_1, \ldots, T_r$ then:

(1) $f(t_1, \ldots, t_r)$ is a *term*,

(2) its *type* is T_0,

(3) $f(t_1, \ldots, t_r)$ *is defined* in X if every t_i is defined in X and f_X is defined at $([\![t_1]\!]_X, \ldots, [\![t_r]\!]_X)$, and

(4) if $f(t_1, \ldots, t_r)$ is defined then its *value* $[\![f(t_1, \ldots, t_r)]\!]_X$ *in* X is $f_X([\![t_1]\!]_X, \ldots, [\![t_r]\!]_X)$ and it is of type T_0.

Here the case $r = 0$ is the basis of recursion. ◁

Remark 4.2 (Strict)**.** In programming terminology, clause (3) says that evaluation of terms is strict. This is a serious restriction — expression evaluation in programming is typically not strict. But the restriction is for a reason. Our functions are mathematical functions in a static structure, not procedures in a program. For illustration, suppose that f is a mathematical function of type $\texttt{Integer} \times \texttt{Integer} \to \texttt{Integer}$. If $f(0, y) = 0$ for all y, your algorithm may take advantage of it. But $\mathrm{Dom}(f)$ consists of pairs (x, y) of integers. If you want to invoke f, you have to provide a pair of integers.

By the way, in our approach, such f may be a single programming variable, a variable array (which may be viewed, if desired, as an array of conventional programming variables). ◁

Remark 4.3 (Ground)**.** Our terms are ground terms, which means that they do not involve variables. We view conventional programming variables as nullary function symbols. While their interpretations may vary from one state to another, they are constant in any given state. ◁

Postulate 1 (Sequential Time)**.** An algorithm $\mathcal{A}$ is given by the following components:

(1) An algorithm-apt vocabulary Voc and a set ImVal of Voc terms, including `true` and `false`.

(2) A nonempty set $\mathcal{S}$ of algorithm-apt Voc structures such that $[\![e]\!]_X = [\![e]\!]_Y$ for all e in ImVal and all $X, Y \in \mathcal{S}$.

(3) A nonempty subset of $\mathcal{I}$ of $\mathcal{S}$ and a subset $\mathcal{T}$ of $\mathcal{S}$.

(4) A partial map τ on the set $\mathcal{S}$ such that every $\tau(X)$ is an algorithm-apt V structure obtained from X by modifying (only) the interpretations of programming variables. $\mathrm{Dom}(\tau)$ contains no $\mathcal{T}$ states. ◁

Notation and terminology 4.4. The components (1)–(4) are the *basic components* of $\mathcal{A}$. Members of $\mathcal{S}$, $\mathcal{I}$, and $\mathcal{T}$ are *states, initial states*, and *terminal states* respectively. Members e of ImVal are *immediate-value* terms; their values are *immediate* values. The map τ is the *state transformer*. We allow modifications to be vacuous, so $\tau(X)$ may be X.

States in $\mathrm{Dom}(\tau)$ are *actionable*. A state is *normative* if it is actionable or terminal. We write $\mathrm{Voc}(\mathcal{A})$, $\mathrm{ImVal}(\mathcal{A})$, $\mathcal{S}(\mathcal{A})$, $\mathcal{I}(\mathcal{A})$, $\mathcal{T}(\mathcal{A})$, and $\tau(X, \mathcal{A})$ if the algorithm $\mathcal{A}$ is not clear from the context. ◁

Discussion 4.5 (Immediate values)**.**

Q: Immediate-values expressions? Are there any in the real world? In what sense are they immediate?

A: Yes, there are. The Booleans and small integers are typical examples. They are immediate in this sense: if you inspect an immediate value — think about it as a string, for simplicity — then you find the corresponding immediate-value expression immediately. Immediate values are trivial systematic variants of the corresponding expressions

Q: What about larger integers?

A: Larger integers are not immediate. Normally, there are only a few immediate-value terms. Their role will become clear in §6. ◁

Discussion 4.6 (The inner and outer views)**.**

Q: The transformer is single-valued over its domain. How is this possible in the presence of nondeterministic oracles?

A: We distinguish between the *inner view* of the computation (as seen by the algorithm itself) and the *outer view* (as seen by an external observer). The outer view is analyzed in [10]. Here we analyze the inner view.

Let us illustrate the two views on an example where Voc contains a plain function f and a single oracle function O, both of type `Integer` $\to$ `Integer`. Let a state X be in $\mathrm{Dom}(\tau)$.

We begin with the outer view. In the course of the computation, the algorithm may need values $O(x)$ for some integers x. Appropriate queries are issued. Depending on the replies, the algorithm may issue more queries, and so on. Eventually, $\tau(X)$ is produced.

The inner view is very different. The algorithm cannot distinguish between evaluating plain functions and evaluating oracle functions. Whether it evaluates $f(7)$ or $O(7)$, the requested value is provided by the environment. In reality, different parts of the environment may be responsible for computing $f_X(7)$ and $O_X(7)$. For example, $f_X(7)$ may be computed by the local operating system while $O_X(7)$ comes via the internet. But the algorithm is not aware of that. In both cases, the result miraculously appears.

In the next step, $O(7)$ may acquire a new value while $\tau(X)$ retains the old value because τ can change only programming variables. In the inner view, it is the environment that can modify the value of $O(7)$. Such modifications by the environment turn $\tau(X)$ into a successor state, and this happens after the current step of the algorithm is completed and before the following step starts. Nondeterminism enters only via such modifications by the environment. The transition function τ and, indeed, the whole algorithm is deterministic.

Q: It seems that the outer view is more expressive and useful. Why bother with the inner view?

A: Partly for expositional reasons. The inner view is simpler. Some subtle issues are easier in the inner view. But it is also important to understand algorithm's perspective. ◁

Definition 4.7. A *finite run* of the algorithm has the form

$$X_0, \tau(X_0), X_1, \tau(X_1), \ldots, \tau(X_{n-1}), X_n$$

where $X_0, \ldots, X_{n-1}$ are actionable states, and every X_{i+1} is produced from $\tau(X_i)$ by the environment, by modifying only the oracle functions. The run is *failing* if X_n is failing or if the environment fails to turn $\tau(X_n)$ into a state. ◁

5 Abstractness postulate

Postulate 2 (Abstractness)**.** The postulate consists of two clauses: Abstract State and Abstract Step below. ◁

In the Abstract State clause, we abstract from the implementation of states; isomorphic states are identical on the abstraction level of the algorithm In the Abstract Step clause, we abstract from the details of how the algorithm transforms a actionable state X into $\tau(X)$.

We interleave the clauses with auxiliary text. Each of the following two subsections is devoted to a single clause.

Abstract Step

Definition 5.1. An *(atomic) update* of a state X has the form $\langle f, (x_1, \ldots, x_r), x_0\rangle$ where f is a programming variable of some type $(T_1 \times \cdots T_r) \to T_0$, and every x_i is of type T_i. The intention is that the value of f_X at the argument tuple $(x_1, ..., x_r)$ is to be changed to x_0 in the structure $\tau(X)$.

The update is *trivial* if $x_0 = f_X(x_1, \ldots, x_r)$. Two updates of the same function f_X at the same arguments but with different values are *contradictory*. A set of updates of X, in short an *update set*, is *consistent* if it contains no contradictory updates. ◁

Definition 5.2. If Δ is a consistent update set of a state X, then $X + \Delta$ is the Voc structure with $|X + \Delta| = |X|$ and the basic functions

$$f_{X+\Delta}(\bar{x}) = \begin{cases} y & \text{if the update } \langle f, \bar{x}, y\rangle \in \Delta, \\ f_X(\bar{x}) & \text{otherwise.} \end{cases}$$

If Δ is inconsistent then $X + \Delta$ is undefined. ◁

Corollary 5.3. *(1) If f is a logic, type, plain, or oracle function symbol in Voc — i.e. not a programming variable — then $f_{X+\Delta} = f_X$.*

(2) $X + \Delta$ is algorithm-apt. ◁

Finally, we are ready to formulate the Abstract Step clause.

Abstract Step For every actionable state, the algorithm produces a consistent finite update set $\Delta(X)$ such that $\tau(X) = X + \Delta(X)$. ◁

We write $\Delta(X, \mathcal{A})$ when the algorithm $\mathcal{A}$ is not clear from the context.

Abstract State

Definition 5.4. An *isomorphism* $\mu : X \to Y$ of structures of the same vocabulary V is a bijection from $|X|$ to $|Y|$ that respects the basic functions, that is,

$$\mu(f_X(x_1, \ldots, x_r)) = f_Y(\mu x_1, \ldots, \mu x_r)$$

for every arity r and every r-ary function symbol f in the vocabulary.

In the case where V is Voc, μ is *immediacy-compliant* if it is the identity map on $\{[\![e]\!]_X : e \text{ is immediate-value}\}$. ◁

Lemma 5.5. *If $\mu : X \to Y$ is an isomorphism of structures of the same vocabulary V then $\mu([\![t]\!]_X) = [\![\mu(t)]\!]_Y$ for all V terms t.*

Proof. Induction on t. □

Definition 5.6. Let $\mu : X \to Y$ be a state isomorphism (i.e. an isomorphism of states). If $\Delta(X)$ is an update set of X then

$$\mu\langle f, (x_1, \ldots, x_r), x_0\rangle = \langle f, (\mu x_1, \ldots, \mu x_r), \mu x_0\rangle,$$
$$\mu(\Delta X) = \{\mu(u) : u \in \Delta(X)\}.$$ ◁

Lemma 5.7. *Every state isomorphism $\mu : X \to Y$ is immediacy-compliant.*

Proof. For every immediate-value term e,

$$\begin{aligned} \mu([\![e]\!]_X) &= [\![e]\!]_Y && \text{because } \mu \text{ is an isomorphism} \\ &= [\![e]\!]_X && \text{because } e \text{ is immediate-value.} \end{aligned}$$ □

Abstract State The sets of states, initial states, and terminal states are closed under immediacy-compliant isomorphisms. If $\mu : X \to Y$ is a state isomorphism then $\mu(\Delta(X)) = \Delta(Y)$. ◁

6 Bounded Exploration Postulate

Notation and terminology 6.1. If f and g are partial functions from a set A to a set B, we write $f(x) \simeq g(x)$ to mean that, for any argument a, the two values $f(a)$ and $g(a)$ are either both defined and equal or both undefined. The sign $\simeq$ is known as *Kleene equality*. ◁

Definition 6.2. Structures X, Y of the same vocabulary *V agree on a V term t* if $[\![t]\!]_X \simeq [\![t]\!]_Y$. They *agree on a set S of terms* if they agree on all terms in S. ◁

Definition 6.3. A set S of terms is *closed under negation* if, together with every unnegated Boolean term β, it contains $\neg\beta$. It is *closed under immediate equalities* if it contains every equality $s = e$ where $s \in S$ and e is an immediate-value term. S is *rounded* if it is closed under subterms, closed under negation, and under immediate equalities (and thus contains the immediate-value terms unless $S = \emptyset$). The *rounded version* $\overline{S}$ of S is the smallest rounded superset of S. ◁

Postulate 3 (Bounded Exploration)**.** There exist a finite, rounded set Tags of Voc($\mathcal{A}$) terms and an assignment, to every actionable state X, of a rounded *explore set* $\mathcal{E}(X) \subseteq \{t \in \text{Tags} : [\![t]\!]_X \text{ is defined}\}$ satisfying the following requirements where X, Y range over normative states.

Determine If X and Y have the same explore set and agree on it then either both states are terminal or both states are actionable and $\Delta(X) = \Delta(Y)$.

Discriminate There is a partial *exploration order* on $\mathcal{E}(X)$ such that if t is a tag in $\mathcal{E}(X) - \mathcal{E}(Y)$, then X, Y disagree on a Boolean term in $\mathcal{E}(X) \cap \mathcal{E}(Y)$ that precedes t in the exploration order for X. ◁

Corollary 6.4. *(1) If normative states X and Y agree on the Boolean terms in* $\mathcal{E}(X) \cap \mathcal{E}(Y)$ *then* $\mathcal{E}(X) = \mathcal{E}(Y)$.

(2) Isomorphic actionable states have the same explore set.

Proof. (1) follows from the Discriminate clause.

(2) Isomorphic states agree on Boolean terms. Apply (1). □

Definition 6.5. An (abstract basic) algorithm is an entity that satisfies Postulates 1–3. ◁

In the rest of the paper, $\mathcal{A}$ is a fixed algorithm. Further, Tags and sets $\mathcal{E}(X)$ are as in the postulate (relative to $\mathcal{A}$); elements of Tags are *tags* (for $\mathcal{A}$).

Commentary 6.6 (Intuition and intent)**.**

`Tags`. Consider an algorithm written in some programming language. Let S be the set of terms (expressions) that occur in the program of our algorithm, and T the rounded version of S. Think of tags as members of T. This presumes that all hidden arguments are made explicit.

The most important issue is the finiteness of the set of tags, but that part of the postulate was justified in [7]. Here we will address newer issues.

Let X, Y be actionable states.

`Explore sets`. The explore set $\mathcal{E}(X)$ of X is the set of tags that the algorithm evaluates in X. Since X is actionable, the tags in $\mathcal{E}(X)$ have well-defined values in X.

Explore sets were introduced in [1] to bolster the Bounded Exploration Postulate by requiring that, in every state X, the algorithm explores only terms t that are defined at X.

`Determine`. Consider an arbitrary moment during the algorithm's calculation of $\tau(X)$ from X, and let T be the set of terms evaluated so far. What terms, if any, will the algorithm evaluate next? The next action depends on the values $[\![t]\!]_X$ of tags t in T.

Importantly, it depends only on those values. For suppose that, at a certain moment in the calculation of $\tau(Y)$ from Y, exactly the tags in T have been evaluated with exactly the same results. Thus, the algorithm has not yet detected any difference between the two calculations. Accordingly, the next action will be the same.

If $\mathcal{E}(X) = \mathcal{E}(Y)$ and X, Y agree on the common explore set then the calculations never diverge, and so the two calculations look identical to the algorithm. Hence, $\Delta(X) = \Delta(Y)$.

`Discriminate`. The exploration order is the order in which the tags are evaluated in X in the process of computing $\Delta(X, \mathcal{A})$. Assume that $\mathcal{E}(X) - \mathcal{E}(Y) \neq \emptyset$ and $t \in \mathcal{E}(X) - \mathcal{E}(Y)$. Obviously, the calculations of $\tau(X)$ and $\tau(Y)$ have diverged at some point. Let's examine the situation at that point.

The two calculations have evaluated the same set T of tags and still pursue the same course of action in the sense that the tags to be evaluated next are exactly the same. Notice that it is not necessarily the case that $[\![t]\!]_X = [\![t]\!]_Y$ for all $t \in T$. For example, suppose that the state Y is obtained from X by replacing the elements $[\![t]\!]_X$, $t \in T$, with fresh elements. Since the nature of state elements is implementation dependent and not visible at the abstraction level of the algorithm, the algorithm would pursue the same course of action in both cases.

How can the two calculations diverge at this point in a way that leads to different explore sets? It is here that immediate values come into play. Suppose that, after evaluating the same set T of tags, the two calculations diverge and that, for simplicity, a single tag s is evaluated at this point and $[\![s]\!]_X \neq [\![s]\!]_Y$. We have $s \in \mathcal{E}(X) \cap \mathcal{E}(Y)$. If Y is obtained from X by replacing the elements $[\![u]\!]_X$, $u \in (T \cup \{s\})$, with fresh elements, one still can argue as above that the replacement is immaterial to the course of action. And this is correct, unless the value of s happens to be an immediate value, the value of some immediate-value term e, so that $[\![s]\!]_X = [\![e]\!]_X = [\![e]\!]_Y \neq [\![s]\!]_Y$. You can't get Y if the replacement by fresh elements involves immediate values.

Since the immediate-value terms are trivially reconstructible from their values, the algorithm discovers that the equality $s = e$ holds in X but not in Y. In that sense, it explores this equality, so it is a Boolean tag that belongs to $\mathcal{E}(X) \cap \mathcal{E}(Y)$. (If s happens to be Boolean, it is already a Boolean tag in $\mathcal{E}(X) \cap \mathcal{E}(Y)$ on which X and Y disagree.) ◁

Q: It seems to me that you use only the explore sets, not Tags. Can't you define Tags to be the union of the explore sets?

A: Yes, that would work, mathematically speaking.

Q: So why bother with Tags?

A: Suppose that we don't use Tags and just require that the union of the explore sets is finite. Then we need to add this additional requirement to Postulate 3, as we did in [1]. A question arises how to motivate the new requirement in this foundational study.

On the other hand, motivating the finiteness of Tags is natural and has been done already in the original study. In fact, we mentioned this motivation in the commentary above. Intuitively, if all programming variables are made explicit, then Tags is the rounded version of the set of expressions that actually occur in the program.

7 Explore sets

This is an auxiliary technical section. States are states of the fixed algorithm $\mathcal{A}$.

Definition 7.1 (Similarity)**.** (1) For a set S of normative states, $\mathcal{E}(S) = \bigcap \{\mathcal{E}(X) : X \in S\}$ and $\mathcal{B}(S)$ is the set of Boolean tags in $\mathcal{E}(S)$.

(2) Normative states X and Y are *similar*, symbolically $X \sim Y$, if they agree on $\mathcal{B}(\{X, Y\})$. If $X \sim Y$ then, by Corollary 6.4, $\mathcal{E}(X) = \mathcal{E}(Y)$. By the Determinate clause, both states are terminal or both are actionable and $\Delta(X) = \Delta(Y)$.

(3) $[X]$ is the similarity class of X, which is *actionable* (resp. *terminal*) if X is so.

(4) The conjunction $G_\sigma = \bigwedge \{\beta \in \mathcal{B}(X) : \ X \models \beta\}$ is the *guard* of σ.

Corollary 7.2. *If $Y \models G_{[X]}$ then $X \sim Y$.*

Corollary 7.3. *There are only finitely many similarity classes.*

Proof. Explore sets consist of tags, and the set of tags is finite. □

Definition 7.4. A set A of normative states is *agreeable* if they agree on $\mathcal{B}(A)$.

Lemma 7.5. *All members of an agreeable set A have the same explore set.*

Proof. Suppose, toward a contradiction, that despite A's agreeability, not all states in A have the same explore set. Then there are $X, Y \in A$ with $\mathcal{E}(X) - \mathcal{E}(A) \neq \emptyset$. Fix an exploration order $<$ on $\mathcal{E}(X)$ as in the Discriminate clause and choose a tag $t \in \mathcal{E}(X) - \mathcal{E}(A)$ that is minimal respect to $<$.

Since $t \in \mathcal{E}(X) - \mathcal{E}(A)$, there is some $Y \in A$ such that $t \in \mathcal{E}(X) - \mathcal{E}(Y)$. By the Discriminate clause, X and Y disagree on some $s < t$ in $\mathcal{B}(\{X, Y\})$. Recall that A is agreeable. If s were to belong to $\mathcal{E}(A)$, then all members of A, including X and Y, would agree on s. Hence $s \in \mathcal{E}(X) - \mathcal{E}(A)$, which contradicts the choice of t. □

Corollary 7.6. *If A is agreeable then all states of A are similar.*

This corollary will be called the **alternate Discriminate clause**.

Construction 7.7. We construct the *explore tree* ET whose nodes are sets of states closed under similarity. ET is ordered by inclusion. The root is the set $\mathcal{S}$ of all states.

If a node A is agreeable then, by the alternate Discriminate clause, all states in A are similar. Since A is closed under similarity, A is a similarity class. Similarity classes are the leaves of ET.

Suppose that a node A is not agreeable. Then the children of A are the maximal proper subsets B of A whose members agree on $\mathcal{B}(A)$. Obviously, each child B is closed under similarity. ◁

Along every branch of ET from the root to a leaf, nodes shrink but their explore sets grow. The intention is that the branches of ET reflect the paths that the algorithm's exploration takes in different states. In every state X, the algorithm explores $\mathcal{E}(\mathcal{S})$. If A is a non-leaf node in ET then, in any $X \in A$, the algorithm explores $\mathcal{E}(A)$ and then, depending on $\{[\![t]\!]_X : t \in \mathcal{B}(A)\}$, the algorithm explores $\mathcal{E}(B) - \mathcal{E}(A)$ for a child B of A.

Note that different nodes have different explore sets. Since all explore sets are subsets of the finite set Tags, the tree is finite.

Proposition 7.8. *Modify Postulate 3 by replacing the Discriminate clause with the alternate one. Then the original Discriminate clause is derivable.*

Proof. First, given an arbitrary state X, we construct an exploration order $<_X$ on $\mathcal{E}(X)$. Consider the branch in ET from the root to the leaf $[X]$, and define $s <_X t$ if the explore set of some node on this branch contains s but not t.

Second, we verify the original Discriminate clause. Suppose that $t \in (\mathcal{E}(X) - \mathcal{E}(Y))$ for some states X and Y. Then X and Y aren't similar and belong to different leaves of ET. Let A be the bifurcation node, so that A contains both X and Y but the two states belong to different children of A. Note that $t \notin \mathcal{E}(A)$ because $t \notin \mathcal{E}(Y)$. By the construction, there is a tag $s \in \mathcal{B}(A)$ such that $[\![s]\!]_X \neq [\![s]\!]_Y$. By the definition of the exploration order, $s <_X t$. □

8 ASM Rules

Let V be an algorithm-apt vocabulary. By default, terms are V terms and structures are algorithm-apt V structures. For each term t, let Sub(t) be the set of subterms of t, including t itself.

Definition 8.1 (Syntax)**.** We use induction to define *(ASM) rules* R of the fixed vocabulary V. The intention is that R is a one-step algorithm capable of working in any algorithm-apt structure.

- `skip` is a rule.
- An *assignment rule* has the form $f(t_1, \ldots, t_r) := t_0$
  where f is a programming variable, the terms $t_1, \ldots, t_r$ have the appropriate argument types for f, and the type of t_0 is the value type of f.
- A *parallel rule* has the form $R_1 \parallel R_2$
  where R_1, R_2 are rules.
- A *conditional rule* has the form `if` γ `then` R
  where γ is a Boolean term and R is a rule. ◁

Definition 8.2 (Status)**.** Let X be a structure. By induction on rule R we define whether X is normative for R.

- X is *normative* for `skip`.
- X is *normative* for an assignment $f(t_1, \ldots, t_r) := t_0$ if all $[\![t_i]\!]_X$ are defined.
- X is *normative* for $P \parallel Q$ if it is normative for both P and for Q and $\Delta(X, P) \cup \Delta(X, Q)$ is consistent.

- X is *normative* for a conditional $\texttt{if } \gamma \texttt{ then } Q$ if $[\![\gamma]\!]_X$ is defined and either $X \models \gamma$ and X is normative for Q or else $X \not\models \gamma$. ◁

Recall that $\overline{S}$ is the rounded version of a set S of terms.

Definition 8.3 (Exploration)**.** By induction on rule R we define the explore sets $\mathcal{E}(X, R)$ for R; here X is a normative structure for R.

- $\mathcal{E}(X, \texttt{skip}) = \overline{\varnothing} = \mathit{ImVal}$.
  (Rounding is superfluous here, but it is used everywhere to simplify exposition.)
- $\mathcal{E}(X, f(t_1, \ldots, t_r) := t_0) = \bigcup \left\{ \overline{\mathrm{Sub}(t_i)} : \ 0 \le i \le r \right\}$.
- $\mathcal{E}(X, P \parallel Q) = \mathcal{E}(X, P) \cup \mathcal{E}(X, Q)$.
- $\mathcal{E}(X, \texttt{if } \gamma \texttt{ then } Q) = \overline{\mathrm{Sub}(\gamma)} \cup \begin{cases} \mathcal{E}(X, Q) & \text{if } X \models \gamma, \\ \varnothing & \text{otherwise.} \end{cases}$ ◁

Definition 8.4 (Update sets)**.** By induction on rule R we define an update set $\Delta(X, R)$ for every structure X normative for R.

- $\Delta(X, \texttt{skip}) = \varnothing$.
- $\Delta(X, f(t_1, \ldots, t_r) := t_0) = \{\langle f, ([\![t_1]\!]_X, \ldots, [\![t_r]\!]_X), [\![t_0]\!]_X \rangle\}$.
- $\Delta(X, P \parallel Q) = \Delta(X, P) \cup \Delta(X, Q)$.
- $\Delta(X, \texttt{if } \gamma \texttt{ then } Q) = \begin{cases} \Delta(X, Q) & \text{if } X \models \gamma, \\ \varnothing & \text{otherwise.} \end{cases}$ ◁

Lemma 8.5. *The Determine clause holds for every rule R.*

Proof. Induction on R. □

Lemma 8.6. *The Discriminate clause holds for every rule R.*

Proof. By Proposition 7.8, it suffices to prove that, for every agreeable set S of structures normative for R, all states in S have the same explore set.

We prove the claim by induction on R. The cases of `skip` and assignment rules are trivial because any two normative structures have the same explore set.

Let $R = (P \parallel Q)$ and S a set of structures normative for R. that agree on the Boolean tags in

$$\begin{aligned} \mathcal{E}(S, P \parallel Q) &= \bigcap \{\mathcal{E}(X, P \parallel Q) : X \in S\} = \bigcap \{\mathcal{E}(X, P) \cup \mathcal{E}(X, Q) : X \in S\} \\ &\supseteq \bigcap \{\mathcal{E}(X, P) : X \in S\} = \mathcal{E}(S, P) \end{aligned}$$

By induction hypothesis, all S set have the same explore set relative to P. Similarly, they all have the same explore set relative to Q. It follows that they have the same explore set relative to R.

Let R be $\texttt{if}\ \gamma\ \texttt{then}\ Q$, $t \in \mathcal{E}(X,R) - \mathcal{E}(Y,R)$, and S be a set of structures normative for R that agree on the Boolean tags in

$$\mathcal{E}(S,R) = \bigcap \{\mathcal{E}(X, \texttt{if}\ \gamma\ \texttt{then}\ Q)\} = \bigcap \left\{ \mathrm{Sub}(\gamma) \cup \begin{cases} \mathcal{E}(X,Q) & \text{if } X \models \gamma, \\ \varnothing & \text{otherwise.} \end{cases} \right\}$$

In particular, they agree on γ. Two cases arise depending on they agree on γ being true or being false. Both cases are straightforward. Only one needs induction hypothesis. □

9 A generic ASM rule

In this section, we prove our Main Theorem which says essentially that every algorithm can be seen as iteration of a single ASM rule.

As before, $\mathcal{A}$ is a fixed algorithm. Throughout the section, by default, the vocabulary is $\mathrm{Voc}(\mathcal{A})$; in particular ASM rules are in the vocabulary $\mathrm{Voc}(\mathcal{A})$. Further, Tags and explore sets are as in Postulate 3, and states are those of $\mathcal{A}$.

Proposition 9.1. *For every similarity class σ of actionable states of $\mathcal{A}$, there is an ASM rule R_σ such that every $X \in \sigma$ is normative for R_σ, $\mathcal{E}(X, R_\sigma) \subseteq \mathcal{E}(X, \mathcal{A})$, and $\Delta(X, \mathcal{A}) = \Delta(X, R_\sigma)$.*

Proof. We start with an auxiliary construction which we call *point surgery*.

Point surgery. Given a structure X and a one-to-one correspondence $\mu_0 : S \to S'$ from a subset S of $|X|$ to a set S' disjoint from $|X|$, extend μ_0 to the map

$$\mu(x) = \begin{cases} \mu_0(x) & \text{if } x \in S, \\ x & \text{otherwise.} \end{cases}$$

from X to the structure X' of the same vocabulary obtained from X by replacing every $x \in S$ with $\mu_0(x)$. Every equality $f(x_1, \ldots, x_n) = x_0$ is preserved in the sense that it becomes $f(\mu x_1, \ldots, \mu x_n) = \mu x_0$ and no new such relationships arise. Thus, μ is an isomorphism. ◁

Lemma 9.2. *For every update $\langle f, (x_1, \ldots, x_r), x_0 \rangle$ in the update set $\Delta(X, \mathcal{A})$ of a actionable state S there are tags $t_0, \ldots, t_r$ in $\mathcal{E}(X)$ such that each $[\![t_i]\!]_X$ is x_i.*

Proof of Lemma. Suppose, toward a contradiction, that some x_i, say x_1, is not the value of any explored tag in X. Let y be an element outside of $|X|$. Use point surgery with $S = \{x_1\}$ and $S' = \{y\}$, and let μ and X' be as in point surgery above. Since $\mathcal{E}(X)$ contains all immediate terms, μ is immediacy compliant. By the Abstract State clause of Postulate 2, X' is a state. By Corollary 6.4(1), $\mathcal{E}(X) = \mathcal{E}(X')$. By the construction of X', μ is the identity of $\mathcal{E}(X)$. By the Determine clause of Postulate 3, $\Delta(X) = \Delta(X')$, so that $\langle f, (x_1, \ldots, x_r), x_0\rangle$ is in $\Delta(X')$ and therefore $x_1 \in |X'|$, which gives the desired contradiction. □

Lemma 9.3. *For every actionable state X of $\mathcal{A}$, there is an ASM rule R_X such that X is normative for R_X, $\mathcal{E}(X, R_X) \subseteq \mathcal{E}(X, \mathcal{A})$, and $\Delta(X, \mathcal{A}) = \Delta(X, R_X)$.*

Proof of Lemma. Let u be an update $\langle f, (x_1, \ldots, x_r), x_0\rangle$ in $\Delta(X, \mathcal{A})$. By Lemma 9.2, there are tags t_i such that every $[\![t_i]\!]_X = x_i$, so that every $[\![t_i]\!]_X$ is defined. Let Q_u be the assignment rule $f(t_1, \ldots, t_r) := t_0$. The desired R_X is $\{\!|Q_u : u \in \Delta(X, \mathcal{A})\}$.

By construction, all terms in R_X are defined in X. By Defintion 8.2, R_X is normative. By Lemma 9.2, $\mathcal{E}(X, R_X) \subseteq \mathcal{E}(X, \mathcal{A})$. By the construction of R_X, if X is actionable for $\mathcal{A}$, then it is so for R_X and $\Delta(X, R_X) = \Delta(X, \mathcal{A})$. □

Lemma 9.4. *Suppose that $\Delta(Z, R_X) = \Delta(Z, \mathcal{A})$ for some state Z isomorphic to a state Y actionable for $\mathcal{A}$. Then $\Delta(Y, R_X) = \Delta(Y, \mathcal{A})$.*

Proof of Lemma. It is easy to check that an isomorphism $\mu : Y \to Z$ transforms $\Delta(Y, \mathcal{A})$ to $\Delta(Z, \mathcal{A})$ and $\Delta(Y, R_X)$ to $\Delta(Z, R_X)$. Since $\Delta(Z, R_X) = \Delta(Z, \mathcal{A})$, we have $\mu\Delta(Y, R_X) = \mu\Delta(Y, \mathcal{A})$. It remains to apply μ^{-1} to the last equality. □

Now we prove claims (P1) and (P3). It suffices to find a state Z isomorphic to Y with $\Delta(Z, R_X) = \Delta(Z, \mathcal{A})$. First we consider the case where $|Y|$ is disjoint from $|X|$. Use point surgery, to construct a state Z isomorphic to Y by replacing $[\![t]\!]_Y$ with $[\![t]\!]_X$ for all t in $\mathcal{E}(Y)$. Since all immediates are explored, the states X, Z have the same explore set and agree on it. By the Determine clause of Postulate 3 — relative to $\mathcal{A}$ — $\Delta(X, \mathcal{A}) = \Delta(Z, \mathcal{A})$. Since R_X explores only terms in $\mathcal{E}(Z, \mathcal{A})$ in Z, we have that $\Delta(X, R_X) = \Delta(Z, R_X)$. By Lemma 9.3, $\Delta(X, R_X) = \Delta(X, \mathcal{A}) = \Delta(Z, \mathcal{A})$. It follows that $\Delta(Z, R_X) = \Delta(Z, \mathcal{A})$.

Second we consider the general case. Point surgery allows us to construct a state Z isomorphic to Y by replacing every non-immediate value in Y that belongs to X with a fresh element. Since Z is isomorphic to Y, it is similar to Y and therefore to X. Since $|Z|$ is disjoint from $|X|$, $\Delta(Z, R_X) = \Delta(Z, \mathcal{A})$ by the first part of proof. □

Definition 9.5. An ASM rule π is *generic* for algorithm $\mathcal{A}$ if every actionable state X of $\mathcal{A}$ is normative for π, $\mathcal{E}(X, \pi) \subseteq \mathcal{E}(X, \mathcal{A})$, and $\Delta(X, \mathcal{A}) = \Delta(X, \pi)$. ◁

Theorem 9.6 (Main Theorem)**.** *For every algorithm $\mathcal{A}$, there is a generic ASM rule R for $\mathcal{A}$.*

Proof. Let the exploration tree ET be as in §7. We write $A \prec B$ if B is a child of A. The following definition generalize Definition 7.1(4).

Definition 9.7. Let A be a node in ET and $X \in A$. A tag $g \in \mathcal{B}(A)$ is a *guardian* for A if $X \models g$. The conjunction of the guardians is the *guard* G_A of A. ◁

Unfortunately the guard may not belong to $\mathcal{E}(A)$, and we can't use it in contexts restricted to tags in $\mathcal{E}(A)$. To this end, we introduce the following abbreviation. If $G = g_1 \wedge g_2 \wedge \cdots \wedge g_n$ and R an ASM rule, then "`If` G_A `then` R" abbreviates

$$\texttt{if}\ g_1\ \texttt{then}\ \texttt{if}\ g_2\ \texttt{then}\ \cdots \texttt{if}\ g_n\ \texttt{then}\ R$$

By induction on ET, from leaves down to the root, we define an ASM rule R_A for each node A.

$$R_A = \begin{cases} \texttt{If}\ \ G_A\ \texttt{then}\ R_\sigma & \text{if } A = \{\sigma\} \text{ and } \sigma \text{ is actionable,} \\ \texttt{||}\ \{\texttt{If}\ G_A\ \texttt{then}\ R_B :\ A \prec B\} & \text{otherwise} \end{cases}$$

Lemma 9.8. *Let A be a node in ET, and X an $\mathcal{A}$-actionable state in A. Then X is normative for R, $\mathcal{E}(X, R_A) \subseteq \mathcal{E}(X, \mathcal{A})$, and $\Delta(X, \mathcal{A}) = \Delta(X, R)_A$.*

Proof of Lemma. Induction on ET. If A is a leaf, the claim is Proposition 9.1. Otherwise A is the union its children in ET, and so X belongs to a child B of A. Note that the guardians of A are also those of B,. By induction hypothesis, $\mathcal{E}(X, R_A) \subseteq \mathcal{E}(X, R_B) \subseteq \mathcal{E}(X, \mathcal{A})$ and $\Delta(X, R_A) = \Delta(X, R_B) = \Delta(X, \mathcal{A})$. □

The desired generic rule π is the rule R_S for the root of ET. □

10 Representation Theorem, and termination

ASM programs are composed from ASM rules. ASM rules are ASM programs, the standard loop constructs may be used to produce ASM programs, and the sequential composition of ASM programs is an ASM program. But, for the representation theorem, we need a special kind of ASM programs. In our context, a representation theorem says that arbitrary algorithms can be step-by-step simulated by appropriate ASMs. To this end, we need to address the termination issue: While algorithms may have terminal states, ASM rules of §8 do not.

Definition 10.1. A *termination aware ASM rule* is an ASM rule R, as in §8, endowed with a set of terminal states closed under the similarity for R. ◁

Our Main Theorem, Theorem 9.6, can be tightened by requiring that the terminal states of $\mathcal{A}$ are exactly the terminal states of π. Just endow the rule constructed in the proof with the set of terminal states of $\mathcal{A}$.

Definition 10.2. ○ An *iterate ASM program* has the form `iterate` R where R is an ASM rule. ◁

○ An *iterate ASM* is given by basic components Voc, ImVal, $\mathcal{S}$, $\mathcal{I}$, and $\mathcal{T}$, as in Postulate 1, and by an iterate ASM program `iterate` R; its state transformer is $\tau(X) = X + \Delta(R)$. ◁

It is routine to check that iterate ASM programs comply with Postulates 1–3.

Definition 10.3. Let $\mathcal{A}$ be an algorithm with basic components Voc, ImVal, $\mathcal{S}$, $\mathcal{I}$, $\mathcal{T}$, and τ, as in Postulate 1, and with explore sets $\mathcal{E}(X, \mathcal{A})$. An algorithm $\mathcal{B}$ with explore sets $\mathcal{E}(X, \mathcal{B})$ *step-by-step simulates* $\mathcal{A}$ if $\mathcal{A}$ and $\mathcal{B}$ have the same basic components, normative states, and terminal states, and, in addition, $\mathcal{E}(X, \mathcal{B}) \subseteq \mathcal{E}(X, \mathcal{A})$. ◁

Theorem 10.4 (Representation Theorem)**.** *For every algorithm $\mathcal{A}$ with explore sets $\mathcal{E}(X, \mathcal{A})$, there is an iterate ASM $\mathcal{B}$ with explore sets $\mathcal{E}(X, \mathcal{B})$ that step-by-step simulates $\mathcal{A}$.*

Proof. Let π be a (termination aware version of a) generic ASM rule for the algorithm $\mathcal{A}$. The desired $\mathcal{B}$ is the ASM with program `iterate` π with the basic components of $\mathcal{A}$, where $\tau(X, \mathcal{B}) = X + \Delta(X, \pi)$, and with explore sets $\mathcal{E}(X, \pi)$. □

Q: Let me understand "`iterate` R" better. R is repeatedly executed. Forever?

A: Not necessarily. You may arrive at a terminal state. Alternatively the environment may stop you.

Q: I see. Termination may be internal or external. Can you make the internal termination condition explicit, like in a while loop?

A: Yes, but .... Recall Definition 7.1, and let H be the set of terminating similarity classes. Then $h = \bigvee \{G_\sigma : \sigma \in H\}$ is a termination condition. A normative state X is terminal if and only if $X \models h$. And the program could be written in the form "`until` h `do` R."

Q: What about that "but"?

A: It should be possible to check termination conditions in every normative state. But the explore sets of some states may not contain h. Due to possible partiality of basic functions, this is a serious problem. Another problem is that h is may be too large to be practical.

11 Example ASM programs

Notation and terminology 11.1.

- `if` γ `then` R_1 `else` R_2 abbreviates
  `if` γ `then` R_1 $\parallel$ `if` $\neg(\gamma)$ `then` R_2.
- Semantically, the binary operation $R_1 \parallel R_2$ is commutative, is associative, and has a unit: `skip`. By analogy with sum and product operations, we write

$$R_1 \parallel R_2 \parallel \cdots \parallel R_k \quad \text{and} \quad \Big\|\{R_i\colon i \in I\}$$

  where I is a finite index set and $\Big\|\{R_i\colon i \in \varnothing\} =$ `skip`. ◁

Example 1, continuation

The program of Example 1 in §2 is already an ASM program where `a,b` are program variables, functions `mod` and `print` are plain, and `Input1, Input2` are oracles.

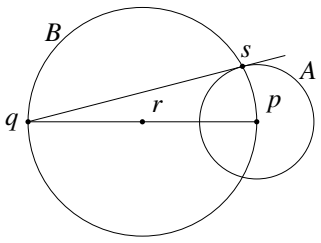
Example 2, continuation.

Introduce:

- Program variables r, s of type `point`, B of type `circle`, and T of type `line`.
- Deterministic functions $C(x, y)$, $L(x, y)$, and $M(x, y)$ such that if x, y are distinct points then $C(x, y)$ is the circle with center x through y, $L(x, y)$ the line through x and y, and $M(x, y)$ the midpoint between x and y.
- An oracle $I(A, B)$ which, given distinct intersecting circles A, B, chooses an intersection point.

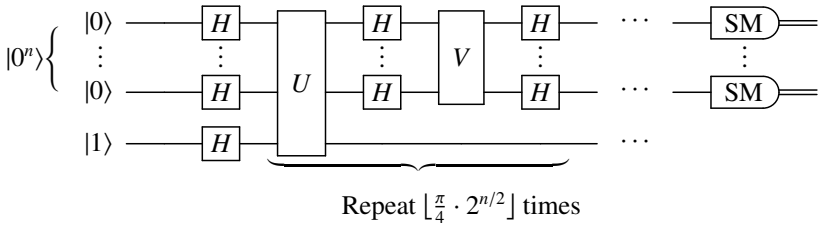
The desired ASM program:

$$\begin{aligned} r &:= M(p, q);\\ B &:= C(r, q);\\ s &:= I(A, B);\\ T &:= L(q, s) \end{aligned}$$



Here `circle`, `line`, and `point` are type symbols; B, r, s, and T are program variables; C and L are plain; and I, p, q are oracles.

Example 3, continuation. The diagram



specifies the algorithm but gives no indication how the gates H, U, V and SM work. In effect these gates are (operated by) oracles. Here is a corresponding ASM program:

$$\texttt{if } \bigwedge_{i=1}^{n+1} H_1(i) \texttt{ then skip;}$$

$$\texttt{repeat } \lfloor \tfrac{\pi}{4} \cdot 2^{n/2} \rfloor \texttt{ times}$$

$$\texttt{if } U(1,2,\ldots,n+1) \texttt{ then if } \bigwedge_{i=1}^{n} H_1(i) \texttt{ then}$$

$$\texttt{if } V(1,2,\ldots,n) \texttt{ then if } \bigwedge_{i=1}^{n} H_2(i) \texttt{ then skip;}$$

$$\bigparallel_{i=1}^{n} \texttt{Readout}(i) := \texttt{SM}(i)$$

Each gate transforms the quantum state. All the quantum work is done by oracles. The algorithm just tells them what to do. In describing the types of oracle functions, we suppress the quantum part. In the case of unitary transformations, the only classical part to report is that the work has been done. To this end, we introduce the one-element type {`true`}, called T below. The measurement gates produce classical bits which — to avoid introducing an additional type — may be treated as Boolean values; in programming, `Bool` is often identified with {0, 1}.

The first and second columns of H gates are operated by an oracle managing function H_1 : `Integer` $\to T$. Since the second and third columns of H gates are executed within the same step, the third column of H gates is operated by another oracle managing function H_2 : `Integer` $\to T$. The gates U, V, and SM are operated by oracles managing functions of types $(\texttt{Integer})^{n+1} \to T$, $(\texttt{Integer})^{n} \to T$, and `Integer` $\to$ `Bool` respectively.

The conditionals make the oracles do their job. In the final step, we make n calls to the measurement oracle and collect the results in the Readout function.

References

[1] A. Blass, N. Dershowitz, and Y. Gurevich, "Exact exploration and hanging algorithms," arXiv:2410.10706, ext. abstract in *Springer LNCS* 6247 (2010) 140–154

[2] A. Blass and Y. Gurevich, "Ordinary interactive small-step algorithms," Parts I,II, and III, *ACM ToCL* 7:2 (2006) 363–419 and 8:3 (2007) articles 15 and 16.

[3] A. Blass, Y. Gurevich, D. Rosenzweig, and B. Rossman, "Interactive small-step algorithms", Parts I and II, *LMCS* 3:4 (2007) papers 3 and 4

[4] N. Dershowitz and Y. Gurevich, "A natural axiomatization of computability and proof of Church's Thesis," *BSL* 14:3 (2008) 299–350

[5] L. Grover, "A fast quantum mechanical algorithm for database search," *STOC* 1996

[6] Y. Gurevich, "Evolving algebra 1993: Lipari guide," in E. Börger (ed), "Specification and Validation Methods," Oxford U. Press, 9–36 (1995), also arXiv:1808.06255

[7] Y. Gurevich, "Sequential abstract state machines capture sequential algorithms," *ACM ToCL* 1:1 (2000) 77–111

[8] Y. Gurevich, "The nature of nondeterministic (probabilistic, quantum, etc.) algorithms," YuriFest 2025, `sites.google.com/view/yurifest2025/home`

[9] Y. Gurevich and A. Blass, "Software science view on quantum circuit algorithms," *Information and Computation* 292 (2023), article 105024, arXiv:2209.13731

[10] Y. Gurevich and A. Blass, "Assisted computations: Outer view" [tentative title]

[11] A. Kolmogorov, "On the concept of algorithm," *Uspekhi Mat. Nauk* VIII 4(56) (1953) 175–176, abstract, skipped in *Russian Mathematical Surveys*

[12] H. Rogers, "Theory of recursive functions and effective computability," McGraw-Hill 1967

[13] A. Turing, "On computable numbers, with an application to the Entscheidungsproblem," *Proc. Lond. Math. Soc*, Ser. 2, vol. 42, parts 3 and 4 (1936), 230–265

The Education Column

BY

Dennis Komm and Thomas Zeume

ETH Zurich, Switzerland and Ruhr University Bochum, Germany

dennis.komm@inf.ethz.ch and thomas.zeume@rub.de

BEATCS no 149

Good In Theory

How Do LLMs Perform In a Theory Course?

Moritz Stocker
Department of Computer Science
ETH Zurich
moritz.stocker@inf.ethz.ch

Dennis Komm
Department of Computer Science
ETH Zurich
dennis.komm@inf.ethz.ch

Abstract

We study the performance of two large language models in an undergraduate second-year Theory of Computing course. The course contains two pen-and-paper exams that are each taken by around 450 students. We had both exams solved by two recent versions of GPT models, with the goal of comparing their scores to those of the students. To avoid any possible bias, we copied the solutions manually, and added them to the stack of exams that were graded by the unsuspecting teaching staff. Our results show that GPT-4o would not have passed either exam, while GPT-5 would not only have passed, but would actually have scored better than the majority of students.

1 Introduction

Following the introduction of ChatGPT to the general public in November 2022, one of the most pressing implications discussed early on was the potential impact of large language models (LLMs) on education [9]. This impact may come in many forms, e.g., with respect to utilizing LLMs and the like in classrooms or lecture halls, or diminishing (or at least re-evaluating) the meaningfulness of homework or take-home assignments. The debate is ongoing, with some researchers pointing out great opportunities for LLM-assisted teaching [13], while other voices are more skeptical [11, 12].

Whatever the concrete use case, one central underlying question is what LLMs are actually capable of doing, i.e., how well they perform in solving certain tasks. In this paper, we focus on undergraduate CS, and more specifically on the performance of two GPT-based models when solving exams of a second-year university course on the *Theory of Computing*. The course content is somewhat standard [8] and comprises *formal languages*, *computability* and *complexity theory*, with around 450 students attending each year.

LLMs are increasingly successful when solving problem statements from different domains—including CS—, and there is a rich body of literature on benchmarking different models, e.g., when solving problems from specific databases.

We follow a different approach. In particular, our goal was to assess LLM performance in a setting that is as close as possible to solving an exam under real-world conditions:

1. *Genuine exam.* We had two models (GPT-4o and GPT-5) work on two real exams that were part of the aforementioned undergraduate course.

2. *Blind grading.* The solutions were graded by teaching staff, who were unaware that two solutions (among the roughly 450) in each exam had not been created by humans.

3. *Lower bound.* We purposely did not train the models on old exams or the course material. We intentionally used two "vanilla" GPT versions without any kind of fine-tuning to determine what is possible in such a *zero-shot* setting.

One motivation was to investigate what an unprepared student having access to a generic LLM for a short period of time—say, during a bathroom break—could gain in an exam.

Concretely, we posed the following two research questions:

- **RQ1.** How well do LLMs, without any fine-tuning or special training, perform in *Theory of Computing* exams?

- **RQ2.** Are there specific types of tasks where LLMs outperform humans and vice versa?

Our results show that, unsurprisingly, GPT-5 consistently outperformed GPT-4o; and while the latter is "below the bar" in both exams, the former would actually receive roughly 80–90% of the points in each of them. Taking into account that some of the points were deducted due to GPT-5 using a notation different from that introduced in the course (without properly introducing it as part of its solution), this is indeed an impressive result.

2 Related Work

As mentioned above, the potential applications of LLMs in educational settings are numerous and subject to ongoing research. A very recent survey on LLMs in education is given by Shi et al. [20], drawing from 88 studies that consider different applications in different settings. Our research questions focus on pure performance in a very specific domain—and under conditions that are as close to a real-world setting as possible.

A great deal of early work on benchmarking LLM performance in education was—not very surprisingly—focused on programming courses and competitions. Pereira and Mello [18] provide a recent overview on how LLMs can assist educators in programming classes, e.g., with generating feedback, debugging code, or personalizing learning. The performance of tools like *GitHub Copilot* received considerable attention in the early days of the "GenAI hype" due to their abilities in solving, say, almost half of the problems stated in a CS1 course on the first try, as reported by Deny et al. [5]. Since then, programming benchmarking has shifted from solving simple tasks, e.g., from the *Mostly Basic Python Problems Dataset* [1] to more challenging problem statements, e.g., from platforms such as *LeetCode* [16]. *AetherCode* [24] is a particularly challenging benchmark, sourcing problems from prestigious competitive programming competitions, such as the *Olympiad in Informatics* and the *International Collegiate Programming Contest*.

While we also applied LLMs within the domain of CS, we considered problem statements from *Theory of Computing*, which lies at the intersection of CS and discrete mathematics. By now, there is a rich literature on the performance of LLMs on different math problems. These works often use established datasets such as *GSM8K* [4] or *MATH* [15]. Similar to programming benchmarks, one source of problem statements are math (and physics) olympiads, such as with *OlympiadBench* [14] or *OlymMATH* [22]. *Math Arena* evaluates the performance of different models "on the latest math competitions and olympiads" [2]. Zhang et al.[1] introduced "a curated dataset of 4,550 questions and their solutions spanning exams and assignments from all courses that form the curriculum for MIT's Mathematics and Electrical Engineering and Computer Science majors" [25]. Employing sophisticated prompt engineering, GPT-4 remarkably achieved "a perfect solve rate" [25] working on a test set (not containing image-based problem statements).

Our goal was to directly compare the performance of LLMs to that of humans, which is somewhat in the spirit of the original benchmark of all AI: the seminal *Turing test* [23]. As mentioned above, the results presented in this article were obtained under conditions that are as authentic as possible. In particular, GPT-

[1]The most recent version of the paper is withdrawn due to not receiving "permission to release the data or model fine-tuned on the data."

generated solutions were graded by TAs who were unaware of this fact—and who had no reason to believe that LLMs were involved in any way.

A similar approach was followed by Richards et al. [19], who received passing scores with GPT-generated solutions to undergraduate end-of-module assessments of different CS modules; the LLM performed much worse on postgraduate material. Staff graded the solutions as part of a "quality assurance" marking exercise and were given five GPT-generated solutions alongside ten student-generated ones. As in our setting, the markers were not aware that artificially generated solutions would be included in their workload. However, due to the much higher ratio of artificial solutions to student solutions, combined with the fact that the exams were solved remotely, the graders collectively flagged all artificial solutions as suspicious. In our case, no such suspicions were raised.

The setting explored by Ding et al. [6] is also rather close to ours; in their work, take-home assignments from an undergraduate *Algorithms* course were solved by two GPT-based models (GPT-4o and o1-preview), and these solutions were then blindly graded by the teaching staff. Problems required combinatorial proofs, such as computing the number of distinct spanning trees in a given complete graph. Their findings include that o1-preview consistently outperformed GPT-4o, and while the latter did not receive a passing score, the former did—even obtaining 92% of the points. Moreover, GPT-4o provided many more unjustified / misleading claims in its reasoning. One key difference from our work is that they used take-home assignments while we use on-campus exams.

Also related to our work is the research by Li et al. [17], which investigates how well then-current GPT models were able to solve exam-level CS tasks. Their set of questions was broader, but did include a number of problem statements regarding *Theory of Computing*. They examined in great detail the response quality of the models when provided with increasingly advanced hints. Their research does not include, however, a direct comparison to student results.

The work by Golesteanu et al. [10] and Dougherty et al. [7] is perhaps the most similar to our approach. The authors also analyzed the scores and error types of a GPT model [10], and compared the scores between two models, one of which was more advanced [7]. One major difference to our approach is that the points were assigned with the knowledge that the answers were provided by an LLM.

Chang et al. [3] provide an overview of how LLMs have been evaluated; their survey covers many more applications than education.

3 Methodology

Our lab teaches an undergraduate *Theory of Computing* course at ETH Zurich. This course is mandatory for second-year CS students and a popular elective for

second-year math students. It is taken by around 500 students each year. As part of this course, students may take a midterm and an end-of-term (final) exam;[2] both are pen-and-paper exams, manually graded by the teaching assistants (TAs) of the course. Each exam lasts 90 minutes, is taken on campus (supervised by the lecturers and TAs), and no auxiliary materials are permitted.

In the 2025 fall semester, we used two GPT-based models (*Azure GPT-4o* and *Azure GPT-5*) to complete two copies of these exams, both for the midterm and the final. We proceeded in three steps:

- *Step 1.* The tasks were supplied to the respective LLM exactly as stated on the exam, without providing the models with lecture notes, old exams, or sample solutions.
- *Step 2.* We then manually copied (i.e., wrote by hand) the answers onto regular exam paper and added these copies to the exams completed by the students for grading. The only change made to the model-provided answers was to convert the *deterministic finite automata*, which the LLMs gave as tables, into the expected graphical form. Due to standard procedure, all exams, including these copies, were anonymized before the TAs received them.
- *Step 3.* The TAs were not informed that LLM-generated responses were included among the exams and graded them as part of their regular duties. Each question was separately graded by a group of TAs; ensuring that each TA consistently graded the same task across all exams. The authors were not involved in the grading process of the LLM-generated solutions.

After the grading of the final was completed, the TAs were informed about the experiment and offered the opportunity to receive detailed information.

IRB approval was granted by the ethics board of ETH Zurich and can be provided on request.

3.1 Exam Questions

The problem statements in the two exams could be roughly divided into three categories: MULTIPLE-CHOICE, RECALL, and TRANSFER. In the MULTIPLE-CHOICE category (questions 2 and 6 in the midterm, questions 2 and 3a in the final), students received

[2]The grading scheme in use grades on a scale from 1 to 6 in steps of 0.25, with 4 being the first passing grade. In our course, the final grade of a student is the average of the grades of the midterm and final, rounded to a multiple of 0.25. There is a peculiarity: a student can discard the grade and instead take a so-called "session exam" that takes place after the end of the course and then gives the final grade.

1 point for each correct answer, no points for an incorrect answer, and 0.5 points if the question was left unanswered. As an example, consider question 2a in the midterm:

Midterm: Question 2a

For each of the following languages, decide whether it is regular or not.

1. $L_1 := \{w \in \{a,b\}^* \mid w = xx^{\mathsf{R}} \text{ for some } x \in \{a,b\}^*\}$
2. $L_2 := \{x \in \{0,1,2,3,4,5,6,7,8,9\}^* \mid x \text{ is the decimal representation of a number divisible by } 17\}$
3. $L_3 := \{a^n w \mid w \in \{a,b\}^*, n \in \mathbb{N} \text{ and } |w|_b = n\}$
4. $L_4 := \{a^n w \mid w \in \{a,b\}^*, n \in \mathbb{N} \setminus \{0\} \text{ and } |w|_b \geq n\}$

In the RECALL category, students were essentially asked to reproduce, in one way or another, information that they had seen explicitly in the lecture (question 5 in the midterm, questions 3b, 3c, and 4 in the final). As an example, consider question 3c in the final:

Final: Question 3c

Order the following complexity classes by their known inclusions: P, DLOG, EXPTIME, NPSPACE, NLOG, NP.

The TRANSFER category contained exercises where students had to apply techniques from the lecture to problems they had not encountered yet (questions 1, 3, and 4 in the midterm, questions 1, 5, and 6 in the final). As an example, consider question 4 in the midterm:

Midterm: Question 4

Show that any deterministic finite automaton that recognizes the language

$$L := \{a^n bw \mid n \in \mathbb{N}, w \in \{a,b\}^*, \ w \text{ contains neither the subword } aa \text{ nor the subword } bb\ \}$$

over the alphabet $\Sigma = \{a,b\}$ must have at least 5 states.

4 Results

Of the two models, GPT-5 was unsurprisingly more successful in both exams. Its answers received very high scores, though not resulting in a best grade, while the

Table 1: Number of students compared to which the GPT-models scored worse, equal to, or better in the two exams, as well as the final grade. The second row contains only the students that also participated in the final, allowing for a better comparison between the two exams. The same holds for the final grade. Note that the final grade is less fine-grained than individual points.

	GPT-4o			GPT-5		
	better	equal	worse	better	equal	worse
midterm	6	3	464	367	28	78
midterm*	0	2	427	324	28	77
final exam	64	7	358	335	9	85
final grade	5	10	414	345	51	33

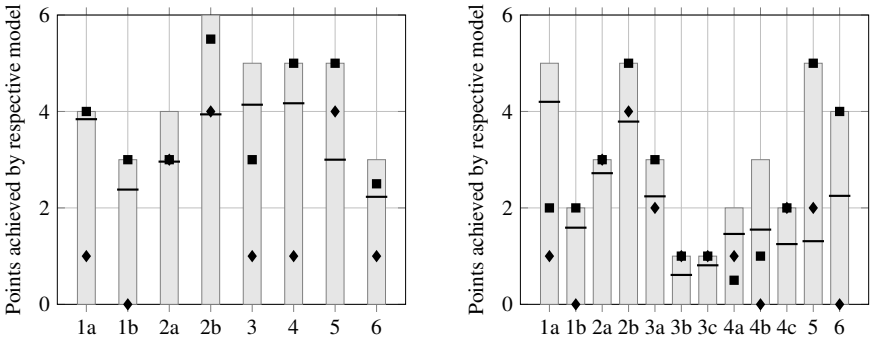
answers by GPT-4o did not receive a passing grade in either exam.

- In the midterm, 473 students participated. GPT-5 reached 31 out of 35 points, receiving more points than 78% of the students. GPT-4o received 15 points, beating only 1% of student scores.
- In the final, 429 students participated. GPT-5 reached 29.5 out of 36 points, again receiving more points than 78% of students. GPT-4o received 17 points, beating 15% of students.
- Over both exams, GPT-5 reached a grade of 5.5, beating 80% of students. GPT-4o received a grade of 3.25, once again beating only 1% of students.

The concrete numbers are shown in detail in Table 1. Comparing the relative performance of the two models in both exams is complicated by the fact that the students who chose not to take the final after participating in the midterm were usually those with weaker scores. If we only consider the students that also took part in the final, GPT-5 beat only 76% of student scores in the midterm, while no students performed worse than GPT-4o (row *midterm** in Table 1). Thus both models scored slightly better in the final than the midterm relative to the students. The individual points are shown in Figure 1.

4.1 Errors Made by Azure GPT-5

The more successful model GPT-5 reached 31 out of 35 points in the midterm, and 29.5 out of 36 points in the final. The points lost appear to fall into four broad categories. Note that these error categories are independent of the question categories described in Section 3.1.



(a) The six tasks of the midterm

(b) The six tasks of the final exam

Figure 1: The performance of both models for both exams. The gray bars indicate how many points were obtainable for the individual tasks. The horizontal line indicates the average number of points achieved by students; ♦: GPT-4o, ■: GPT-5

Background Knowledge. For many concepts covered in a *Theory of Computing* course, there are conflicting definitions. Usually, they are equivalent for all intents and purposes. However, to teach these concepts and argue about them, it is important to choose one such definition and stick with it.

In our course, we sometimes chose a definition that is not the one usually found in the literature. Since the GPT model was given the problems without direct access to the lecture material, its answers were unsurprisingly occasionally based on other definitions. As an example, consider questions 4a and 4b in the final:

Final: Question 4

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ be a 1-tape-TM that always halts with a space complexity of $s(n)$ for some function s with $s(n) \geq \log_2 n$.

(a) Describe the 4 components constituting an inner configuration of M.

(b) Show that there is a constant c such that the number $|\text{InConf}(n)|$ of possible inner configurations of M on instances of length n can be bounded from above by $c^{s(n)}$.

These two questions were meant to check knowledge of a specific proof technique shown in the course. In the course, a 1-tape-TM was defined as a multi-tape-TM (MTM) with one working tape and an additional separate read-only input tape. In other literature [21], an MTM is often defined in such a way that the input is given on the first working tape.

It was obvious that GPT-5 had based its answer on such a model and was therefore unable to answer satisfactorily. As a consequence, it received 0.5 out of 2 points for question 4a, and 1 out of 3 points for question 4b. A second example is question 1a in the final:

Final: Question 1

Consider the language

$$L_{\geq 3} = \{\text{Code}(M) \mid M \text{ is a TM that accepts at least 3 words}\}\,.$$

(a) Show that $L_{\geq 3} \leq_{\text{m}} L_{\text{U}}$ by providing an explicit reduction and proving that your reduction is correct.

Here, Code(M) refers to the encoding of a Turing machine (TM) and $\leq_{\text{m}}$ refers to a *many-one reduction*. The answer provided by GPT-5 contained a valid many-one reduction, but the wrong language L_{U}. While students were familiar with L_{U} denoting the *universal language* $\{\text{Code}(M)\#w \mid w \in L(M)\}$, GPT-5 apparently interpreted it as $\{\text{Code}(M) \mid L(M) \neq \emptyset\}$.

Mathematical Notation. We copied the output of GPT-5 as it was provided and tried not to correct mathematical notation based on our assumptions. Often, the output of formulae was given in a manner that could not be interpreted by the TAs as valid. As an example, consider question 3 from the midterm:

Midterm: Question 3

Show that the language

$$L := \{0^i 1^j 0^k \mid i, j, k \in \mathbb{N}, i \neq j \text{ or } i \neq k \text{ or } j \neq k\}$$

over the alphabet $\{0, 1\}$ is non-regular.

The answer provided by GPT-5 started from a language denoted by $U = 010^*$ [sic]. This was not standard notation from the course. The proof that followed was essentially correct under the interpretation of $U = \{0^i 1^j 0^k \mid i, j, k \in \mathbb{N}\}$. As the model did not define U explicitly as part of its solution, this was not judged as clear by the grading TA.

Unanswered Questions. As mentioned above, parts of both exams consisted of multiple-choice questions.

For two of these questions, a significant amount of time after asking them, the

connection to the server was lost and no answer was provided. We tried again in a new instance with the same result and decided to interpret this as no answer. The first of these questions was question 2b:

Midterm: Question 2b

Which of the following statements are *always* true (meaning they hold for all corresponding languages L), only *sometimes* true (meaning they hold for some languages, but not for all), or *never* true (meaning there are no languages, for which they hold)?

6. Let $L_1, L_2 \subseteq \{a, b\}^*$ be non-regular, then

   $$L = \{vw \mid v \in L_1, w \in L_2\}$$

   is also non-regular.

This statement is sometimes true and sometimes false, though it is slightly harder to find a pair of languages L_1 and L_2 such that it is false. The second such question was part of a set of simple true/false questions:

Midterm: Question 6

True or false:

2. Let M be a 1-tape-TM with $m \geq 4$ states. Then there exists an equivalent 2-tape-TM that only has the states q_0, q_{accept}, and q_{reject}.

The statement was meant to be true, though it may also hinge on the exact definition of a 1-tape-TM. Thus, this might also be a case of the first type of error discussed, namely *background knowledge*.

Question 2a-4 in the Midterm. This single question has to stand in a category of its own. It was again a simple true/false question, part of a series of questions where students were asked to decide whether a given language was regular or not. Students received one point for a correct answer, no points for an incorrect answer, and 0.5 points for a missing answer.

The question was undoubtedly the hardest question of the exam. Of the 473 students who participated in the midterm, only 58 answered it correctly; and a further 30 did not answer.

Midterm: Question 2a-4

Is the language L_4 regular?

$$L_4 := \{a^n w \mid w \in \{a,b\}^*, n \in \mathbb{N} \setminus \{0\} \text{ and } |w|_b \geq n\}$$

The language looks like a classic example of a non-regular language. However, it is actually regular, since there is no requirement that w has to start with b. In fact, it holds that $L_4 = \{aw \mid w \in \{a,b\}^*, |w|_b > 0\}$, which can be more easily recognized as regular. This was the only multiple-choice question in both the midterm and the final that GPT-5 answered incorrectly.

4.2 Errors Made by Azure GPT-4o

The errors made by GPT-4o are harder to classify than those made by GPT-5. It produced quite a large volume of material, much of which was judged by the grading TAs as more or less meaningless, echoing the findings of Ding et al. [6] that GPT-4o produces unjustified / misleading claims.

It is perhaps worth noting, however, that both questions for which GPT-5 provided no answer at all were answered incorrectly by GPT-4o. While GPT-4o did answer several other multiple-choice questions incorrectly, this may indicate that these questions were in fact difficult.

5 Discussion

Recall the two research questions we stated in Section 1: (RQ1) how well do LLMs, without any fine-tuning or special training, perform in *Theory of Computing* exams; and (RQ2) whether there are specific types of tasks where LLMs outperform humans and vice versa.

Regarding **RQ1**, the answer emerging from our results is that their performance is relatively good. Since the question asked about the possible performance, the models should be judged by the more successful of the two. This model, GPT-5, achieved a very good grade, though not the highest possible. As mentioned above, it received 31 out of 35 points in the midterm, and 29.5 of 36 points in the final; and therefore 85% of all available points—corresponding to a grade of 5.5 out of 6.

To answer **RQ2**, we use the question categories defined in Section 3.1: MULTIPLE-CHOICE, RECALL, and TRANSFER. We also refer to the specific points for individual questions shown in Figure 1.

In the MULTIPLE-CHOICE category (questions 2 and 6 in the midterm, questions 2 and 3a in the final), GPT-5 scored above average in all questions, especially in the final. In the midterm, it lost points in three of these questions, as discussed in

Section 4.1. GPT-4o was less consistent, scoring above average in some questions and below average in others.

In the RECALL category, for most questions, GPT-5 answered perfectly and GPT-4o answered above-average. The exceptions were questions 4a and 4b in the final, for which both models received below-average scores. These two errors fall into the *background knowledge* category discussed in Sections 4.1 and 4.2.

In the TRANSFER category (questions 1, 3, and 4 in the midterm, questions 1, 5, and 6 in the final), GPT-5 scored perfectly in four of the questions (questions 1 and 4 in the midterm, questions 5 and 6 in the final) and clearly below average in the remaining two (question 3 in the midterm, question 1 in the final). Of these two questions, its performance in question 1 in the final again appears to be due to missing *background knowledge* from the course as discussed in Section 4.1. GPT-4o performed very poorly in this category, reaching a slightly above-average score in only one question and scoring clearly below average in all others.

In summary, it appears that both models scored poorly relative to human students in questions where specific knowledge from the course was required (as opposed to general knowledge of the field). This is to be expected. In addition, GPT-4o struggled greatly with TRANSFER tasks, constructions, and proofs. However, GPT-5 in particular performed better than human students in MULTIPLE-CHOICE questions, though it did not answer perfectly.

This leads to a sobering conclusion: if the goal is to design a question that GPT-5 cannot answer as well as a student, the solution may not lie in the TRANSFER category—as might have been suspected some years ago—but in the *background knowledge* error category: questions can be made resistant to LLMs by asking for and building on information that the model simply does not have access to. However, this could presumably be circumvented by a malicious student with little effort.

6 Limitations

Our work has certain limitations. Most notably, our dataset consists of only 12 exercises (including sub-exercises). We would also like to emphasize once more that our goal was to establish a lower bound on LLM performance. As described in Section 4, it is reasonable to assume that many of the "mistakes" made by GPT-5 were due to its use of notation different from that introduced in the course (accounted for under the *background knowledge* category in Sections 4.1 and 4.2). A model specifically prompted with, or even trained on, the course material would presumably perform better.

It should be noted of course that many of the problem statements from both exams are somewhat standard, and variations surely appear in some training sets.

For instance, there are a few typical non-regular languages to which the *pumping lemma* is commonly applied. The corresponding non-regularity proofs thus usually follow a very similar structure and do not contain fundamentally novel techniques.

Due to the large number of students taking the exams, individual questions were typically graded not by a single person but by a small group of TAs. To maximize consistency, they worked closely together and discussed any edge cases among themselves and we have no reason to suspect any systematic differences between them.

While transcribing the answers of GPT-5, an error occurred. In question 4 of the midterm (see Section 3.1), we erroneously swapped two entries in a table, which resulted in an initial score of 4 out of 5 points. The error was discovered later. Since it was clear that the correct entries would have received full marks, we adjusted the score accordingly. We are confident that this is the only such error that happened in the entire process.

7 Conclusion and Future Work

We analyzed the performance of GPT-based LLMs in a *Theory of Computing* course under realistic conditions. Our results give us good reason to believe that our findings represent a rather loose lower bound, and that GPT-5 in particular would be able to obtain an excellent grade if fine-tuned.

In addition, the results obtained by GPT-based models may not be representative of the broader landscape of LLMs. It might be worth comparing the performance of other models such as Gemini or DeepSeek. Given the diverse landscape of LLMs, their respective strengths may also lie in different question types.

Even so, based on the results from this work, a deeper comparison between multiple-choice and open questions would be of interest. Since GPT-5 did not answer all multiple-choice questions correctly, specifically exploring this type of question relative to student performance might be interesting, especially given that open questions typically demand considerably more effort from students.

Acknowledgment

The authors would like to thank the teaching staff of the 2025 course *Theoretical Computer Science* at ETH Zurich and Hans-Joachim Böckenhauer for carefully proofreading the first draft of the paper.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. https://arxiv.org/abs/2108.07732

[2] Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. 2025. MathArena: Evaluating LLMs on Uncontaminated Math Competitions. In *Proceedings of the 39th Conference on Neural Information Processing Systems (NeurIPS 2025)*. 31 pages.

[3] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, Wei Ye, Yue Zhang, Yi Chang, Philip S. Yu, Qiang Yang, and Xing Xie. 2024. A Survey on Evaluation of Large Language Models. *ACM Transactions on Intelligent Systems and Technology* 15, 3, Article 39 (March 2024), 45 pages. https://doi.org/10.1145/3641289

[4] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *CoRR* abs/2110.14168 (2021).

[5] Paul Denny, Viraj Kumar, and Nasser Giacaman. 2023. Conversing with Copilot: Exploring Prompt Engineering for Solving CS1 Problems Using Natural Language. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2023)*. ACM, 1136–1142. https://doi.org/10.1145/3545945.3569823

[6] Ming Ding, Federico Soldà, Weixuan Yuan, and Rasmus Kyng. 2025. Assessing GPT Performance in a Proof-Based University-Level Course Under Blind Grading. *Bulletin of EATCS* 146 (2025).

[7] Ryan E. Dougherty, Matei A. Golesteanu, and Garrett B. Vowinkel. 2025. Large Language Models with Reasoning on Theory Course Exams. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 2 (ITiCSE 2025)*. ACM, 785. https://doi.org/10.1145/3724389.3730783

[8] Ryan E. Dougherty, Tim Randolph, Tzu-Yi Chen, Jeff Erickson, Matthew Ferland, Dennis Komm, Jonathan Liu, Timothy Ng, Ana Smaranda Sandu, Michael Shindler, Edward Talmage, and Thomas Zeume. 2024. A Survey of Undergraduate Theory of Computation Curricula in the United States. In *Working Group Reports on 1st ACM Virtual Global Computing Education Conference (SIGCSE Virtual-WGR 2024)*. ACM, 1–14. https://doi.org/10.1145/3708550.3730559

[9] James Finnie-Ansley, Paul Denny, Brett A. Becker, Andrew Luxton-Reilly, and James Prather. 2022. The Robots Are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming. In *Proceedings of the 24th Australasian Computing Education Conference (ACE 2022)*. ACM, 10–19. https://doi.org/10.1145/3511861.3511863

[10] Matei A. Golesteanu, Garrett B. Vowinkel, and Ryan E. Dougherty. 2024. Can ChatGPT Pass a Theory of Computing Course?. In *Proceedings of the 2024 ACM Virtual Global Computing Education Conference V. 1 (SIGCSE Virtual 2024)*. ACM. https://doi.org/10.1145/3649165.3690116

[11] Carolin Hahnel, Jamie Gorson Benario, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Dennis Komm, Colleen Lewis, Dominic Lohr, Brent Reeves, Jaromír Šavelka, Jacqueline Staub, and Christina Weers. 2026. GenAI in Programming Education: Hypes, Hoaxes, and Hopes. In *Generative AI in Programming Education (Dagstuhl Seminar 25311)*, Michelle Craig, Paul Denny, Natalie Kiesler, and James Prather (Eds.). Vol. 15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 253–279. https://doi.org/10.4230/DagRep.15.7.253

[12] Emma Harvey, Allison Koenecke, and Rene F. Kizilcec. 2025. Don't Forget the Teachers: Towards an Educator-Centered Understanding of Harms from Large Language Models in Education. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI 2025)*. ACM, Article 1064, 19 pages. https://doi.org/10.1145/3706598.3713210

[13] Orit Hazzan and Yael Erez. 2025. Rethinking Computer Science Education in the Age of GenAI. *ACM Transactions on Computing Education* 25, 3, Article 26 (June 2025), 9 pages. https://doi.org/10.1145/3732792

[14] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. 2024. OlympiadBench: A Challenging Benchmark for Promoting AGI with Olympiad-Level Bilingual Multimodal Scientific Problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, Volume 1: Long Papers (ACL 2024)*. ACL, 3828–3850. https://doi.org/10.18653/v1/2024.acl-long.211

[15] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks*, Joaquin Vanschoren and Sai-Kit Yeung (Eds.). https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/be83ab3ecd0db773eb2dc1b0a17836a1-Abstract-round2.html

[16] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *The 13th International Conference on Learning Representations*. https://openreview.net/forum?id=chfJJYC3iL

[17] Nero Li, Shahar Broner, Yubin Kim, Katrina Mizuo, Elijah Sauder, Claire A. To, Albert Wang, Ofek Gila, and Michael Shindler. 2025. Investigating the Capabilities of Generative AI in Solving Data Structures, Algorithms, and Computability Problems. In *Proceedings of the 56th ACM Technical Symposium on*

Computer Science Education V. 1 (SIGCSE 2025). ACM, 659–665. https://doi.org/10.1145/3641554.3701946

[18] Andre Fabiano Pereira and Rafael Ferreira Mello. 2025. A Systematic Literature Review on Large Language Models Applications in Computer Programming Teaching Evaluation Process. *IEEE Access* 13 (2025), 113449–113460. https://api.semanticscholar.org/CorpusID:279775486

[19] Mike Richards, Kevin Waugh, Mark Slaymaker, Marian Petre, John Woodthorpe, and Daniel Gooch. 2024. Bob or Bot: Exploring ChatGPT's Answers to University Computer Science Assessment. *ACM Transactions on Computing Education* 24, 1, Article 5 (Jan. 2024), 32 pages. https://doi.org/10.1145/3633287

[20] Yuhong Shi, Kun Yu, Yifei Dong, and Fang Chen. 2026. Large language models in education: a systematic review of empirical applications, benefits, and challenges. *Computers and Education: Artificial Intelligence* 10 (2026), 100529. https://doi.org/10.1016/j.caeai.2025.100529

[21] Michael Sipser. 2013. *Introduction to the Theory of Computation* (third ed.). Cengage Learning.

[22] Haoxiang Sun, Yingqian Min, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, Zheng Liu, Zhongyuan Wang, and Ji-Rong Wen. 2025. Challenging the Boundaries of Reasoning: An Olympiad-Level Math Benchmark for Large Language Models.

[23] Alan M. Turing. 1950. Computing Machinery And Intelligence. *Mind* LIX, 236 (10 1950), 433–460. https://doi.org/10.1093/mind/LIX.236.433

[24] Zihan Wang, Jiaze Chen, Zhicheng Liu, Markus Mak, Yidi Du, Geonsik Moon, Luoqi Xu, Aaron Tua, Kunshuo Peng, Jiayi Lu, Mingfei Xia, Boqian Zou, Chenyang Ran, Guang Tian, Shoutai Zhu, Yeheng Duan, Zhenghui Kang, Zhenxing Lin, Shangshu Li, Qiang Luo, Qingshen Long, Zhiyong Chen, Yihan Xiao, Yurong Wu, Daoguang Zan, Yuyi Fu, Mingxuan Wang, and Ming Ding. 2025. AetherCode: Evaluating LLMs' Ability to Win In Premier Programming Competitions. https://arxiv.org/abs/2508.16402

[25] Sarah J. Zhang, Samuel Florin, Ariel N. Lee, Eamon Niknafs, Andrei Marginean, Annie Wang, Keith Tyser, Zad Chin, Yann Hicke, Nikhil Singh, Madeleine Udell, Yoon Kim, Tonio Buonassisi, Armando Solar-Lezama, and Iddo Drori. 2023. Exploring the MIT Mathematics and EECS Curriculum Using Large Language Models. https://arxiv.org/abs/2306.08997

The Concurrency Column

by

Marino Miculan and Nobuko Yoshida

Department of Mathematics, Computer Science and Physics
University of Udine
Via delle Scienze 206, Udine, Italy
marino.miculan@uniud.it

Department of Computer Science
University of Oxford
Wolfson Building, Parks Road, Oxford, UK
nobuko.yoshida@cs.ox.ac.uk

Welcome to this installment of *The Concurrency Column*. In this issue, we are pleased to present two outstanding contributions that significantly advance the formal verification and behavioral safety of message-passing concurrent systems through global protocol models.

The first contribution is by Elaine Li, summarizing the foundational theoretical and practical breakthroughs of her doctoral thesis, "Decision Problems for Global Protocol Specifications." As message-passing concurrency forms the bedrock of modern distributed systems, verifying communication protocols remains a critical challenge. High-level global protocol specifications (such as Message Sequence Charts, session types, and choreographies) offer a powerful top-down paradigm by describing interactions from a synchronous, bird's-eye view, ruling out major coordination errors by construction. However, implementing these specifications across asynchronous distributed networks introduces deep algorithmic complexities. In this article, Li addresses three central problems in global protocol verification: implementability, synthesis, and subtyping. Rather than treating these challenges in isolation, she provides a unifying semantic lens rooted in formal language and automata theory. This foundational approach yields precise, sound, and complete characterizations, highlighted by the formulation of "Coherence Conditions." No-

tably, these results are fully mechanized in the Rocq interactive theorem prover and implemented practically within the verification tool SPROUT.

The second contribution, “Typing Scalas-Yoshida Benchmarks” by Franco Barbanera, Mariangiola Dezani-Ciancaglini, and Ugo de’ Liguoro, explores the practical expressive power of modern session type frameworks. The authors demonstrate how the challenging Scalas-Yoshida "Less Is More" benchmark protocols—including service-client authentication and recursive Map/Reduce—can be successfully represented and typed within the Simple Multiparty Sessions (SMPS) framework. By reconstructing suitable global types, they show that SMPS can seamlessly type protocols that are typically problematic or unprojectable in standard multiparty session type settings, all while firmly retaining critical behavioral guarantees like session fidelity and lock-freedom.

Together, these two papers provide an essential reference for researchers and practitioners designing robust, typesafe concurrent frameworks.

Decision Problems for Global Protocol Specifications

Elaine Li

Abstract

Concurrency is ubiquitous in modern computing, message passing is a major concurrency paradigm, and communication protocols are therefore a key target for formal verification. Global protocol specifications synchronously describe the message-passing behaviors of all protocol participants from a bird's-eye view, and thus rule out large classes of communication errors by construction. Global protocols are adopted in industry by the ITU standard and UML, and are widely studied in academia in the form of high-level message sequence charts, session types and choreographic programs. This paper summarizes theoretical and practical contributions from the author's Ph.D. thesis to three problems central to global protocol verification: implementability, synthesis, and subtyping. Implementability asks whether a protocol admits a distributed implementation, synthesis in turn computes one, and subtyping asks whether an admissible implementation can be substituted in whole or part to yield fewer behaviors. This paper situates these results in relation to one another, through the unifying semantic lens of formal language and automata theory.

1 Introduction

Concurrency is ubiquitous in modern computing, message passing is a major concurrency paradigm, and communication protocols are therefore a prime target for formal verification. Writing implementations for each protocol participant individually, such that their composition is free from communication errors and deadlocks, is challenging and error-prone. One salient methodology for verifying communication protocols centers around the abstraction of a *global protocol*. Global protocols specify the message-passing behaviors of all protocol participants synchronously, and from a bird's eye view, thus ruling out large classes of communication errors by construction. Global protocols are then used as a blueprint from which to synthesize correct-by-construction distributed implementations. Due to their attractive simplicity, global protocols are widely studied in academia, predominantly in the

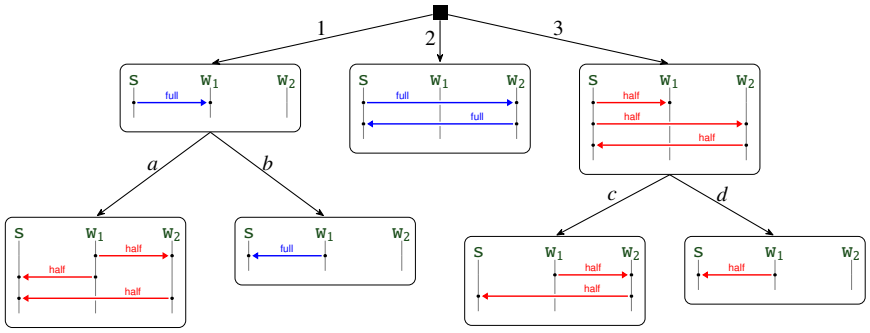


Figure 1: Task scheduling protocol.

form of high-level message sequence charts (HMSCs) [52], multiparty session types (MSTs) [29], and choreographic programs [13]. Message sequence charts represent global protocols using a graph-based visual formalism, and their theoretical aspects have been thoroughly investigated [23–25, 52, 64]. MSCs found early industry adoption by the ITU standard [33] in 1993, was subsequently incorporated into UML [60] in 2005, and is part of the Web Service Choreography Description Language [72]. MST frameworks represent global protocols as syntactic types, and provide an end-to-end workflow from local type projection to type checking of distributed processes. MST frameworks have been implemented in over a dozen programming languages, including Python [16, 55, 57], Java [30, 31], C [58], Go [6, 38], Scala [7], Rust [14, 36], OCaml [32] and F# [56]. Applications of MSTs include operating systems [19], high performance computing [28, 54, 59], cyber-physical systems [50, 51], and web services [74]. Choreographic programming frameworks represent global protocols as global programs, and have been implemented in Java [26], Haskell [66], Rust [34, 39] and applied to distributed architecture [61], cryptographic security protocols [22], and cyber-physical systems [12]. We refer the reader to [73] and [53] for a comprehensive survey of MST and choreography applicability respectively.

We use the example of a task scheduling protocol to illustrate the top-down verification methodology of global protocols, and introduce its key decision problems.

Global protocol specification. Fig. 1 depicts the global specification of the task scheduling protocol as an HMSC. In each HMSC vertex, vertical lines depict protocol participants, and horizontal arrows from the sender to the receiver depict synchronous message exchanges. HMSC edges depict global control flow constructs like recursion and non-deterministic choice. The task scheduling protocol involves three participants: a scheduler `S`, and two workers $\texttt{W}_1$ and $\texttt{W}_2$. The protocol

operates in a loop, but we omit back-edges to the initial vertex for readability. In each loop iteration, s chooses between assigning the entirety of a task to w_1 only, to w_2 only, or splitting the workload between the two workers. The first two cases correspond to branches 1 and 2, where s sends a full message to the respective worker. The third case corresponds to branch 3, where s sends a half message to both workers. Worker w_2 always completes its assigned task immediately and sends it back to s, by echoing the message it received from s. Worker w_1 may choose to behave like w_2 (branches *b* and *d*), or may choose to delegate some or all of its work to w_2 (branches *a* and *c*).

Implementability. The implementability problem was introduced for MSCs in [2], where it is referred to as *deadlock-free realizability*, or *safe realizability*. Implementability is also referred to as *projectability* in the MST literature. Implementability asks whether the global protocol, specified from the perspective of an omniscient coordinator, can be realized in a distributed setting, i.e. in the absence of such a coordinator, where participants can only gain information about the global protocol state by sending and receiving messages asynchronously. In particular, local participants should never deadlock, and should altogether behave consistently according to a single global run, executing send and receive actions in exactly the prescribed order. We refer to the latter property as *protocol fidelity*. Non-implementability arises when a participant's local information is insufficient for correctly determining its next action.

Synthesizing local implementations. If the protocol is implementable, the next step is to synthesize local implementations. Synthesis from specifications was first formulated by Church in 1957 [9] for logical circuits, and has been widely studied for different specification formalisms targeting different implementation models. Examples include distributed synthesis [20,35], reactive synthesis [62,63], syntax-guided synthesis [1], counterexample-guided inductive synthesis [67], and neurosymbolic program synthesis [8]. Synthesis is referred to as *projection* in the MST literature, and can be considered a special case in which the desired behavior is completely specified. Candidate local implementations for participants s, w_1 and w_2 are depicted in Figs. 2 to 4 respectively. Unlike the global protocol, which enjoys the illusion of synchrony, local implementations are asynchronous, i.e. sending and receiving are no longer specified atomically. Throughout the paper, we use $\mathtt{p} \triangleright \mathtt{q}!m$ to denote the send event where participant p sends *m* to q, and $\mathtt{q} \triangleleft \mathtt{p}?m$ to denote the receive event where q receives message *m* from p. In Figs. 2 to 4, the active participant is omitted from transition labels for clarity, e.g. in Fig. 2, transition label w_1!full denotes s sending full to w_1, and w_2?half denotes s receiving half from w_2.

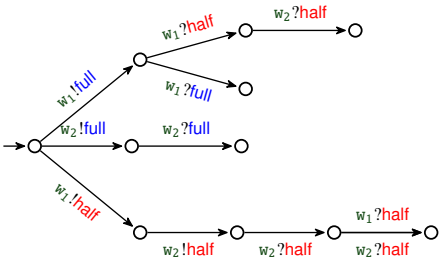


Figure 2: Local implementation for s

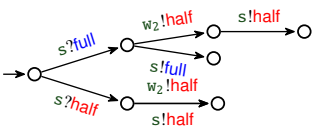


Figure 3: Local implementation for w_1

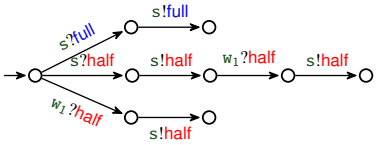


Figure 4: Local implementation for w_2

Subtyping and refinement. The task scheduling protocol offers w_1 multiple choice points at which to decide whether or not to delegate work to w_2. To reduce communication overhead and restrict non-determinism, we may wish to consider a *refinement* of the global protocol that removes some of the power of choice from w_1, for example by eliminating branch $1a$. The problems of *subtyping and refinement* ask whether the resulting global protocol remains implementable, and whether a given candidate implementation correctly implements it.

Implementability, synthesis and subtyping are very relevant and challenging problems, These have been studied using different techniques, most notably automata-based approaches within MSC and type-based systems within MST.

This paper presents sound and complete solutions to the aforementioned problems that improve prior work along the dimensions of expressivity, precision and optimality. This paper summarizes results from the author's Ph.D. thesis [40], parts of which have been published in [41–43, 45–47]. The summary aims to highlight the unifying semantic perspective adopted by [40] in addressing these problems: viewing global protocols and their local implementations as automata, and viewing their behavior as formal languages over an alphabet of asynchronous send and receive events. In particular, we show how the main contribution of [40], a characterization of implementability as a set of reachability and co-reachability 2-hyperproperties, called *Coherence Conditions*, can be used to derive the other obtained theoretical results, including matching complexity bounds for various finite protocol fragments, asymptotically optimal decision procedures for implementability, and sound and complete synthesis algorithms. The preciseness of the

characterization has been mechanized in the Rocq prover, and algorithms for deciding implementability have been implemented in a tool called Sprout. We further show that this characterization is naturally extensible along two orthogonal axes: to protocols with infinite states and data, and to alternative models of asynchrony beyond the commonly assumed peer-to-peer FIFO.

The remainder of the paper is structured as follows. In §2, we present preliminaries. In §3, we present our unifying representation of global protocols, called *Global Communicating Labeled Transition Systems* (GCLTS). We then present the set of Coherence Conditions (*CC*), sketch its equivalence to GCLTS implementability, and use it to derive synthesis and complexity results. In §4, we extend *CC* to the infinite-state setting, and in §5, we extend *CC* to the network-parametric setting. In §6, we briefly discuss aspects of *CC*'s mechanization and implementation.

2 Preliminaries

Words. Let Σ be an alphabet. Σ^* denotes the set of finite words over Σ, Σ^ω the set of infinite words, and Σ^∞ their union $\Sigma^* \cup \Sigma^\omega$. A word $u \in \Sigma^*$ is a *prefix* of word $v \in \Sigma^\infty$, denoted $u \leq v$, if there exists $w \in \Sigma^\infty$ with $u \cdot w = v$; we denote all prefixes of u with $\mathrm{pref}(u)$. Given a word $w = w_0 \dots w_n$, we use $w[i]$ to denote the i-th symbol $w_i \in \Sigma$, and $w[0..i]$ to denote the subword between and including w_0 and w_i, i.e. $w_0 \dots w_i$.

Message Alphabets. Let $\mathcal{P}$ be a possibly infinite set of participants and $\mathcal{V}$ be a possibly infinite data domain. We define the set of *synchronous events* $\Gamma_{sync} := \{\mathtt{p} \to \mathtt{q} : m \mid \mathtt{p}, \mathtt{q} \in \mathcal{P} \text{ and } m \in \mathcal{V}\}$ where $\mathtt{p} \to \mathtt{q} : m$ denotes a message exchange of m from sender $\mathtt{p}$ to receiver $\mathtt{q}$. For a participant $\mathtt{p} \in \mathcal{P}$, we define the alphabet $\Gamma_{\mathtt{p}} := \{\mathtt{p} \to \mathtt{q} : m \mid \mathtt{q} \in \mathcal{P},\ m \in \mathcal{V}\} \cup \{\mathtt{q} \to \mathtt{p} : m \mid \mathtt{q} \in \mathcal{P},\ m \in \mathcal{V}\}$, denoting the set of synchronous events in which $\mathtt{p}$ is either the sender or receiver in the message exchange. A synchronous event is split into a send and receive *asynchronous event* for the sender and receiver respectively. For a participant $\mathtt{p} \in \mathcal{P}$, we define the alphabet $\Sigma_{\mathtt{p},!} := \{\mathtt{p} \triangleright \mathtt{q}!m \mid \mathtt{q} \in \mathcal{P},\ m \in \mathcal{V}\}$ of *send* events and the alphabet $\Sigma_{\mathtt{p},?} := \{\mathtt{p} \triangleleft \mathtt{q}?m \mid \mathtt{q} \in \mathcal{P},\ m \in \mathcal{V}\}$ of *receive* events. The asynchronous event $\mathtt{p} \triangleright \mathtt{q}!m$ denotes participant $\mathtt{p}$ sending a message m to $\mathtt{q}$, and $\mathtt{p} \triangleleft \mathtt{q}?m$ denotes participant $\mathtt{p}$ receiving a message m from $\mathtt{q}$. We write $\Sigma_{\mathtt{p}} = \Sigma_{\mathtt{p},!} \cup \Sigma_{\mathtt{p},?}$, $\Sigma_! = \bigcup_{\mathtt{p} \in \mathcal{P}} \Sigma_{\mathtt{p},!}$, and $\Sigma_? = \bigcup_{\mathtt{p} \in \mathcal{P}} \Sigma_{\mathtt{p},?}$. Finally, the set of *asynchronous* events is $\Sigma_{async} = \Sigma_! \cup \Sigma_?$.

Projections. We map synchronous words to asynchronous words using a homomorphism $\mathtt{split}$, defined as $\mathtt{split}(\mathtt{p} \to \mathtt{q} : m) := \mathtt{p} \triangleright \mathtt{q}!m.\, \mathtt{q} \triangleleft \mathtt{p}?m$. Because $\mathtt{split}$ is injective, there exists a unique inverse, which we denote $\mathtt{split}^{-1}$. We say that $\mathtt{p}$

is *active* in x if $x \in \Gamma_{sync}$ and $x \in \Gamma_{\mathtt{p}}$, or if $x \in \Sigma_{async}$ and $x \in \Sigma_{\mathtt{p}}$. For each participant $\mathtt{p} \in \mathcal{P}$, we define a homomorphism $\Downarrow_{\Gamma_{\mathtt{p}}}$, where $x\Downarrow_{\Gamma_{\mathtt{p}}} = x$ if $x \in \Gamma_{\mathtt{p}}$ and ε otherwise, and a homomorphism $\Downarrow_{\Sigma_{\mathtt{p}}}$, where $x\Downarrow_{\Sigma_{\mathtt{p}}} = x$ if $x \in \Sigma_{\mathtt{p}}$ and ε otherwise. We define a class of projections based on pattern-matching of alphabet symbols, denoted $\Downarrow_{_}$. The result of the projection is determined by the unspecified parts of the pattern. For example, $\Downarrow_{\mathtt{p}\triangleright_!_}$ projects the event $\mathtt{p} \triangleright \mathtt{q}!m$ onto $(\mathtt{q}, m)$, and non-send events and send events that do not have $\mathtt{p}$ as the sender onto ε. The function $\Downarrow_{\mathtt{q}\triangleleft\mathtt{p}?_}$ projects receive events of $\mathtt{p}$ from $\mathtt{q}$ of any message value onto the message value, and all other events to ε.

Our starting point for specifying global protocols is a labeled transition system over the synchronous alphabet Γ_{sync}.

Labeled Transition Systems A *labeled transition system* (LTS) is a tuple $\mathcal{S} = (S, \Gamma, T, s_0, F)$ where S is a set of states, Γ is a set of labels, T is a set of transitions from $S \times \Gamma \times S$, $F \subseteq S$ is a set of final states, and $s_0 \in S$ is the initial state. We use $p \xrightarrow{\alpha} q$ to denote the transition $(p, \alpha, q) \in T$. Runs and traces of an LTS are defined in the expected way. A run is *maximal* if it is either finite and ends in a final state, or is infinite. The language of an LTS $\mathcal{S}$, denoted $\mathcal{L}(\mathcal{S})$, is defined as the set of maximal traces. A state $s \in S$ is a *deadlock* if it is not final and has no outgoing transitions. An LTS is *deadlock-free* if no reachable state is a deadlock. An LTS is *deterministic* if for every $s \xrightarrow{x_1} s_1, s \xrightarrow{x_2} s_2 \in T$, $x_1 = x_2$ implies $s_1 = s_2$. Given an LTS $\mathcal{S} = (S, \Gamma, T, s_0, F)$ and a state $s \in S$, we use $\mathcal{S}_s$ to denote the LTS obtained by replacing s_0 with s as the initial state: (S, Γ, T, s, F).

Our distributed implementation model is based on communicating state machines (CSMs) [5]. CSMs consist of a collection of deterministic finite state machines, one for each participant, that communicate via pairwise FIFO channels. CLTS generalize CSMs by lifting the restriction that the number of participants and the state space of each local state machine must be finite.

Communicating LTS. $\mathcal{T} = \{T_{\mathtt{p}}\}_{\mathtt{p}\in\mathcal{P}}$ is a *communicating labeled transition system* (CLTS) over $\mathcal{P}$ and $\mathcal{V}$ if $T_{\mathtt{p}}$ is a deterministic LTS over $\Sigma_{\mathtt{p}}$ for every $\mathtt{p} \in \mathcal{P}$, denoted by $(Q_{\mathtt{p}}, \Sigma_{\mathtt{p}}, \delta_{\mathtt{p}}, q_{0,\mathtt{p}}, F_{\mathtt{p}})$. Let $\prod_{\mathtt{p}\in\mathcal{P}} Q_{\mathtt{p}}$ denote the set of global states and $\mathsf{Chan} = \{(\mathtt{p}, \mathtt{q}) \mid \mathtt{p}, \mathtt{q} \in \mathcal{P}, \mathtt{p} \neq \mathtt{q}\}$ denote the set of channels. A *configuration* of $\mathcal{A}$ is a pair $(\vec{s}, \xi)$, where $\vec{s}$ is a global state and $\xi : \mathsf{Chan} \to \mathcal{V}^*$ is a mapping from each channel to a sequence of messages. We use $\vec{s}_{\mathtt{p}}$ to denote the state of $\mathtt{p}$ in $\vec{s}$. The CLTS transition relation, denoted $\rightarrow$, is defined as follows.

- $(\vec{s}, \xi) \xrightarrow{\mathtt{p}\triangleright\mathtt{q}!m} (\vec{s}', \xi')$ if $(\vec{s}_{\mathtt{p}}, \mathtt{p} \triangleright \mathtt{q}!m, \vec{s}'_{\mathtt{p}}) \in \delta_{\mathtt{p}}$, $\vec{s}_{\mathtt{r}} = \vec{s}'_{\mathtt{r}}$ for every participant $\mathtt{r} \neq \mathtt{p}$, $\xi'(\mathtt{p}, \mathtt{q}) = \xi(\mathtt{p}, \mathtt{q}) \cdot m$ and $\xi'(c) = \xi(c)$ for every other channel $c \in \mathsf{Chan}$.

- $(\vec{s}, \xi) \xrightarrow{\mathtt{q} \triangleleft \mathtt{p}?m} (\vec{s}', \xi')$ if $(\vec{s}_{\mathtt{q}}, \mathtt{q} \triangleleft \mathtt{p}?m, \vec{s}'_{\mathtt{q}}) \in \delta_{\mathtt{q}}$, $\vec{s}_{\mathtt{r}} = \vec{s}'_{\mathtt{r}}$ for every participant $\mathtt{r} \neq \mathtt{q}$, $\xi(\mathtt{p}, \mathtt{q}) = m \cdot \xi'(\mathtt{p}, \mathtt{q})$ and $\xi'(c) = \xi(c)$ for every other channel $c \in \mathsf{Chan}$.

In the initial configuration $(\vec{s}_0, \xi_0)$, each participant's state in $\vec{s}_0$ is the initial state $q_{0,\mathtt{p}}$ of $A_{\mathtt{p}}$, and ξ_0 maps each channel to ε. A configuration $(\vec{s}, \xi)$ is *final* iff $\vec{s}_{\mathtt{p}}$ is final for every $\mathtt{p}$ and ξ maps each channel to ε. Runs and traces are defined in the expected way. A run is *maximal* if either it is finite and ends in a final configuration, or it is infinite. The language $\mathcal{L}(\mathcal{T})$ of the CLTS $\mathcal{T}$ is defined as the set of maximal traces. A configuration $(\vec{s}, \xi)$ is a *deadlock* if it is not final and has no outgoing transitions. A CLTS is *deadlock-free* if no reachable configuration is a deadlock. Equivalently, a CLTS is deadlock-free if every trace has a maximal extension.

Note that CLTS fixes a peer-to-peer communication topology with FIFO queues as its message channels. We refer to the communication topology and channel data structure as the *network architecture*, denoted $\mathbb{A}$. We defer the formal definition of $\mathbb{A}$ to §5, where we also generalize the definition of CLTS to be parametric in the choice of $\mathbb{A}$; for now fix $\mathbb{A}$ to be peer-to-peer communication over FIFO channels.

Channel Compliance. Before we define the semantics of global protocols, it is useful to disentangle the behavior of a given network architecture $\mathbb{A}$ from that of any particular CLTS $\mathcal{T}$. We extend our semantic perspective to network architectures, and introduce the notion of *channel compliance*, which describes the set of all words consistent with $\mathbb{A}$. We define channel compliance in terms of a *universal* CLTS whose local implementations accept all traces. Thus, the only restrictions on traces of a universal CLTS are imposed by the network architecture $\mathbb{A}$. Formally, let $\mathcal{U}_{\mathbb{A}}$ be the *universal CLTS* for $\mathbb{A}$, with $\mathcal{U}_{\mathbb{A}} = \{U_{\mathtt{p}}\}_{\mathtt{p} \in \mathcal{P}}$ such that $\mathcal{L}(U_{\mathtt{p}}) = \Sigma^*_{\mathtt{p}}$ for every $\mathtt{p} \in \mathcal{P}$. We say that $w \in \Sigma^*_{async}$ is $\mathbb{A}$-channel-compliant if w is a trace of $\mathcal{U}_{\mathbb{A}}$. If $\mathbb{A}$ is understood, we simply say w is channel-compliant. Moreover, if in fact $w \in \mathcal{L}(\mathcal{U}_{\mathbb{A}})$ holds, then we call w *channel-matched*. Intuitively, if in any given CLTS for $\mathbb{A}$ one can execute a channel-matched word, then in the reached configuration all buffers will be empty. We use $\mathcal{L}(\mathbb{A}) \subseteq \Sigma^*_{async}$ to denote all channel-compliant words.

Global Protocol Semantics. We next define the asynchronous semantics of global protocol $\mathcal{S}$, denoted $C^{\sim}(\mathcal{S}) \subseteq \Sigma^{\infty}$. The starting point for the semantics $C^{\sim}(\mathcal{S})$ is the synchronous language $\mathcal{L}(\mathcal{S})$. A word in $\mathcal{L}(\mathcal{S})$ specifies the coordination of events across all protocol participants, in addition to a total order of events per participant. From $\mathcal{L}(\mathcal{S})$ we obtain a set of 1-synchronous asynchronous words through `split`, which simply splits each atomic message exchange into its send and receive counterparts, denoted $\mathtt{split}(\mathcal{L}(\mathcal{S}))$. We want to include all asynchronous words that are equal to these 1-synchronous words under local projection and the given network architecture $\mathbb{A}$.

We handle the finite and infinite words separately to define the global protocol semantics as the union of its finite and infinite semantics:

$$C^{\sim}(\mathcal{S}) = C^{\sim}_{\text{fin}}(\mathcal{S}) \cup C^{\sim}_{\text{inf}}(\mathcal{S})$$

The finite semantics is obtained by following the above recipe, but restricting $\texttt{split}(\mathcal{L}(\mathcal{S}))$ to finite words:

$$C^{\sim}_{\text{fin}}(\mathcal{S}) = [\Sigma^{*}_{\mathit{async}} \cap \texttt{split}(\mathcal{L}(\mathcal{S}))]_{\equiv_{\mathcal{P}}} \cap \mathcal{L}(\mathbb{A}) \ .$$

The infinite semantics are those words whose prefixes are extensible to some word in $\mathcal{L}(\mathcal{S})$ modulo equality under local projection and the network semantics:

$$C^{\sim}_{\text{inf}}(\mathcal{S}) = \{w \in \Sigma^{\infty}_{\mathit{async}} \mid \forall u \leq w.\ u \in \text{pref}([\texttt{split}(\mathcal{L}(\mathcal{S}))]_{\equiv_{\mathcal{P}}} \cap \mathcal{L}(\mathbb{A}))\} \ .$$

Finally, we define protocol implementability.

Protocol Implementability. A protocol $\mathcal{S}$ is *implementable* if there exists a CLTS $\{T_{\mathtt{p}}\}_{\mathtt{p}\in\mathcal{P}}$ satisfying the following two properties:

(i) *protocol fidelity*: $\mathcal{L}(\{T_{\mathtt{p}}\}_{\mathtt{p}\in\mathcal{P}}) = \mathcal{L}(\mathcal{S})$, and

(ii) *deadlock freedom*: $\{T_{\mathtt{p}}\}_{\mathtt{p}\in\mathcal{P}}$ is deadlock-free.

We say that such a $\{T_{\mathtt{p}}\}_{\mathtt{p}\in\mathcal{P}}$ implements $\mathcal{S}$.

3 Implementability

3.1 Motivation

We first analyze the implementability of the task scheduling protocol in Fig. 1 by a CLTS with peer-to-peer FIFO channels. Consider the two executions u_1 and u_2.

$u_1 = \mathtt{s} \triangleright \mathtt{w}_1!\mathsf{full} \cdot \mathtt{w}_1 \triangleright \mathtt{w}_2!\mathsf{half}$ $\qquad$ $u_2 = \mathtt{s} \triangleright \mathtt{w}_1!\mathsf{half} \cdot \mathtt{w}_1 \triangleleft \mathtt{s}?\mathsf{half} \cdot \mathtt{w}_1 \triangleright \mathtt{w}_2!\mathsf{half}$

In u_1, participants $\mathtt{w}_1$ and $\mathtt{s}$ decide to follow branch $1a$, whereas in u_2, they decide to follow branch $3c$. However, from the perspective of $\mathtt{w}_2$, who has observed no events in the protocol thus far, the two executions are indistinguishable: both lead to the same local state q, in which only half from $\mathtt{w}_1$ is available in the channel between $\mathtt{w}_1$ and $\mathtt{w}_2$ (the half from $\mathtt{s}$ is delayed in u_2). Thus, in a distributed setting, $\mathtt{w}_1$ cannot tell the two executions apart. In order to correctly follow the protocol, $\mathtt{w}_2$'s next action in u_1 should be to send half to $\mathtt{s}$, whereas in u_2, $\mathtt{w}_2$ should first wait for the arrival of half from $\mathtt{s}$. If $\mathtt{w}_2$ were to wait in state q, this would lead u_1 to deadlock, and if $\mathtt{w}_2$ were to send half to server, this would lead u_2 to violate protocol fidelity. Thus, the protocol in Fig. 1 is not implementable.

A possible repair is to replace the message value half sent from $\mathtt{w}_1$ to $\mathtt{w}_2$ on branch $1a$ with delegate. Now, $\mathtt{w}_2$ can tell the two branches apart: it can wait until either half is available in its channel from $\mathtt{s}$, indicating that the other participants are following branch 3, or delegate is available in the channel from $\mathtt{w}_1$, indicating that the other participants are following branch $1a$. Since the two cases are exclusive, $\mathtt{w}_2$ can make its decision as soon as it observes one of the two. This change renders the protocol implementable.

3.2 Global Communicating Labeled Transition Systems

Unfortunately, the implementability problem for global protocols is undecidable in general [48]. To restore decidability, we impose three conditions on the class of global protocols as LTS we consider that are inspired by the syntactic structure of global multiparty session types.

Global communicating labeled transition systems. An LTS $\mathcal{S} = (S, \Gamma_{sync}, T, s_0, F)$ is a *global communicating labeled transition system* (GCLTS) if it satisfies the following conditions:

(1) *sink-finality*: for every final state $s \in F$, there does not exist $l \in \Gamma_{sync}$ and $s' \in S$ with $s \xrightarrow{l} s' \in T$;

(2) *sender-driven choice*: for all states $s, s_1, s_2 \in S$ and $l_1, l_2 \in \Gamma_{sync}$ such that $s \xrightarrow{l_i} s_i \in T$ for $i \in \{1, 2\}$, there is a participant $\mathtt{p} \in \mathcal{P}$ who is the sender for both, i.e. $\mathtt{split}(l_i) \in \Sigma_{\mathtt{p},!}$ for $i \in \{1, 2\}$, and furthermore $l_1 = l_2 \implies s_1 = s_2$;

(3) *deadlock freedom*: $\mathcal{S}$ is deadlock-free.

Condition (1) is only required to ensure that the *finite* language of an implementation for global protocol $\mathcal{S}$ matches the *finite* semantics of $\mathcal{S}$. The condition can be waived if we define the semantics of our implementation model in terms of traces. Condition (2) generalizes classical multiparty session types fragments, which require not only a dedicated sender but also a dedicated receiver, a condition we refer to as *directed choice*. In contrast, *mixed choice* lifts all restrictions on choice, and amounts to only requiring determinism. [48] showed that realizability is undecidable for high level message sequence charts satisfying determinism and Condition (3). Sender-driven choice thus represents a good middle ground, allowing to express interesting communication patterns while retaining decidability of implementability. Condition (3) simply requires that protocols do not specify deadlocking behaviors.

In the remainder of this paper, we refer to a GCLTS simply as a *protocol*.

3.3 Coherence Conditions

The non-implementability in our running example can be attributed to insufficient local information about protocol control flow. Existing MST works soundly detect insufficient local information through conservative, type-based projection algorithms [11, 29, 65, 68], which are partial functions that map global types to collections of local types, and are only defined for implementable global types. In other words, projection algorithms solve implementability and synthesis altogether in one shot. [42] presents an automata-based projection algorithm that is both sound and complete, but still requires synthesizing local implementations upfront to check implementability, which is in PSPACE.

Our approach instead solves implementability by analyzing the global protocol directly, without synthesizing a candidate implementation upfront. This is especially significant in the infinite case, discussed in §4, when synthesizing a candidate implementation is itself challenging and not always possible. Our key insight is that non-implementability can be blamed solely on the existence of certain pairs of states in the global protocol. Specifically, non-implementability arises when two states are *locally indistinguishable* to a participant — that is, they are both reachable via the same sequence of that participant's local events — yet require that participant to behave differently. This observation naturally gives rise to a characterization of implementability as 2-hyperproperties with a non-interference flavor. We develop this characterization, called the Coherence Conditions, in the remainder of the section.

Let $\mathcal{S} = (S, \Gamma_{sync}, T, s_0, F)$ be a protocol and let $\mathtt{p} \in \mathcal{P}$ be a participant. We begin by formalizing a participant's local information about the protocol using two variations on the standard notion of reachability. We first define a notion of reachability that restricts the transitions to only the actions observable by a single participant.

Participant-based Reachability. We say that $s \in S$ is *reachable for* $\mathtt{p}$ *on* $u \in \Gamma_{\mathtt{p}}^*$ when there exists $w \in \Gamma_{sync}^*$ such that $s_0 \xrightarrow{w}{}^* s \in T$ and $w{\Downarrow_{\Gamma_{\mathtt{p}}}} = u$, which we denote $s_0 \overset{u}{\underset{\mathtt{p}}{\Rightarrow}}{}^* s$.

Simultaneous Reachability. We say that $s_1, s_2 \in S$ are *simultaneously reachable* for participant $\mathtt{p}$ on $u \in \Gamma_{\mathtt{p}}^*$, denoted $s_0 \overset{u}{\underset{\mathtt{p}}{\Rightarrow}}{}^* s_1, s_2$, if there exist $w_1, w_2 \in \Gamma_{sync}^*$ such that $s_0 \xrightarrow{w_1}{}^* s_1 \in T$, $s_0 \xrightarrow{w_2}{}^* s_2 \in T$ and $w_1{\Downarrow_{\Gamma_{\mathtt{p}}}} = w_2{\Downarrow_{\Gamma_{\mathtt{p}}}} = u$.

Simultaneous reachability captures the notion of *locally indistinguishable* states: to a participant, two states are locally indistinguishable if they are simultaneously reachable.

Next, we analyze the possible asynchronous behavior of a participant when it is in two locally indistinguishable states, starting from send events. Since send events are always enabled, and are moreover committal to a particular protocol run, we require that any message that can be sent from a state can also be sent from all other states that are locally indistinguishable to the sender. This condition is captured by Send Coherence, defined below.

Definition 1 (Send Coherence)**.** $\mathcal{S}$ satisfies Send Coherence (SC) when for every $s_1 \xrightarrow{\mathtt{p}\to\mathtt{q}:m} s_2 \in T, s_1' \in S$:

$$(\exists u \in \Gamma_{\mathtt{p}}^*.\ s_0 \underset{\mathtt{p}}{\overset{u}{\Rightarrow}}{}^* s_1, s_1') \implies (\exists s_2' \in S.\ s_1' \underset{\mathtt{p}}{\overset{\mathtt{p}\to\mathtt{q}:m}{\Longrightarrow}}{}^* s_2') \ .$$

Next, we consider receive events. Unlike send events, receive events are not always enabled. The availability of messages from other participants encodes information about the global protocol state, and receptions correspondingly update the receiver's local view. The information-carrying role that receptions serve, coupled with the fact that in each protocol run, a participant's events are totally ordered, gives rise to Receive Coherence, which requires that receptions must uniquely distinguish two locally indistinguishable states for the receiver.

Definition 2 (Receive Coherence)**.** $\mathcal{S}$ satisfies Receive Coherence (RC) when for every $s_1 \xrightarrow{\mathtt{p}\to\mathtt{q}:m} s_2, s_1' \xrightarrow{\mathtt{r}\to\mathtt{q}:m} s_2' \in T$ with $\mathtt{r} \neq \mathtt{p}$:

$$\begin{aligned}&(\exists u \in \Gamma_{\mathtt{q}}^*.\ s_0 \underset{\mathtt{q}}{\overset{u}{\Rightarrow}}{}^* s_1, s_1') \implies \\ &\forall w \in \mathrm{pref}(\mathcal{L}(\mathcal{S}_{s_2'})).\ w{\Downarrow_{\Sigma_{\mathtt{q}}}} \neq \varepsilon \ \vee\ \mathcal{V}(w{\Downarrow_{\mathtt{p}\triangleright\mathtt{q}!_}}) \neq \mathcal{V}(w{\Downarrow_{\mathtt{q}\triangleleft\mathtt{p}?_}}) \cdot m) \ .\end{aligned}$$

Finally, we require that participants cannot equivocate between sending and receiving messages in two locally indistinguishable states. We call this condition No Mixed Choice.

Definition 3 (No Mixed Choice)**.** A protocol $\mathcal{S} = (S, \Gamma_{sync}, T, s_0, F)$ satisfies No Mixed Choice (NMC) when for every $s_1 \xrightarrow{\mathtt{p}\to\mathtt{q}:m} s_2, s_1' \xrightarrow{\mathtt{r}\to\mathtt{p}:m} s_2' \in T$:

$$(\exists u \in \Gamma_{\mathtt{p}}^*.\ s_0 \underset{\mathtt{p}}{\overset{u}{\Rightarrow}}{}^* s_1, s_1') \implies \bot \ .$$

Our semantic characterization of protocol implementability is the conjunction of the above three conditions.

Definition 4 (Coherence Conditions)**.** A protocol satisfies Coherence Conditions (CC) when it satisfies Send Coherence, Receive Coherence and No Mixed Choice.

We show that *CC* is equivalent to implementability, i.e. it soundly and completely characterizes implementable protocols.

Theorem 5 (Preciseness of Coherence Conditions)**.** *Let $\mathcal{S}$ be a protocol. Then, $\mathcal{S}$ is implementable if and only if it satisfies CC.*

The proof of Theorem 5 is fully mechanized in Rocq. We sketch the proof, in particular that of soundness, which has immediate consequences for synthesis.

Soundness requires showing that any protocol satisfying *CC* is implementable. We choose the *canonical implementation* as the existential witness: a CLTS $\{T_{\mathtt{p}}\}_{\mathtt{p}\in\mathcal{P}}$ is a canonical implementation for $\mathcal{S}$ if for every $\mathtt{p} \in \mathcal{P}$, $T_{\mathtt{p}}$ satisfies $\mathcal{L}(T_{\mathtt{p}}) = \mathcal{L}(\mathcal{S})\Downarrow_{\Sigma_{\mathtt{p}}}$ and $\mathrm{pref}(\mathcal{L}(T_{\mathtt{p}})) = \mathrm{pref}(\mathcal{L}(\mathcal{S})\Downarrow_{\Sigma_{\mathtt{p}}})$. The canonical implementation is an obvious choice because it contains exactly the behaviors prescribed by the global protocol: no more, and no less. Thus, it is easy to show that the canonical implementation exhibits at least the global protocol's behaviors. On the contrary, it is challenging to show that the canonical implementation does not introduce new, unspecified behaviors. The proof amounts to showing that the invariant of possible run set non-emptiness is inductive thanks to the Coherence Conditions.

Completeness is proved constructively by modus tollens: assuming the negation of each Coherence Condition in turn, we construct a concrete trace that any implementation must admit, but which is not a prefix of the protocol language, thereby violating either protocol fidelity or deadlock freedom. The modularity of the Coherence Conditions means that implementability violations can be localized to individual transitions for a given participant, a fact that Sprout exploits for actionable error reporting.

3.4 Synthesis

The role that the canonical implementation plays in the proof of soundness immediately yields the following corollary.

Corollary 6 (Canonical implementation is all you need)**.** *A protocol $\mathcal{S}$ is implementable if and only if its canonical implementation implements it.*

Furthermore, the canonical implementation's definition immediately suggests a direct synthesis algorithm that is sound and complete thanks to the above: first, project the global protocol onto each participant's local alphabet, then determinize the result. For finite global protocols, this amounts to a modified subset construction, and, recalling that space complexity disregards output tape usage, yields the following complexity result for the synthesis problem:

Theorem 7 (Synthesis of finite global protocols is in PSPACE)**.** *For finite protocols, the canonical implementation for each participant can be computed in PSPACE.*

The exponential state space blowup incurred by subset construction cannot be avoided when used for synthesizing canonical implementations. The following family of implementable global protocols exhibits the exponential blowup. We represent the global protocols as global types $\mathbf{G}_n$ for $n \in \mathbb{N}$ such that $\mathbf{G}_n$ has size linear in n. In global types, synchronous message exchanges $\mathtt{p} \to \mathtt{q} : m$ appear along with + denoting choice, μt binding a recursion variable t, and 0 denoting termination. The construction of the $\mathbf{G}_n$'s builds on the regular expression $(a^*(ab^*)^n a)^*$, which can only be recognized by a deterministic finite state machine that grows exponentially with n [17, Theorem 11].

Definition 8 (Global type family $\mathbf{G}_n$ with exponential canonical implementation)**.** $\mathbf{G}_n \coloneqq G(G_n)$, with G_i and G(-) are defined as:

$$G_0 \coloneqq \mathtt{p} \to \mathtt{q} : a.\, t_1 \quad \text{and} \quad G_i \coloneqq \mathtt{p} \to \mathtt{q} : a.\, \mu t_{3,i}. + \begin{cases} \mathtt{p} \to \mathtt{r} : m_3.\, \mathtt{p} \to \mathtt{q} : b.\, t_{3,i} \\ \mathtt{p} \to \mathtt{r} : n_3.\, G_{i-1} \end{cases} \quad \text{for } i > 0$$

$$G(G') \coloneqq \mu t_1.\ + \begin{cases} \mathtt{p} \to \mathtt{r} : m_1.\, \mu t_2. + \begin{cases} \mathtt{p} \to \mathtt{r} : m_2.\, \mathtt{p} \to \mathtt{q} : a.\, t_2 \\ \mathtt{p} \to \mathtt{r} : n_2.\, G' \end{cases} \\ \mathtt{p} \to \mathtt{r} : n_1.\, 0 \end{cases} .$$

Each G_i for $i > 0$ generates (ab^*), G_0 adds the last a, and the outermost Kleene star and the first a^* are defined using the scaffolding provided by G(-). Participant $\mathtt{p}$'s choice to send either m_3 or n_3 to $\mathtt{r}$ respectively encodes the choice to continue iterating b's or to stop in b^*; participant $\mathtt{q}$ however, is not involved in this exchange and thus $\mathtt{q}$'s local language is isomorphic to $(ab^*)^i a$. As $\mathbf{G}_n$ is implementable, the subset construction for each participant implements it by Corollary 6. By the definition of canonicity, we have $\mathcal{L}(\mathbf{G}_n)\Downarrow_{\Sigma_\mathtt{q}}$ for participant $\mathtt{q}$ over alphabet $\Sigma_\mathtt{q}$. Because $\mathcal{L}(\mathbf{G}_n)\Downarrow_{\Sigma_\mathtt{q}}$ can only be recognized by a deterministic finite state machine with size exponential in n, the corresponding local language preserving implementation also has size exponential in n.

Like implementability, synthesis is undecidable for the general class of protocols. However, for many expressive fragments of protocols that still feature infinite data, e.g. symbolic finite automata [15, 66] and certain classes of timed and register automata [4, 10], off-the-shelf determinization algorithms [3, 70, 71] can be used directly to compute canonical implementations.

3.5 Complexity

CC directly suggests the following algorithm for deciding implementability of protocols with finitely many states and transitions.

We defer the formal definition of avail to the next section. Using *CC*, we can also establish that implementability of finite protocols is co-NP-complete.

Algorithm 1 Checking *CC* for finite protocols

▷ *Let LTS* $\mathcal{S} = (S, \Gamma_{sync}, T, s_0, F)$
▷ *Checking Send Coherence*
for $s_1 \xrightarrow{\mathtt{p}\to\mathtt{q}:m} s_2 \in T$ **do**
  for $s \neq s_1 \in S$ **do**
    if $\mathcal{L}(S, \Gamma_{\mathtt{p}} \uplus \{\varepsilon\}, T_{\mathtt{p}}, s_0, \{s\}) \cap \mathcal{L}(S, \Gamma_{\mathtt{p}} \uplus \{\varepsilon\}, T_{\mathtt{p}}, s_0, \{s_1\}) \neq \emptyset$ **then**
      $b \leftarrow \bot$
      for $s_3 \xrightarrow{\mathtt{p}\to\mathtt{q}:m} s_4 \in T$ **do** $b \leftarrow b \vee \left(s \underset{\mathtt{p}}{\overset{\varepsilon}{\Rightarrow}}{}^{*} s_3\right)$
      end for
      if $\neg b$ **then return** $\bot$
      end if
    end if
  end for
end for
▷ *Checking Receive Coherence*
for $s_1 \xrightarrow{\mathtt{p}\to\mathtt{q}:m} s_2, s_3 \xrightarrow{\mathtt{r}\to\mathtt{q}:m} s_4 \in T, s_1 \neq s_2, \mathtt{p} \neq \mathtt{r}$ **do**
  if $\mathcal{L}(S, \Gamma_{\mathtt{q}} \uplus \{\varepsilon\}, T_{\mathtt{q}}, s_0, \{s_1\}) \cap \mathcal{L}(S, \Gamma_{\mathtt{q}} \uplus \{\varepsilon\}, T_{\mathtt{q}}, s_0, \{s_3\}) \neq \emptyset$ **then**
    if $\mathsf{avail}_{\mathtt{p},\mathtt{q},\{\mathtt{q}\}}(m, s_4)$ **then return** $\bot$
    end if
  end if
end for
▷ *Checking No Mixed Choice*
for $s_1 \xrightarrow{\mathtt{p}\to\mathtt{q}:m} s_2, s_3 \xrightarrow{\mathtt{r}\to\mathtt{p}:m} s_4 \in T, s_1 \neq s_2$ **do**
  if $\mathcal{L}(S, \Gamma_{\mathtt{q}} \uplus \{\varepsilon\}, T_{\mathtt{q}}, s_0, \{s_1\}) \cap \mathcal{L}(S, \Gamma_{\mathtt{q}} \uplus \{\varepsilon\}, T_{\mathtt{q}}, s_0, \{s_3\}) \neq \emptyset$ **then return** $\bot$
  end if
end for
return $\top$

Theorem 9 (Complexity of implementability for finite protocols)**.** *Implementability of finite protocols is co-NP-complete.*

Proof sketch. To see that implementability is in co-NP, observe that violations of Send Coherence and No Mixed Choice can be checked in NP, by guessing a participant $\mathtt{p}$ and a pair of states s_1, s_2 satisfying the respective preconditions, and verifying simultaneous reachability of s_1 and s_2 for $\mathtt{p}$. For Receive Coherence, $\mathsf{avail}_{\mathtt{p},\mathtt{q},\{\mathtt{q}\}}(m, s_2)$ can be checked in NP by guessing a simple path in $\mathcal{S}$ from s_2 to a state with an outgoing transition $\mathtt{p} \to \mathtt{q} : m$, and evaluating the availability predicate along that path in polynomial time. We can restrict ourselves to simple paths because the blocked set $\mathcal{B}$ monotonically increases along a path in $\mathcal{S}$, and avail is antitone in $\mathcal{B}$.

For co-NP-hardness, we reduce 3-SAT to non-implementability. Given a 3-SAT instance $\varphi = C_1 \wedge \ldots \wedge C_k$ with variables $x_1, \ldots, x_n$ and literals L_{ij}, denoting the jth literal of clause C_i, with $1 \leq i \leq k$ and $1 \leq j \leq 3$, we construct a global protocol $\mathcal{S}_\varphi$ such that ϕ is unsatisfiable iff $\mathcal{S}_\varphi$ is implementable.

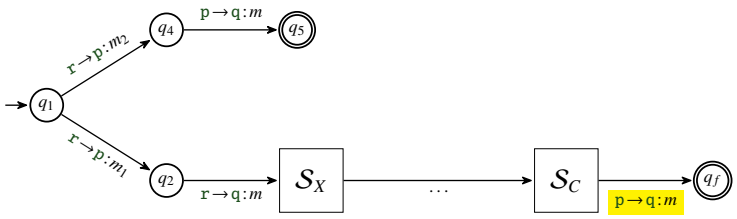


Figure 5: 3-SAT reduction to non-implementability.

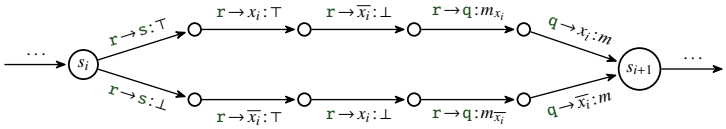


Figure 6: Variable assignment gadget $\mathcal{S}_X$.

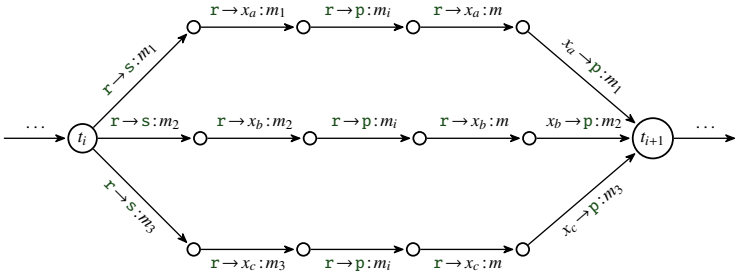


Figure 7: Clause selection gadget $\mathcal{S}_C$.

The construction (summarized pictorially in Fig. 5) relies on two gadgets: $\mathcal{S}_X$, a gadget that encodes a variable assignment to variables $x_1, \ldots, x_n$ (Fig. 6), and $\mathcal{S}_C$, a gadget that encodes literal selection for clauses $C_1, \ldots, C_k$ (Fig. 7). The highlighted message in Fig. 5 is available for participant q in q_2 if and only if φ is satisfiable. Consequently, protocol $\mathcal{S}_\varphi$ is non-implementable if and only if the highlighted message is available for participant q in q_2, if and only if φ is satisfiable. □

Observe that $\mathcal{S}_\varphi$, $\mathcal{S}_X$ and $\mathcal{S}_C$ all satisfy directed choice. Thus, the co-NP hardness lower bound holds even for the syntactically defined fragment of multiparty session types with directed choice [29], and we have the following corollary.

Corollary 10. *Implementability of global multiparty session types is co-NP-complete.*

4 Extension to Infinite Protocols

In this section, we generalize *CC* to infinite-state protocols. We first illustrate how our task delegation protocol can be made infinite-state, then introduce our formalism for specifying infinite-state protocols, called *symbolic protocols*, and finally show how to generalize *CC* to symbolic protocols.

4.1 Motivation

Our repaired task delegation protocol features finitely many message payloads: half, full and done. To model real-world verification targets, we desire for our global protocol specifications to be as expressive as possible. For example, we may wish to express that in branch 3, s can choose to split the workload between w_1 and w_2 into two parts l_1 and l_2, which are two reals that sum up to 1. Thus, there are infinitely many choices of l_1 and l_2, and the global protocol, in addition to specifying control flow, now also specifies data dependencies expressed as numeric constraints.

Data dependencies make the implementability problem significantly more challenging to analyze because it can influence control flow without relying on branching choice, as illustrated in the following pair of examples $\mathbb{S}_1$ (using ⓐ) and $\mathbb{S}'_1$ (using ⓑ) in Fig. 8. In both versions, participant p chooses a branch, even or odd, without explicitly informing r of their choice. In both branches, r is required to subtract the second value that is sent from the first value that is sent, and send the result back to p. However, due to asynchrony, both messages can arrive in r's message channels simultaneously. If r cannot distinguish the two branches, then it cannot tell which value was sent first, and may subtract the values in the wrong order. This is the case in ⓐ, in which r receives a value y from p that satisfies constraint $y > x$, but does not reveal any information about p's next choice. In ⓑ, however, with the constraint on y changed from $y > x$ to $y = x + 2$, the value y now indirectly encodes p's next choice of even or odd through its parity, which in turn informs r of the order in which it should receive messages from p and q. In other words, the incorporation of dependent refinements also enables a new method of protocol repair: one that does not change the communication structure of the original protocol.

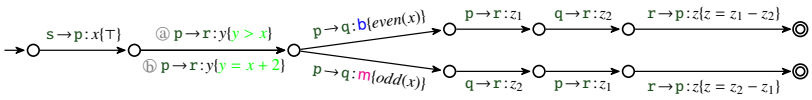


Figure 8: Two protocols: $\mathbb{S}_1$ using ⓐ with receive order violation $\mathbb{S}'_1$ using ⓑ without receive order violation.

We address the additional complexity introduced by data refinements with a semantic separation of concerns. First, we make the key observation that the Coherence Conditions remain sound and complete with respect to implementability even for protocols with infinitely many states and data. Unlike in the finite case, *CC* cannot be checked directly, i.e. by iterating over all states and transitions. Thus, the only remaining task is to reformulate the Coherence Conditions in a symbolic, semantics-preserving way that is effectively checkable for some finite representation of infinite protocols. We introduce this representation, called *symbolic protocols*, below.

In other words, we want to define symbolic conditions that are valid on the symbolic protocol if and only if its concrete protocol is implementable. Specifically, we provide a sound and complete reduction to the first-order fixpoint logic μCLP [69]. Validity of these formulas can then be discharged using off-the-shelf tools, which we demonstrate in the Sprout tool. The μCLP calculus can express recursive predicates with least and greatest fixpoint semantics where the predicate body is constrained by a first-order logic formula over a background theory. The expressive power of μCLP is needed because Send Coherence reasons about both reachability and coreachability, which naturally translate to least and greatest fixpoints over the protocol's transition relation. The reduction thus constitutes an algorithm that is complete relative to an assumed oracle for solving μCLP satisfiability.

4.2 Symbolic Protocols

Our definition of symbolic protocols assumes a fixed but unspecified first-order background theory of message values (e.g. linear real arithmetic). We assume standard syntax and semantics of first-order formulas and denote by $\mathcal{F}$ the set of first-order formulas with free variables drawn from an infinite set X and interpreted over the set of message values $\mathcal{V}$. For a valuation $\rho \in X \to \mathcal{V}$ and $\varphi \in \mathcal{F}(X)$, we write $\rho \models \phi$ to indicate that φ evaluates to true under ρ in the underlying theory.

Definition 11. A symbolic protocol is a tuple $\mathbb{S} = (S, R, \Delta, s_0, \rho_0, F)$ where

- S is a finite set of control states, $F \subseteq S$ is a set of final states,
- R is a finite set of register variables,
- $\Delta \subseteq S \times \mathcal{P} \times X \times \mathcal{P} \times \mathcal{F} \times S$ is a finite set that consists of symbolic transitions of the form $s \xrightarrow{\mathrm{p}\to\mathrm{q}:x\{\varphi\}} s'$ where the formula φ with free variables $R \uplus R' \uplus \{x\}$ expresses a transition constraint that relates the old and new register values R and R' with the sent value x,
- $s_0 \in S$ is the initial control state,
- $\rho_0 \colon R \to \mathcal{V}$ is the initial register assignment.

In our definition, the transition constraint φ altogether specifies constraints on the message value and register updates. To this end, for each register variable $r \in R$, we define a primed copy r' that refers to the same register in the post-state of a transition, and we define $R' = \{r' \mid r \in R\}$. We use r_1, r_2, r_3 to denote register variables, and x, y, z to denote communication variables. Thus, the free variables in φ are either variables from R describing registers in the pre-state, variables from R' describing registers in the post-state, or a communication variable x. For example, $\mathtt{p} \to \mathtt{q} : x\{even(x) \wedge r_1' = r_1 + 1 \wedge r_2' = x\}$ describes $\mathtt{p}$ sending $\mathtt{q}$ an even number x, incrementing the value of register r_1 by 1, and storing the value of x in register r_2.

4.3 Symbolic Coherence Conditions

We first show that the building blocks of *CC* can be encoded as least and greatest fixpoints over a symbolic protocol's transition relation. Then, we use these building blocks to define Symbolic Coherence Conditions. In the remainder of the section, we fix a symbolic protocol $\mathbb{S} = (S, R, \Delta, s_0, \rho_0, F)$ whose concretization is a GCLTS. Additionally, we define two copies of the symbolic protocol, denoted $\mathbb{S}_1$ and $\mathbb{S}_2$. Each copy $\mathbb{S}_i = (R_i, S, \Delta_i, \rho_i, s_0, F)$ with $i \in \{1, 2\}$ is obtained from $\mathbb{S}$ by renaming each register r to a fresh register r_i, each unique communication variable x to x_i, and substituting the new register and communication variables into the transition constraints and initial register assignment accordingly; the control states remain the same.

The first building block used by all three Coherence Conditions is the definition of simultaneously reachable pairs of protocol states, lifted to the symbolic setting. Given a participant and a pair of control states (s_1, s_2) in $\mathbb{S}$, the predicate $\mathsf{prodreach}_{\mathtt{p}}(s_1, \boldsymbol{r_1}, s_2, \boldsymbol{r_2})$ describes the register assignments corresponding to pairs of concrete states (s_1, ρ_1), (s_2, ρ_2) that are simultaneously reachable in the concretization of $\mathbb{S}$, where $\boldsymbol{r_i}$ are vectors of the registers in R_i ordered according to some fixed total order. We define $\mathsf{prodreach}$ as a least fixpoint as follows.

Definition 12 (Simultaneous reachability in product symbolic protocol)**.** Let $\mathtt{p} \in \mathcal{P}$ be a participant and let $s_1, s_1', s_2, s_2' \in S$. Then,

$$
\begin{aligned}
\mathsf{prodreach}_{\mathtt{p}}(s_1', \boldsymbol{r_1'}, s_2', \boldsymbol{r_2'}) :=_{\mu}\ & (\ s_1' = s_0 \wedge s_2' = s_0 \wedge \boldsymbol{r_1'} = \rho_0 \wedge \boldsymbol{r_2'} = \rho_0\ ) \\
\vee\ & \Big(\bigvee_{\substack{(s_1,\, \mathtt{r}\to\mathtt{s}:x_1\{\varphi_1\},\, s_1') \in \Delta_1 \\ (s_2,\, \mathtt{r}\to\mathtt{s}:x_2\{\varphi_2\},\, s_2') \in \Delta_2 \\ \mathtt{p}=\mathtt{r} \vee \mathtt{p}=\mathtt{s}}} \exists x_1 x_2 \boldsymbol{r_1} \boldsymbol{r_2}.\ \mathsf{prodreach}_{\mathtt{p}}(s_1, \boldsymbol{r_1}, s_2, \boldsymbol{r_2}) \wedge \varphi_1 \wedge \varphi_2 \wedge x_1 = x_2\ \Big) \\
\vee\ & \Big(\bigvee_{(s_1,\, \mathtt{r}\to\mathtt{s}:x_1\{\varphi_1\},\, s_1') \in \Delta_1 \,\wedge\, \mathtt{p}\neq\mathtt{r} \wedge \mathtt{p}\neq\mathtt{s}} \exists x_1 \boldsymbol{r_1}.\ \mathsf{prodreach}_{\mathtt{p}}(s_1, \boldsymbol{r_1}, s_2', \boldsymbol{r_2'}) \wedge \varphi_1\ \Big) \\
\vee\ & \Big(\bigvee_{(s_2,\, \mathtt{r}\to\mathtt{s}:x_2\{\varphi_2\},\, s_2') \in \Delta_2 \,\wedge\, \mathtt{p}\neq\mathtt{r} \wedge \mathtt{p}\neq\mathtt{s}} \exists x_2 \boldsymbol{r_2}.\ \mathsf{prodreach}_{\mathtt{p}}(s_1', \boldsymbol{r_1'}, s_2, \boldsymbol{r_2}) \wedge \varphi_2\ \Big)\ .
\end{aligned}
$$

The definition encodes $\mathbb{S}_1$ and $\mathbb{S}_2$ executing the witness u independently. The second top-level disjunct in the definition after the base case handles the cases where $\mathbb{S}_1$ and $\mathbb{S}_2$ synchronize on a common action involving $\mathtt{p}$. The remaining two disjuncts correspond to the cases where either $\mathbb{S}_1$ or $\mathbb{S}_2$ follows an ε transition.

Given a pair of simultaneously reachable states (s_1, ρ_1), (s_2, ρ_2) in $\mathtt{p}$, Send Coherence now checks whether all values x_1 that can be sent to some $\mathtt{q}$ in (s_1, ρ_1) can also be sent from (s_2, ρ_2), modulo following ε transitions to reach the actual state where $\mathtt{p}$ can send to $\mathtt{q}$. We thus need to express ε-reachability. We formalize the dual: the predicate $\mathsf{unreach}^{\varepsilon}_{\mathtt{p},\mathtt{q}}(s_2, \boldsymbol{r_2}, x_1)$ expresses that $\mathtt{p}$ *cannot* reach any state where it may send x_1 to $\mathtt{q}$, by following ε transitions from symbolic state $(s_2, \boldsymbol{r_2})$. This is formulated as a greatest fixpoint as follows:

Definition 13 (ε-unreachability of $\mathtt{p}$ sending x to $\mathtt{q}$)**.** For $\mathtt{p}, \mathtt{q} \in \mathcal{P}$ and $s \in S$, let

$$\mathsf{unreach}^{\varepsilon}_{\mathtt{p},\mathtt{q}}(s, \boldsymbol{r}, x) :=_{\nu} \left(\bigwedge\nolimits_{(s,\, \mathtt{p}\to\mathtt{q}:y\{\varphi\},\, s') \in \Delta} \neg\varphi[x/y]\right) \\ \wedge \left(\bigwedge\nolimits_{\substack{(s,\, \mathtt{r}\to\mathtt{t}:y\{\varphi\},\, s') \in \Delta \\ \mathtt{p}\neq\mathtt{r} \wedge \mathtt{p}\neq\mathtt{t}}} \forall y\, \boldsymbol{r'}.\, \varphi \Rightarrow \mathsf{unreach}^{\varepsilon}_{\mathtt{p},\mathtt{q}}(s', \boldsymbol{r'}, x)\right) \ .$$

The first conjunct checks that whenever $\mathtt{p}$ reaches a state with an outgoing send transition to $\mathtt{q}$, it cannot send the value x because the transition constraint φ is not satisfied. The second conjunct checks that every outgoing ε transition is either disabled ($\neg\varphi$ holds) or following the transition does not reach an appropriate send state.

We combine the auxiliary predicates to define Symbolic Send Coherence.

Definition 14 (Symbolic Send Coherence)**.** A symbolic protocol $\mathbb{S}$ satisfies Symbolic Send Coherence when for each transition $s_1 \xrightarrow{\mathtt{p}\to\mathtt{q}:x_1\{\varphi_1\}} s_1' \in \Delta_1$ and state $s_2 \in S$, the following is valid:

$$\mathsf{prodreach}_{\mathtt{p}}(s_1, \boldsymbol{r_1}, s_2, \boldsymbol{r_2}) \ \wedge \ \varphi_1 \wedge \mathsf{unreach}^{\varepsilon}_{\mathtt{p},\mathtt{q}}(s_2, \boldsymbol{r_2}, x_1) \implies \bot \ .$$

A keen reader may have noticed that because the symbolic characterization of Send Coherence involves a greatest fixpoint, it is a liveness property. Thus, proving Send Coherence, in general, involves a termination argument. To see this, consider the two protocols shown in Figs. 9 and 10. Consider the pair of states $(q_1, [c \mapsto 0])$ and $(q_3, [c \mapsto 0])$ which are simultaneously reachable for $\mathtt{r}$ in both protocols. The send transition for $\mathtt{r}$ enabled in q_1 needs to be matched with a corresponding send transition in an ε-reachable state from q_3. The only candidate states for this match in both protocols are those at control state q_4. These states are reachable from q_3 if and only if the loop in q_3 terminates, which it does in Fig. 9 but not in Fig. 10.

Receive Coherence is conditioned on two simultaneously reachable states $(s_1, \boldsymbol{r_1})$ and $(s_2, \boldsymbol{r_2})$ for a participant $\mathtt{q}$. It checks that if $\mathtt{q}$ can receive x from $\mathtt{p}$ in the first state, $\mathtt{q}$ cannot also receive x as the first message from $\mathtt{p}$ in the second state, in which it can also receive from a different participant $\mathtt{r}$, unless $\mathtt{p}$ sending x causally

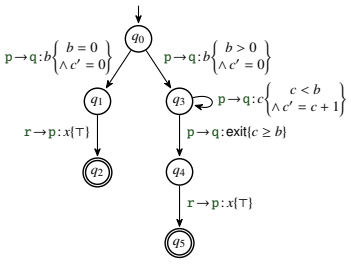


Figure 9: Example where states q_1 and q_3 satisfy Send Coherence for r.

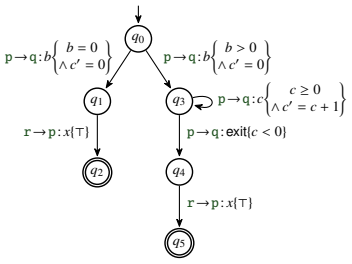


Figure 10: Example where states q_1 and q_3 violate Send Coherence for r.

depends on q first receiving from r. We thus need to define a predicate that captures whether x_1 may be available as the first message from q to p, while tracking causal dependencies. We introduce a family of predicates $\mathsf{avail}_{\mathtt{p},\mathtt{q},\mathcal{B}}(x_1, s_2, \boldsymbol{r_2})$ for this purpose. Here, $\mathcal{B}$ tracks the set of participants that are blocked from sending a message because their send action causally depends on q first receiving from r. Note that because $\mathbb{S}$ has finitely many transitions and thus participants, there are finitely many $\mathcal{B}$ sets. The predicates are defined as the least fixpoint of the following mutually recursive definition.

Definition 15 (Symbolic Availability)**.**

$$\begin{aligned}\mathsf{avail}_{\mathtt{p},\mathtt{q},\mathcal{B}}(x_1, s, \boldsymbol{r}) :=_{\mu} \quad & \Big(\bigvee_{\substack{(s,\, \mathtt{r}\to\mathtt{t}:x\{\varphi\},\, s') \in \Delta, \mathtt{r} \in \mathcal{B} \\ \mathtt{r}\neq\mathtt{p} \vee \mathtt{t}\neq\mathtt{q}}} \exists x\, \boldsymbol{r'}.\, \mathsf{avail}_{\mathtt{p},\mathtt{q},\mathcal{B}\cup\{\mathtt{t}\}}(x_1, s', \boldsymbol{r'}) \wedge \varphi \Big) \\ & \vee \Big(\bigvee_{\substack{(s,\, \mathtt{r}\to\mathtt{t}:x\{\varphi\},\, s') \in \Delta, \mathtt{r} \notin \mathcal{B} \\ \mathtt{r}\neq\mathtt{p} \vee \mathtt{t}\neq\mathtt{q}}} \exists x\, \boldsymbol{r'}.\, \mathsf{avail}_{\mathtt{p},\mathtt{q},\mathcal{B}}(x_1, s', \boldsymbol{r'}) \wedge \varphi \Big) \\ & \vee \Big(\bigvee_{(s,\, \mathtt{p}\to\mathtt{q}:x\{\varphi\},\, s') \in \Delta, \mathtt{p} \notin \mathcal{B}} \varphi[x_1/x] \Big)\ .\end{aligned}$$

The last disjunct in the definition handles the cases where the message x_1 from p is immediately available to be received by q in symbolic state $(s, \boldsymbol{r})$ and p has not been blocked from sending. The other two disjuncts handle the cases when x_1 becomes available after some other message exchange between r and t. Here, if r is blocked, then t also becomes blocked since it depends on r sending before it can receive (the first disjunct). Otherwise, no participant is added to the blocked set (the second disjunct). The predicate $\mathsf{avail}_{\mathtt{p},\mathtt{q},\{\mathtt{q}\}}(m, s)$ from Algorithm 1 is defined in terms of the symbolic availability predicate $\mathsf{avail}_{\mathtt{p},\mathtt{q},\{\mathtt{q}\}}(x, s, \boldsymbol{r})$, where $\boldsymbol{r}$ is empty.

With the available message predicate in place, we can now define Symbolic Receive Coherence.

Definition 16 (Symbolic Receive Coherence)**.** A symbolic protocol $\mathbb{S}$ satisfies Symbolic Receive Coherence when for every pair of transitions $s_1 \xrightarrow{\mathtt{p}\to\mathtt{q}:x_1\{\varphi_1\}} s_1' \in \Delta_1$ and $s_2 \xrightarrow{\mathtt{r}\to\mathtt{q}:x_2\{\varphi_2\}} s_2' \in \Delta_2$ with $\mathtt{p} \neq \mathtt{r}$:

$$\mathsf{prodreach}_{\mathtt{q}}(s_1, \boldsymbol{r_1}, s_2, \boldsymbol{r_2}) \ \wedge \ \varphi_1 \ \wedge \ \varphi_2 \ \wedge \ \mathsf{avail}_{\mathtt{p},\mathtt{q},\{\mathtt{q}\}}(x_1, s_2', \boldsymbol{r_2'}) \implies \bot \ .$$

Finally, No Mixed Choice is conditioned on two simultaneously reachable states $(s_1, \boldsymbol{r_1})$ and $(s_2, \boldsymbol{r_2})$ with outgoing send and receive transitions for a participant $\mathtt{p}$.

Definition 17 (Symbolic No Mixed Choice)**.** A symbolic protocol $\mathbb{S}$ satisfies Symbolic No Mixed Choice when for every pair of transitions $s_1 \xrightarrow{\mathtt{p}\to\mathtt{q}:x_1\{\varphi_1\}} s_1' \in \Delta_1$ and $s_2 \xrightarrow{\mathtt{r}\to\mathtt{p}:x_2\{\varphi_2\}} s_2' \in \Delta_2$:

$$\mathsf{prodreach}_{\mathtt{p}}(s_1, \boldsymbol{r_1}, s_2, \boldsymbol{r_2}) \ \wedge \ \varphi_1 \ \wedge \ \varphi_2 \implies \bot \ .$$

The preciseness of our Symbolic Coherence Conditions is stated as follows.

Theorem 18 (Preciseness of Symbolic Coherence Conditions)**.** $\mathbb{S}$ *is implementable if and only if it satisfies Symbolic Send Coherence, Symbolic Receive Coherence, and Symbolic No Mixed Choice.*

4.4 Complexity

Like in §3.5, we use our Symbolic Coherence Conditions to analyze the complexity of implementability for symbolic representations of finite protocols. More precisely, we consider the fragment of symbolic protocols where $\mathcal{V}$ is the set of Booleans and all transition constraints φ are given by propositional formulas. We show that for this class of symbolic protocols, the implementability problem is PSPACE-complete.

Theorem 19. *Implementability of symbolic finite protocols is PSPACE-complete.*

Proof sketch. To show that implementability is in PSPACE, we show that a witness to the negation of *CC* can be checked in nondeterministic polynomial space. This follows by a reduction to the reachability problem for extended finite state machines, which is in PSPACE [27]. By Savitch's Theorem, it follows that the negation of *CC* is in PSPACE. Because PSPACE is closed under complement and *CC* precisely characterizes implementability, it follows that implementability is in PSPACE.

We show PSPACE-hardness of the implementability problem by a reduction from the problem of deciding reachability for 1-safe Petri nets [18]. Let (N, M_0) be a 1-safe Petri net, with $N = (S, T, F)$. Let M be a marking of N.

We construct a symbolic protocol that is implementable iff N does not reach M. For ease of exposition, we present this symbolic protocol as a symbolic dependent global type $\mathbf{G}_N$ with the understanding that the encoding of $\mathbf{G}_N$ as a symbolic protocol is clear.

We first describe the construction of $\mathbf{G}_N$. The outermost structure of $\mathbf{G}_N$ consists of a participant $\mathtt{r}$ communicating a choice between two branches to $\mathtt{s}$ where the bottom branch solely consists of $\mathtt{p}$ sending l to $\mathtt{q}$: $\mathbf{G}_N \coloneqq (\mathtt{r} \to \mathtt{s} : m_1\{\top\}.\, G_t + \mathtt{r} \to \mathtt{s} : m_2\{\top\}.\, \mathtt{p} \to \mathtt{q} : l\{\top\}.\, 0)$. Since $\mathtt{p}$ is not informed about the choice of the branch taken by $\mathtt{s}$, it will have to be able to match this send transition in every run that follows the continuation G_t of the top branch. We will construct G_t such that this match is possible iff M is reachable in N.

In G_t, participants $\mathtt{r}$ and $\mathtt{s}$ enter a loop that simulates N:

$$G_t \coloneqq \mu s[v \coloneqq M_0].\ + \begin{cases} \sum_{t \in T}\ \mathtt{r} \to \mathtt{s} : m_t\{v \Rightarrow t^-\}.\, s[v \coloneqq ((v \wedge \neg t^-) \vee t^+)] \\ \mathtt{r} \to \mathtt{s} : \mathit{restart}\{\top\}.\, s[v \coloneqq M_0] \\ \mathtt{r} \to \mathtt{s} : \mathit{reach}_M\{v = M\}.\, \mathtt{p} \to \mathtt{q} : l\{\top\}.\, 0 \end{cases}$$

The loop variable v is a $|S|$-length bitvector that tracks the current marking of the net. It is initialized to M_0. Inside the loop, $\mathtt{r}$ has the following choices. First, it may pick any transition $t \in T$ of the net and send an m_t message to $\mathtt{s}$, provided the transition is enabled for firing (i.e., the input places of t all contain a token: $v \Rightarrow t^-$). After this communication, v is updated according to the fired transition t.

The last branch of the choice in the loop is enabled if v is equal to M. Here, $\mathtt{r}$ can send reach_M to $\mathtt{s}$, which gives $\mathtt{p}$ the opportunity to send the l message to $\mathtt{q}$, allowing it to match the send transition from the lower branch in the top level choice of G_N.

Finally, the middle branch allows $\mathtt{r}$ to abort the simulation at any point and start over. This ensures that if the simulation ever reaches a dead state due to firing a transition that would render M unreachable, it can recover by starting again from M_0. Thus, for all states of the simulator, $\mathtt{p}$ has an ε path from that state to a state where it can send l to $\mathtt{q}$ iff M is reachable from M_0 in N. The only other sender is $\mathtt{r}$ which makes all choices and, hence, never reaches two different states along the same prefix trace, thus satisfying Send Coherence trivially. It follows that Send Coherence for $\mathtt{p}$ holds iff M is reachable from M_0 in N. To see that Receive Coherence holds, observe that no participant receives messages from two different senders. No Mixed Choice similarly holds trivially.

G_N is deadlock-free because the branch in the loop of G_t where $\mathtt{r}$ sends the *restart* message is always enabled. Moreover, it is easy to see that G_N is deterministic because each branch of a choice sends a different message value.

In summary, G_N is a GCLTS that is implementable iff N reaches M. The size of $\mathbf{G}_N$ is linear in the size of N, so we obtain the desired reduction. □

5 Extension to Network Architectures

In this section, we motivate how implementability depends on the model of asynchrony, specifically the network architecture of the assumed implementation model. We then introduce network-parametric global protocol semantics, CLTS and the implementability problem, and show how *CC* can be extended to solve the network-parametric setting.

5.1 Motivation

We revisit the non-implementable example from Fig. 1, for which we proposed one method of protocol repair in §3. Observe that, as a specification mechanism, Fig. 1does not a priori commit to a particular network model. While peer-to-peer FIFO communication is ubiquitous in practice, other network models also hold practical and theoretical interest. For example, Erlang and Go implement mailbox communication, and many correctness proofs of classic distributed algorithms such as leader election [21] and clock synchronization [37] rely on bag semantics. We consider the effect of the chosen network architecture in our original analysis. Perhaps surprisingly, replacing the peer-to-peer FIFO network by another asynchronous network architecture does not constitute a viable method of protocol repair. Non-implementability is solely due to the asynchronous nature of communication, specifically, the fact that the two send events $\mathtt{s} \triangleright \mathtt{w}_2!\mathsf{half}$ and $\mathtt{w}_1 \triangleright \mathtt{w}_2!\mathsf{half}$ in the $3c$ run do not causally depend on each other. They can therefore happen concurrently, causing the two messages to arrive at $\mathtt{w}_2$ in any order with arbitrary delays. Thus, the protocol is non-implementable for any asynchronous network architecture.

In general, however, implementability depends on the specific network architecture. To see this, observe that the repair proposed in §3 does not help establish implementability under the mailbox network architecture, in which all messages sent to the same recipient are collected in one FIFO buffer. Consider the execution u_3 under a mailbox network, following branch $3c$.

$$u_3 = \mathtt{s} \triangleright \mathtt{w}_1!\mathsf{full} \cdot \mathtt{w}_1 \triangleleft \mathtt{s}?\mathsf{full} \cdot \mathtt{w}_1 \triangleright \mathtt{w}_2!\mathsf{half} \cdot \mathtt{s} \triangleright \mathtt{w}_2!\mathsf{full}$$

In u_3 the full message sent to $\mathtt{w}_2$ by $\mathtt{s}$ is delayed by network asynchrony, and reordered after the half message sent to $\mathtt{w}_2$ by $\mathtt{w}_1$. Since $\mathtt{w}_2$'s mailbox buffers messages in FIFO order of arrival, this execution forces $\mathtt{w}_2$ to first receive half from $\mathtt{w}_1$ before being able to retrieve the one from $\mathtt{s}$ in the buffer. The issue with mailbox is that in the $3c$ branch, the network may still asynchronously reorder the two messages sent to $\mathtt{w}_2$ by delaying the message from $\mathtt{s}$. Since messages are buffered in FIFO order of arrival, this would force $\mathtt{w}_2$ to first receive the message from $\mathtt{w}_1$ before being able to retrieve the one from $\mathtt{s}$ in the buffer. The resulting execution violates

protocol fidelity. Note that this time, the ensuing violation of implementability has nothing to do with incomplete information by any of the protocol participants about what branch the other participants are following. Instead, it is solely due to the ability of the network architecture to reorder independent events in executions of protocol runs. Moreover, the violation cannot be avoided by changing the candidate local implementation. A possible repair of the protocol for mailbox networks is to introduce a causal dependency between $\mathtt{s} \triangleright \mathtt{w}_2!\mathsf{half}$ and $\mathtt{w}_1 \triangleright \mathtt{w}_2!\mathsf{half}$, e.g., by inserting an additional message exchange $\mathtt{s} \triangleright \mathtt{w}_1!\mathsf{done}$ between the two exchanges, forcing $\mathtt{w}_1$ to wait until $\mathtt{s}$ has sent its message to $\mathtt{w}_2$.

Finally, if we swap the network architecture from mailbox to *mailbag*, where the single FIFO queue per recipient is replaced by an unordered multiset, the protocol becomes implementable again even without the proposed second repair for mailbox.

5.2 Network-Parametric Framework

We generalize our distributed implementation model and global protocol semantics along the axis of network parametricity. First, we formalize the definition of network architectures $\mathbb{A}$.

Definition 20 (Network architecture)**.** A *network architecture* over a set of participants $\mathcal{P}$ and message values $\mathcal{V}$ is a tuple $\mathbb{A} = (\mathsf{Chan}, \mathsf{B}, \mathsf{bf}, \mathsf{ins}, \mathsf{rem}, b_0)$ where Chan is a set of channels, B a set of *channel contents* and $\mathsf{bf} : \mathcal{P} \times \mathcal{P} \to \mathsf{Chan}$ a map that associates each sender and receiver with a channel. Intuitively, $\mathsf{bf}(\mathtt{p}, \mathtt{q})$ denotes the message buffer to which messages sent from $\mathtt{p}$ to $\mathtt{q}$ are deposited. We refer to bf as the *communication topology* of $\mathbb{A}$. The set of *channel states* is $\Xi = \mathsf{Chan} \to \mathsf{B}$. Messages $m \in M$ in channel contents are tagged with their sender and receiver, i.e., $M = \mathcal{P} \times \mathcal{P} \times \mathcal{V}$. Channel contents are equipped with partial insert and remove operations, $\mathsf{ins}, \mathsf{rem} : M \to \mathsf{B} \to \mathsf{B}$, where $\mathsf{ins}(m)(b)$ being undefined indicates that b blocks on inserting m and $\mathsf{rem}(m)(b)$ is only defined when m is available for removal in b. Finally, $b_0 \in \mathsf{B}$ is the empty channel contents.

Example 21. We define eight concrete network architectures that we will revisit later. We consider four communication topologies: n-to-n, in which all senders and receivers share the same channel, one-to-n, in which receivers share the same channel to receive from a single sender, n-to-one, in which senders share the same channel to send to a single receiver, and one-to-one, in which each sender and receiver pair have a unique channel. We consider two message buffer data structures: ordered FIFO queues, and unordered multisets. The aforementioned network architectures often appear under the names of global bus (n-to-n FIFO), message soup (n-to-n multiset), and mailbox (one-to-n FIFO). Message soups are commonly found in leader election protocols such as Paxos and Raft, and one-to-n

FIFO is found in work stealing patterns in parallel programming. Finally, the one-to-one or peer-to-peer FIFO is the standard network architecture for CSMs, and widely assumed in the theory and practice of message-passing concurrency.

The four communication topologies are defined as follows (with our naming conventions given in parenthesis, where "B" refers to the name of one of the buffer types below):

- one-to-one (peer-to-peer B): $\mathsf{Chan} = \mathcal{P} \times \mathcal{P}, \mathsf{bf}(\mathtt{p}, \mathtt{q}) = (\mathtt{p}, \mathtt{q})$,
- one-to-n (senderB): $\mathsf{Chan} = \mathcal{P}, \mathsf{bf}(\mathtt{p}, \mathtt{q}) = \mathtt{p}$,
- n-to-one (mailB): $\mathsf{Chan} = \mathcal{P}, \mathsf{bf}(\mathtt{p}, \mathtt{q}) = \mathtt{q}$,
- n-to-n (monoB): $\mathsf{Chan} = \{0\}, \mathsf{bf}(\mathtt{p}, \mathtt{q}) = 0$,

and the two buffer types are FIFO queues (B=box), $\mathsf{B} = M^*$, and multisets (B=bag), $\mathsf{B} = M \to \mathbb{N}$. In the case of FIFO queues, *insert* corresponds to appending at the end of the queue, *remove* corresponds to removing from the head, and the empty buffer contents b_0 is ε; in the case of multisets, *insert* is multiset addition, *remove* multiset deletion, and $\emptyset$ is the empty buffer.

We note that the four network architectures with homogeneous bag channels are operationally equivalent and collapse to the monobag case: since messages are all labeled with their sender and receiver, one can "on demand" separate the message soup into $\mathcal{P}^2$ multisets, or $\mathcal{P}$ multisets by sender or receiver whenever messages are sent or received, and thus simulate the other network architectures. This leaves us with the four FIFO network architectures, which we refer to as `p2p`, `mb`, `sb`, and `monob` in the rest of the paper, and the collapsed case for bag channels (`bag`).

We generalize CLTS to be parametric in a choice of network architecture $\mathbb{A} = (\mathsf{Chan}, \mathsf{B}, \mathsf{bf}, \mathsf{ins}, \mathsf{rem}, b_0)$. The only change lies in the definition of the CLTS transition relation, in which we replace the hardcoded insert and remove operations for FIFO buffers with *insert* and *remove* defined in terms of ins and rem from $\mathbb{A}$ by lifting ins and rem to channel states $\xi \in \Xi$ of $\mathbb{A}$: we define $\mathit{insert}(\xi, \mathtt{p}, \mathtt{q}, m) = \xi'$ where $\xi'(\mathsf{bf}(\mathtt{p}, \mathtt{q})) = \mathsf{ins}(\mathtt{p}, \mathtt{q}, m)(\xi(\mathsf{bf}(\mathtt{p}, \mathtt{q})))$ and all other channel contents remain the same; we define $\mathit{remove}(\xi, \mathtt{p}, \mathtt{q}, m)$ analogously.

- $(\vec{s}, \xi) \xrightarrow{\mathtt{p} \triangleright \mathtt{q}!m} (\vec{s}', \xi')$ if $(\vec{s}_{\mathtt{p}}, \mathtt{p} \triangleright \mathtt{q}!m, \vec{s}'_{\mathtt{p}}) \in \delta_{\mathtt{p}}$, $\vec{s}_{\mathtt{r}} = \vec{s}'_{\mathtt{r}}$ for every participant $\mathtt{r} \neq \mathtt{p}$, $\xi' = \mathit{insert}(\xi, \mathtt{p}, \mathtt{q}, m)$.
- $(\vec{s}, \xi) \xrightarrow{\mathtt{q} \triangleleft \mathtt{p}?m} (\vec{s}', \xi')$ if $(\vec{s}_{\mathtt{q}}, \mathtt{q} \triangleleft \mathtt{p}?m, \vec{s}'_{\mathtt{q}}) \in \delta_{\mathtt{q}}$, $\vec{s}_{\mathtt{r}} = \vec{s}'_{\mathtt{r}}$ for every participant $\mathtt{r} \neq \mathtt{q}$, $\xi' = \mathit{remove}(\xi, \mathtt{p}, \mathtt{q}, m)$.

For example, in the CLTS with $T_{\mathtt{s}}, T_{\mathtt{w}_1}, T_{\mathtt{w}_2}$ defined by Figs. 2 to 4 in §1, and $\mathbb{A} = \mathtt{mb}$, the configuration reached on u_3 is a deadlock, whereas in the same CLTS but with $\mathbb{A} = \mathtt{p2p}$ or $\mathbb{A} = \mathtt{bag}$, the configuration reached on u_3 is not a deadlock.

Having defined $\mathbb{A}$ formally, the definition of channel compliance from §2 can now be instantiated for different network architectures beyond peer-to-peer FIFO. For example, u_1, u_2 from §3and u_3 above are $\mathbb{A}$-channel-compliant for all $\mathbb{A} = \mathtt{p2p}, \mathtt{mb}, \mathtt{sb}, \mathtt{monob}, \mathtt{bag}$. The prefix $\mathtt{s} \triangleright \mathtt{w}_1!\mathsf{full} \cdot \mathtt{w}_1 \triangleleft \mathtt{s}?\mathsf{full}$ of u_3 is channel-matched, $\mathtt{w}_1 \triangleleft \mathtt{s}?\mathsf{full} \cdot \mathtt{s} \triangleright \mathtt{w}_1!\mathsf{full}$ is not $\mathbb{A}$-channel-compliant for any choice of $\mathbb{A}$, and $u_3 \cdot \mathtt{w}_2 \triangleleft \mathtt{s}?\mathsf{full}$ is $\mathtt{p2p}$-channel-compliant but not $\mathtt{mb}$- or $\mathtt{monob}$-channel-compliant.

Our global protocol semantics is thus already network-parametric, as $\mathbb{A}$ is included as a parameter to the definition provided in §2. Defining the network-parametric implementability problem is likewise straightforward:

Definition 22 (Network-Parametric Protocol Implementability)**.** A protocol $\mathcal{S}$ is *implementable* under network architecture $\mathbb{A}$ if there exists a CLTS $\mathcal{T}_{\mathbb{A}} = \{T_{\mathtt{p}}\}_{\mathtt{p}\in\mathcal{P}}$ such that the following two properties hold: (i) *protocol fidelity*: $\mathcal{L}(\{T_{\mathtt{p}}\}_{\mathtt{p}\in\mathcal{P}}) = C^{\sim}_{\mathbb{A}}(\mathcal{S})$, and (ii) *deadlock freedom*: $\{T_{\mathtt{p}}\}_{\mathtt{p}\in\mathcal{P}}$ is deadlock-free. We say that $\{T_{\mathtt{p}}\}_{\mathtt{p}\in\mathcal{P}}$ implements $\mathcal{S}$ under $\mathbb{A}$.

5.3 Network-Parametric Coherence Conditions

Next, we present the generalization of *CC* to the network-parametric setting. We focus on the delta between *CC* for $\mathtt{p2p}$ and for the network-parametric setting, which consists of a modular modification of Receive Coherence, and the addition of a brand new condition, Prefix Extensibility. Send Coherence and No Mixed Choice remain sound and complete in their original form across all network architectures.

Generalized Receive Coherence. The key observation is that irrespective of network architecture, it is sufficient and necessary to guarantee that no two distinct receptions are simultaneously enabled for any participant from any configuration. Furthermore, the notion of a message m being receivable by $\mathtt{q}$ from $\mathtt{p}$ after executing a word w can be captured in a network-parametric way, as the channel compliance of $w \cdot \mathtt{q} \triangleleft \mathtt{p}?m$.

Definition 23 (Generalized Receive Coherence [47])**.** A protocol $\mathcal{S}$ satisfies *Generalized Receive Coherence* (GRC) when for every two simultaneously reachable states for $\mathtt{q}$, s_1 and s_2, with transitions $s_1 \xrightarrow{\mathtt{p}\to\mathtt{q}:m_1} s_1'$ and $s_2 \xrightarrow{\mathtt{r}\to\mathtt{q}:m_2} s_2'$, if $\mathtt{r} \neq \mathtt{p}$ or $m_2 \neq m_1$, then for any $w \in \mathrm{pref}(\mathcal{L}(\mathcal{S}_{s_2}))$ such that $w{\Downarrow_{\Sigma_{\mathtt{q}}}} = \varepsilon$ and $\mathtt{r} \triangleright \mathtt{q}!m_2 \leq w{\Downarrow_{\Sigma_{\mathtt{r}}}}$, the extension $w \cdot \mathtt{q} \triangleleft \mathtt{p}?m_1$ is not channel-compliant.

Generalized Receive Coherence differs from RC for $\mathtt{p2p}$ in two ways: w is a protocol prefix from s_2 rather than s_2', and the condition requires an explicit send from $\mathtt{r}$ to appear in w. For $\mathtt{p2p}$ channel compliance, the two formulations are equivalent because per-sender channels are independent; retaining the more general form allows handling $\mathtt{mb}$ and $\mathtt{monob}$ without ruling them out a priori.

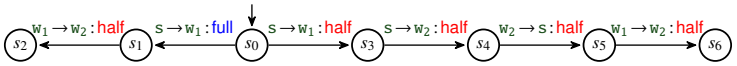


Figure 11: Subprotocol of task scheduler from Fig. 1, containing parts of branch 1*a* (left) and branch 3*c* (right).

Prefix Extensibility. Our new condition, called Prefix Extensibility, captures a source of non-implementability that never arises in `p2p` networks but arises in other architectures. To illustrate this new source of non-implementability, consider the simple straight-line global specification consisting only of the synchronous word $\mathtt{p}_1 \to \mathtt{q} : m \cdot \mathtt{p}_2 \to \mathtt{q} : m$. Despite its simplicity, there does not exist a deadlock-free `mb` (or `monob`) CLTS that implements this specification. Any candidate CLTS must exhibit the following deadlocking trace: $\mathtt{p}_2 \triangleright \mathtt{q}!m \cdot \mathtt{p}_1 \triangleright \mathtt{q}!m$. Because the network can reorder $\mathtt{p}_2$'s send event before $\mathtt{p}_1$'s send event, yet the local actions of `q` tell it to receive in the opposite order, the only way for the CLTS to not deadlock is for $T_{\mathtt{q}}$ to admit the local trace $\mathtt{q} \triangleleft \mathtt{p}_2?m \cdot \mathtt{q} \triangleleft \mathtt{p}_1?m$.

To rule out such cases of non-implementability, Prefix Extensibility requires that protocol prefixes are closed under per-participant equality.

Definition 24 (Prefix Extensibility [47])**.** A protocol $\mathcal{S}$ satisfies *Prefix Extensibility* (PE) under $\mathbb{A}$ when for every $\rho \in \mathrm{pref}(\mathcal{L}(\mathcal{S}))$ and every $\mathbb{A}$-channel-compliant $w \in \Sigma^*_{async}$ that agrees with ρ per participant, there exists $u \in \Sigma^*_{async}$ such that wu is channel-compliant and $wu \equiv_{\mathcal{P}} \mathtt{split}(\rho)$.

Prefix Extensibility holds trivially for *any* protocol under `p2p`, `sb`, and `bag`: for these architectures, any channel-compliant word agreeing with a synchronous trace can be extended to complete that trace while preserving channel compliance [47]. For `mb` and `monob`, Prefix Extensibility must be checked on a given protocol's runs.

In Fig. 11, depicting a subprotocol of the task scheduling protocol from Fig. 1, the protocol run corresponding to the bottom branch violates Prefix Extensibility.

Our network-parametric characterization of implementability is thus defined as the conjunction of four conditions.

Definition 25 (Generalized Coherence Conditions)**.** A protocol $\mathcal{S}$ satisfies Generalized Coherence Conditions under network architecture $\mathbb{A}$ when it satisfies Send Coherence, Generalized Receive Coherence, No Mixed Choice, and Prefix Extensibility.

5.4 Channel Compliance Axioms

In this section, we provide sufficient conditions under which our Generalized Coherence Conditions are precise for a given network architecture. These conditions

describe network architectures, which we define semantically as channel compliance definitions, and thus we refer to the conditions as *channel compliance axioms*. A network architecture $\mathbb{A} \in \mathfrak{A}$ satisfies our axiomatic network model if $\mathbb{A}$ satisfies **F1** through **F6**:

Definition 26 (Channel compliance axioms)**.** Let $w \in \Sigma^*_{async}$ be channel-compliant.

- **F1** For all $x \in \Sigma_!$, wx is channel-compliant.
- **F2** For all $\mathtt{p} \neq \mathtt{q} \in \mathcal{P}$ and $m \in \mathcal{V}$, $|w{\Downarrow_{\mathtt{p}\triangleright\mathtt{q}!m}}| \geq |w{\Downarrow_{\mathtt{q}\triangleleft\mathtt{p}?m}}|$.
- **F3** For all $\rho \in \Gamma^*_{sync}$, $\mathtt{split}(\rho)$ is channel-compliant.
- **F4** For all $\rho \in \Gamma^*_{sync}$, if $w \equiv_{\mathcal{P}} \mathtt{split}(\rho)$, then w is channel-matched.
- **F5** For all $x \in \Sigma_!$, $y \in \Sigma_?$, if wy is channel-compliant then wxy is channel-compliant.
- **F6** Let $\alpha, \beta \in \Gamma^*_{sync}$, $\mathtt{p} \neq \mathtt{q} \in \mathcal{P}$ and $m \in \mathcal{V}$ such that for all $\mathtt{p} \in \mathcal{P}$, $w{\Downarrow_{\Sigma_\mathtt{p}}} \leq \mathtt{split}(\alpha\beta){\Downarrow_{\Sigma_\mathtt{p}}}$, and $w{\Downarrow_{\Sigma_\mathtt{q}}} = \mathtt{split}(\alpha){\Downarrow_{\Sigma_\mathtt{q}}}$, and $w{\cdot}\mathtt{q}\triangleleft\mathtt{p}?m$ is channel-compliant. Then, there exists w' such that w' is compliant with β, $w'{\Downarrow_{\Sigma_\mathtt{q}}} = \varepsilon$ and $w' \cdot \mathtt{q} \triangleleft \mathtt{p}?m$ is channel-compliant.

FIFO queues and multisets both satisfy all eight axioms. Therefore, all five considered architectures are in $\mathfrak{A}$. Heterogeneous architectures mixing FIFO and bag channels also satisfy the axioms. However, priority queues, channels with duplication, and bounded channels are ruled out.

The following theorem states that our Generalized Coherence Conditions are sound and complete with respect to implementability, assuming a network architecture that satisfies our channel compliance axioms. The proof of Theorem 27 is fully mechanized in Rocq, and builds on the mechanization of Theorem 5.

Theorem 27 (Preciseness of Generalized Coherence Conditions)**.** *Let $\mathbb{A}$ be a network architecture satisfying* **F1** *through* **F6** *and let $\mathcal{S}$ be a global protocol. Then, $\mathcal{S}$ is implementable under $\mathbb{A}$ if and only if it satisfies Generalized Coherence Conditions under $\mathbb{A}$.*

5.5 Derived Results

The matching complexity bounds, synthesis result, and generalization to infinite-state protocols via μCLP encodings all transfer to the network-parametric setting. Here, we briefly discuss selected takeaways.

First, the network-parametric generalization reinforces the separation between synthesis and implementability: once implementability is established, the same

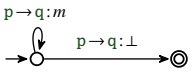


Figure 12: A protocol $\mathcal{S}_a$ that is not bag-implementable but p2p-implementable.

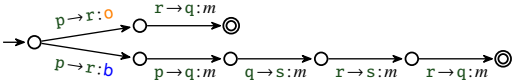


Figure 13: A protocol $\mathcal{S}_b$ that is not p2p-implementable but sb-implementable.

canonical implementation suffices to implement it. Second, the μCLP encoding of Generalized Receive Coherence yields the following relationships between the classes of implementable global protocols under each network architecture. The strictness is witnessed by the protocols $\mathcal{S}_a$ and $\mathcal{S}_b$ depicted in Figs. 12 to 13.

Lemma 28 (Implementability relationships)**.** *Any* bag*-implementable protocol is* p2p*-implementable, and any* p2p*-implementable protocol is* sb*-implementable. Both inclusions are strict.*

Third, on complexity, the hardness reduction relies on program order and not network reorderings for Receive Coherence, and thus the same lower bound construction with a small modification works for the remaining network architectures. In order to show that the construction carries over to bag and senderbox networks, one is only required to analyze the unique message receptions that appear in each gadget. Similarly, the directed choice case falls out for free.

Finally, we conclude this section by mentioning that [47] takes network parametricity a step further by reducing the channel compliance axioms to *buffer axioms*, which allows one to obtain heterogeneous channel architectures for free, as well as not having to reprove the channel compliance axioms from e.g. senderbox to mailbox. As this result is not included in the author's thesis, it is omitted here.

6 Tool Implementation and Mechanization

6.1 The Sprout Tool

All decision procedures for implementability discussed in this paper are implemented in an open-source tool called Sprout, which is hosted at `https://github.com/nyu-acsys/sprout`. We refer the reader to [45] for details about Sprout's implementation, including optimizations to the μCLP encodings presented in §4 that make validity checking tractable for Sprout's backend solver MuVal. [45] also describes μCLP encodings for checking whether a symbolic protocol falls into the GCLTS fragment. We additionally refer the reader to Sprout's source code for a comprehensive benchmark suite of communication protocols drawn from the MST literature.

6.2 Rocq Mechanization

We conclude by discussing a subtle bug in the infinite-word semantics of global protocols uncovered during the mechanization of Theorem 5, which originated from [49] and is present in several subsequent works [41, 42] and preprints [44]. The bug resulted in a definition of global protocol semantics that is spuriously non-implementable due to selectively excluding infinite traces that are indistinguishable to a CLTS. We say that two words $w, w' \in \Sigma^{\omega}_{async}$ are *indistinguishable* to a CLTS when w is a CLTS trace iff w' is a CLTS trace. Note that indistinguishability is network-specific: two words that are `p2p`-indistinguishable may not be `mb`-indistinguishable.

Indistinguishability of pairs of finite words is straightforward to define, given that w and w' must be permutations of each other, and moreover, the total order of events for each participant must be identical. Thus, to show that *any* CLTS executes w', we only require that each participant's local implementation accepts $w{\Downarrow_{\Sigma_\mathtt{p}}}$, which is given from the fact that the CLTS recognizes w. For infinite words, however, indistinguishability can no longer be defined purely alphabetically, due to the possibility for a CLTS to infinitely reorder independent events, achieving the effect of indefinite delay. For example, the pair of infinite words $v_1 = \mathtt{p} \triangleright \mathtt{q}!m^{\omega}$ and $v_2 = \mathtt{r} \triangleright \mathtt{s}!m \cdot \mathtt{p} \triangleright \mathtt{q}!m^{\omega}$ are not indistinguishable in general because v_1 provides no information about the local implementation of participant `r`, yet to know that v_2 is a trace of a given CLTS requires us to know that participant `r`'s local implementation recognizes the trace $\mathtt{r} \triangleright \mathtt{s}!m$.

We show by counterexample that the flawed definition of infinite protocol semantics, given below, is not closed under CLTS indistinguishability of infinite words.

$$\begin{aligned}\overline{C}^{\sim}_{inf}(\mathcal{S}) = \{w' \in \Sigma^{\omega}_{async} \mid \exists w \in \Sigma^{\omega}_{async}.\, w \in \mathtt{split}(\mathcal{L}(\mathcal{S})) \wedge \forall v' \leq w'.\, \exists u, u' \in \Sigma^{*}_{async}.\\ v' \cdot u' \text{ is channel-compliant } \wedge\ u \leq w\ \wedge \forall \mathtt{p} \in \mathcal{P}.\, (v' \cdot u'){\Downarrow_{\Sigma_\mathtt{p}}} = u{\Downarrow_{\Sigma_\mathtt{p}}}\}\ .\end{aligned}$$

Consider the simple protocol depicted in Fig. 14, involving four participants `p`, `q`, `r`, `s` and two message values m, m'. Fig. 15 depicts a protocol $\mathcal{S}_d$ obtained by a simple state renaming of $\mathcal{S}_c$ that is semantically identical to $\mathcal{S}_c$. Clearly, both protocols are implementable, and moreover by the same canonical CLTS, which admits the infinite trace $\overline{v} = \mathtt{r} \triangleright \mathtt{s}!m \cdot \mathtt{p} \triangleright \mathtt{q}!m^{\omega}$. We should thus expect that $\overline{v}$ is included in the infinite semantics of both $\mathcal{S}_c$ and $\mathcal{S}_d$. Unfortunately, per the above definition, $\overline{v}$ is included in $\overline{C}^{\sim}_{inf}(\mathcal{S}_d)$ but excluded by $\overline{C}^{\sim}_{inf}(\mathcal{S}_c)$: in $\mathcal{S}_c$, there exists no infinite run that includes any events by `r` or `s`, and thus no existential witness for w that satisfies the subsequent conjuncts.

Fortunately, the correction amounts to simply swapping the first two quantifiers in $\overline{C}^{\sim}_{inf}(\mathcal{S})$, see the definition in §2. The correct definition weakens the requirement that all finite prefixes of a infinite CLTS trace must be consistent with a *single*

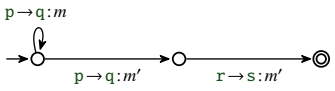


Figure 14: Example protocol with infinite words $\mathcal{S}_c$.

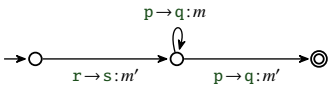


Figure 15: Example protocol with infinite words $\mathcal{S}_d$.

infinite global protocol run. Instead, each finite prefix may be consistent with a different finite run *prefix*, which may in turn be part of a finite or infinite maximal run in $\mathcal{S}$. The revised definition more closely matches simulation-based notions of trace equivalence, for example in [75], and incurred minimal requisite changes to proofs. We refer the reader to [46] for more details.

Funding Acknowledgement. This work was supported by the National Science Foundation under grant agreement 2304758.

References

[1] Alur, R., Bodík, R., Juniwal, G., Martin, M.M.K., Raghothaman, M., Seshia, S.A., Singh, R., Solar-Lezama, A., Torlak, E., Udupa, A.: Syntax-guided synthesis. In: Formal Methods in Computer-Aided Design, FMCAD 2013, Portland, OR, USA, October 20-23, 2013. pp. 1–8. IEEE (2013), https://ieeexplore.ieee.org/document/6679385/

[2] Alur, R., Etessami, K., Yannakakis, M.: Inference of message sequence charts. IEEE Trans. Software Eng. **29**(7), 623–633 (2003), https://doi.org/10.1109/TSE.2003.1214326

[3] Bertrand, N., Bouyer, P., Brihaye, T., Carlier, P.: When are stochastic transition systems tameable? J. Log. Algebraic Methods Program. **99**, 41–96 (2018), https://doi.org/10.1016/j.jlamp.2018.03.004

[4] Bertrand, N., Stainer, A., Jéron, T., Krichen, M.: A game approach to determinize timed automata. Formal Methods Syst. Des. **46**(1), 42–80 (2015), https://doi.org/10.1007/s10703-014-0220-1

[5] Brand, D., Zafiropulo, P.: On communicating finite-state machines. J. ACM **30**(2), 323–342 (1983), https://doi.org/10.1145/322374.322380

[6] Castro-Perez, D., Hu, R., Jongmans, S., Ng, N., Yoshida, N.: Distributed programming using role-parametric session types in go: statically-typed endpoint apis for dynamically-instantiated communication structures. Proc. ACM Program. Lang. **3**(POPL), 29:1–29:30 (2019), https://doi.org/10.1145/3290342

[7] Castro-Perez, D., Yoshida, N.: Dynamically updatable multiparty session protocols: Generating concurrent go code from unbounded protocols. In: Ali, K., Salvaneschi, G. (eds.) 37th European Conference on Object-Oriented Programming, ECOOP 2023, July 17-21, 2023, Seattle, Washington, United States. LIPIcs, vol. 263, pp. 6:1–6:30. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2023), https://doi.org/10.4230/LIPIcs.ECOOP.2023.6

[8] Chaudhuri, S., Ellis, K., Polozov, O., Singh, R., Solar-Lezama, A., Yue, Y.: Neurosymbolic programming. Found. Trends Program. Lang. **7**(3), 158–243 (2021). https://doi.org/10.1561/2500000049

[9] Church, A.: Applications of recursive arithmetic to the problem of circuit synthesis. In: Summaries of Talks Presented at the Summer Institute for Symbolic Logic. pp. 3–50. Cornell University (1957)

[10] Clemente, L., Lasota, S., Piórkowski, R.: Determinisability of register and timed automata. Log. Methods Comput. Sci. **18**(2) (2022), https://doi.org/10.46298/lmcs-18(2:9)2022

[11] Coppo, M., Dezani-Ciancaglini, M., Padovani, L., Yoshida, N.: A gentle introduction to multiparty asynchronous session types. In: Bernardo, M., Johnsen, E.B. (eds.) Formal Methods for Multicore Programming - 15th International School on Formal Methods for the Design of Computer, Communication, and Software Systems, SFM 2015, Bertinoro, Italy, June 15-19, 2015, Advanced Lectures. Lecture Notes in Computer Science, vol. 9104, pp. 146–178. Springer (2015), https://doi.org/10.1007/978-3-319-18941-3_4

[12] Cruz-Filipe, L., Montesi, F.: Choreographies in practice. In: Albert, E., Lanese, I. (eds.) Formal Techniques for Distributed Objects, Components, and Systems - 36th IFIP WG 6.1 International Conference, FORTE 2016, Held as Part of the 11th International Federated Conference on Distributed Computing Techniques, DisCoTec 2016, Heraklion, Crete, Greece, June 6-9, 2016, Proceedings. Lecture Notes in Computer Science, vol. 9688, pp. 114–123. Springer (2016), https://doi.org/10.1007/978-3-319-39570-8_8

[13] Cruz-Filipe, L., Montesi, F.: A core model for choreographic programming. Theor. Comput. Sci. **802**, 38–66 (2020), https://doi.org/10.1016/j.tcs.2019.07.005

[14] Cutner, Z., Yoshida, N., Vassor, M.: Deadlock-free asynchronous message reordering in rust with multiparty session types. In: Lee, J., Agrawal, K., Spear, M.F. (eds.) PPoPP '22: 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, Seoul, Republic of Korea, April 2 - 6, 2022. pp. 246–261. ACM (2022), https://doi.org/10.1145/3503221.3508404

[15] D'Antoni, L., Veanes, M.: The power of symbolic automata and transducers. In: Majumdar, R., Kuncak, V. (eds.) Computer Aided Verification - 29th International Conference, CAV 2017, Heidelberg, Germany, July 24-28, 2017, Proceedings, Part

I. Lecture Notes in Computer Science, vol. 10426, pp. 47–67. Springer (2017), https://doi.org/10.1007/978-3-319-63387-9_3

[16] Demangeon, R., Honda, K., Hu, R., Neykova, R., Yoshida, N.: Practical interruptible conversations: distributed dynamic verification with multiparty session types and python. Formal Methods Syst. Des. **46**(3), 197–225 (2015), https://doi.org/10.1007/s10703-014-0218-8

[17] Ellul, K., Krawetz, B., Shallit, J.O., Wang, M.: Regular expressions: New results and open problems. J. Autom. Lang. Comb. **10**(4), 407–437 (2005), https://doi.org/10.25596/jalc-2005-407

[18] Esparza, J., Nielsen, M.: Decidability issues for petri nets - a survey. J. Inf. Process. Cybern. **30**(3), 143–160 (1994)

[19] Fähndrich, M., Aiken, M., Hawblitzel, C., Hodson, O., Hunt, G.C., Larus, J.R., Levi, S.: Language support for fast and reliable message-based communication in singularity OS. In: Berbers, Y., Zwaenepoel, W. (eds.) Proceedings of the 2006 EuroSys Conference, Leuven, Belgium, April 18-21, 2006. pp. 177–190. ACM (2006), https://doi.org/10.1145/1217935.1217953

[20] Finkbeiner, B., Schewe, S.: Uniform distributed synthesis. In: 20th IEEE Symposium on Logic in Computer Science (LICS 2005), 26-29 June 2005, Chicago, IL, USA, Proceedings. pp. 321–330. IEEE Computer Society (2005), https://doi.org/10.1109/LICS.2005.53

[21] Gallager, R.G., Humblet, P.A., Spira, P.M.: A distributed algorithm for minimum-weight spanning trees. ACM Trans. Program. Lang. Syst. **5**(1), 66–77 (1983), https://doi.org/10.1145/357195.357200

[22] Gancher, J., Gibson, S., Singh, P., Dharanikota, S., Parno, B.: Owl: Compositional verification of security protocols via an information-flow type system. In: 44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023. pp. 1130–1147. IEEE (2023), https://doi.org/10.1109/SP46215.2023.10179477

[23] Gazagnaire, T., Genest, B., Hélouët, L., Thiagarajan, P.S., Yang, S.: Causal message sequence charts. In: Caires, L., Vasconcelos, V.T. (eds.) CONCUR 2007 - Concurrency Theory, 18th International Conference, CONCUR 2007, Lisbon, Portugal, September 3-8, 2007, Proceedings. Lecture Notes in Computer Science, vol. 4703, pp. 166–180. Springer (2007), https://doi.org/10.1007/978-3-540-74407-8_12

[24] Genest, B., Muscholl, A.: Message sequence charts: A survey. In: Fifth International Conference on Application of Concurrency to System Design (ACSD 2005), 6-9 June 2005, St. Malo, France. pp. 2–4. IEEE Computer Society (2005), https://doi.org/10.1109/ACSD.2005.25

[25] Genest, B., Muscholl, A., Peled, D.A.: Message sequence charts. In: Desel, J., Reisig, W., Rozenberg, G. (eds.) Lectures on Concurrency and Petri Nets, Advances

in Petri Nets [This tutorial volume originates from the 4th Advanced Course on Petri Nets, ACPN 2003, held in Eichstätt, Germany in September 2003. In addition to lectures given at ACPN 2003, additional chapters have been commissioned]. Lecture Notes in Computer Science, vol. 3098, pp. 537–558. Springer (2003), https://doi.org/10.1007/978-3-540-27755-2_15

[26] Giallorenzo, S., Montesi, F., Peressotti, M.: Choral: Object-oriented choreographic programming. ACM Trans. Program. Lang. Syst. **46**(1), 1:1–1:59 (2024), https://doi.org/10.1145/3632398

[27] Godefroid, P., Yannakakis, M.: Analysis of boolean programs. In: Piterman, N., Smolka, S.A. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 19th International Conference, TACAS 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16-24, 2013. Proceedings. Lecture Notes in Computer Science, vol. 7795, pp. 214–229. Springer (2013), https://doi.org/10.1007/978-3-642-36742-7_16

[28] Honda, K., Marques, E.R.B., Martins, F., Ng, N., Vasconcelos, V.T., Yoshida, N.: Verification of MPI programs using session types. In: Träff, J.L., Benkner, S., Dongarra, J.J. (eds.) Recent Advances in the Message Passing Interface - 19th European MPI Users' Group Meeting, EuroMPI 2012, Vienna, Austria, September 23-26, 2012. Proceedings. Lecture Notes in Computer Science, vol. 7490, pp. 291–293. Springer (2012), https://doi.org/10.1007/978-3-642-33518-1_37

[29] Honda, K., Yoshida, N., Carbone, M.: Multiparty asynchronous session types. In: Necula, G.C., Wadler, P. (eds.) Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2008, San Francisco, California, USA, January 7-12, 2008. pp. 273–284. ACM (2008), https://doi.org/10.1145/1328438.1328472

[30] Hu, R., Yoshida, N.: Hybrid session verification through endpoint API generation. In: Stevens, P., Wasowski, A. (eds.) Fundamental Approaches to Software Engineering - 19th International Conference, FASE 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2-8, 2016, Proceedings. Lecture Notes in Computer Science, vol. 9633, pp. 401–418. Springer (2016), https://doi.org/10.1007/978-3-662-49665-7_24

[31] Hu, R., Yoshida, N.: Explicit connection actions in multiparty session types. In: Huisman, M., Rubin, J. (eds.) Fundamental Approaches to Software Engineering - 20th International Conference, FASE 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22-29, 2017, Proceedings. Lecture Notes in Computer Science, vol. 10202, pp. 116–133. Springer (2017), https://doi.org/10.1007/978-3-662-54494-5_7

[32] Imai, K., Neykova, R., Yoshida, N., Yuen, S.: Multiparty session programming with global protocol combinators. In: Hirschfeld, R., Pape, T. (eds.) 34th European Conference on Object-Oriented Programming, ECOOP 2020, November 15-17,

2020, Berlin, Germany (Virtual Conference). LIPIcs, vol. 166, pp. 9:1–9:30. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2020), https://doi.org/10.4230/LIPIcs.ECOOP.2020.9

[33] International Telecommunication Union: ITU-T Recommendation Z.120: Message Sequence Chart (MSC). ITU-T Recommendation Z.120, International Telecommunication Union, Geneva (February 2011), https://www.itu.int/rec/T-REC-Z.120-201102-I/en

[34] Kashiwa, S., Shen, G., Zare, S., Kuper, L.: Portable, efficient, and practical library-level choreographic programming (2023), https://arxiv.org/abs/2311.11472

[35] Kupferman, O., Vardi, M.Y.: Synthesizing distributed systems. In: 16th Annual IEEE Symposium on Logic in Computer Science, Boston, Massachusetts, USA, June 16-19, 2001, Proceedings. pp. 389–398. IEEE Computer Society (2001), https://doi.org/10.1109/LICS.2001.932514

[36] Lagaillardie, N., Neykova, R., Yoshida, N.: Stay safe under panic: Affine rust programming with multiparty session types (artifact). Dagstuhl Artifacts Ser. **8**(2), 09:1–09:16 (2022), https://doi.org/10.4230/DARTS.8.2.9

[37] Lamport, L.: Time, clocks, and the ordering of events in a distributed system. In: Concurrency: the Works of Leslie Lamport, pp. 179–196 (2019), https://doi.org/10.1145/3335772.3335934

[38] Lange, J., Ng, N., Toninho, B., Yoshida, N.: A static verification framework for message passing in go using behavioural types. In: Chaudron, M., Crnkovic, I., Chechik, M., Harman, M. (eds.) Proceedings of the 40th International Conference on Software Engineering, ICSE 2018, Gothenburg, Sweden, May 27 - June 03, 2018. pp. 1137–1148. ACM (2018), https://doi.org/10.1145/3180155.3180157

[39] Languages, Systems, and Data Lab, UC Santa Cruz: Chorus: Choreographic programming in rust. https://lsd-ucsc.github.io/ChoRus/introduction.html, accessed: 2025-07-10

[40] Li, E.: Decision Problems for Global Protocol Specifications. Ph.D. thesis (2025), https://www.proquest.com/dissertations-theses/decision-problems-global-protocol-specifications/docview/3255120134/se-2, copyright - Database copyright ProQuest LLC; ProQuest does not claim copyright in the individual underlying works; Last updated - 2025-12-06

[41] Li, E., Stutz, F., Wies, T.: Deciding subtyping for asynchronous multiparty sessions. In: Weirich, S. (ed.) Programming Languages and Systems - 33rd European Symposium on Programming, ESOP 2024, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2024, Luxembourg City, Luxembourg, April 6-11, 2024, Proceedings, Part I. Lecture Notes in Computer Science, vol. 14576, pp. 176–205. Springer (2024), https://doi.org/10.1007/978-3-031-57262-3_8

[42] Li, E., Stutz, F., Wies, T., Zufferey, D.: Complete multiparty session type projection with automata. In: Enea, C., Lal, A. (eds.) Computer Aided Verification - 35th International Conference, CAV 2023, Paris, France, July 17-22, 2023, Proceedings, Part III. Lecture Notes in Computer Science, vol. 13966, pp. 350–373. Springer (2023), https://doi.org/10.1007/978-3-031-37709-9_17

[43] Li, E., Stutz, F., Wies, T., Zufferey, D.: Characterizing implementability of global protocols with infinite states and data. Proc. ACM Program. Lang. **9**(OOPSLA1), 1434–1463 (2025), https://doi.org/10.1145/3720493

[44] Li, E., Stutz, F., Wies, T., Zufferey, D.: Characterizing implementability of global protocols with infinite states and data (2025), https://arxiv.org/abs/2411.05722

[45] Li, E., Stutz, F., Wies, T., Zufferey, D.: Sprout: A verifier for symbolic multiparty protocols (2025), https://doi.org/10.1007/978-3-031-98682-6, 1st edition, ISBN for softcover and eBook

[46] Li, E., Wies, T.: Certified implementability of global multiparty protocols. In: Forster, Y., Keller, C. (eds.) 16th International Conference on Interactive Theorem Proving, ITP 2025, Reykjavik, Iceland, September 28 - October 1, 2025. pp. 15:1–15:20. LIPIcs, Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2025), https://doi.org/10.4230/LIPIcs.ITP.2025.15

[47] Li, E., Wies, T.: Implementability of global distributed protocols modulo network architectures. CoRR **abs/2602.10320** (2026), https://doi.org/10.48550/arXiv.2602.10320

[48] Lohrey, M.: Realizability of high-level message sequence charts: closing the gaps. Theor. Comput. Sci. **309**(1-3), 529–554 (2003), https://doi.org/10.1016/j.tcs.2003.08.002

[49] Majumdar, R., Mukund, M., Stutz, F., Zufferey, D.: Generalising projection in asynchronous multiparty session types. In: Haddad, S., Varacca, D. (eds.) 32nd International Conference on Concurrency Theory, CONCUR 2021, August 24-27, 2021, Virtual Conference. LIPIcs, vol. 203, pp. 35:1–35:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2021), https://doi.org/10.4230/LIPIcs.CONCUR.2021.35

[50] Majumdar, R., Pirron, M., Yoshida, N., Zufferey, D.: Motion session types for robotic interactions (brave new idea paper). In: Donaldson, A.F. (ed.) 33rd European Conference on Object-Oriented Programming, ECOOP 2019, July 15-19, 2019, London, United Kingdom. LIPIcs, vol. 134, pp. 28:1–28:27. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2019), https://doi.org/10.4230/LIPIcs.ECOOP.2019.28

[51] Majumdar, R., Yoshida, N., Zufferey, D.: Multiparty motion coordination: from choreographies to robotics programs. Proc. ACM Program. Lang. **4**(OOPSLA), 134:1–134:30 (2020), https://doi.org/10.1145/3428202

[52] Mauw, S., Reniers, M.A.: High-level message sequence charts. In: Cavalli, A.R., Sarma, A. (eds.) SDL '97 Time for Testing, SDL, MSC and Trends - 8th International SDL Forum, Evry, France, 23-29 September 1997, Proceedings. pp. 291–306. Elsevier (1997)

[53] Montesi, F.: Introduction to Choreographies. Cambridge University Press (2023). https://doi.org/10.1017/9781108981491

[54] de Muijnck-Hughes, J., Vanderbauwhede, W.: A Typing Discipline for Hardware Interfaces. In: Donaldson, A.F. (ed.) 33rd European Conference on Object-Oriented Programming (ECOOP 2019). Leibniz International Proceedings in Informatics (LIPIcs), vol. 134, pp. 6:1–6:27. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2019), https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2019.6

[55] Neykova, R., Bocchi, L., Yoshida, N.: Timed runtime monitoring for multiparty conversations. Formal Aspects Comput. **29**(5), 877–910 (2017), https://doi.org/10.1007/s00165-017-0420-8

[56] Neykova, R., Hu, R., Yoshida, N., Abdeljallal, F.: A session type provider: compile-time API generation of distributed protocols with refinements in f#. In: Dubach, C., Xue, J. (eds.) Proceedings of the 27th International Conference on Compiler Construction, CC 2018, February 24-25, 2018, Vienna, Austria. pp. 128–138. ACM (2018), https://doi.org/10.1145/3178372.3179495

[57] Neykova, R., Yoshida, N.: Multiparty session actors. Log. Methods Comput. Sci. **13**(1) (2017), https://doi.org/10.23638/LMCS-13(1:17)2017

[58] Ng, N., Yoshida, N., Honda, K.: Multiparty session C: safe parallel programming with message optimisation. In: Furia, C.A., Nanz, S. (eds.) Objects, Models, Components, Patterns - 50th International Conference, TOOLS 2012, Prague, Czech Republic, May 29-31, 2012. Proceedings. Lecture Notes in Computer Science, vol. 7304, pp. 202–218. Springer (2012), https://doi.org/10.1007/978-3-642-30561-0_15

[59] Niu, X., Ng, N., Yuki, T., Wang, S., Yoshida, N., Luk, W.: EURECA compilation: Automatic optimisation of cycle-reconfigurable circuits. In: Ienne, P., Najjar, W.A., Anderson, J.H., Brisk, P., Stechele, W. (eds.) 26th International Conference on Field Programmable Logic and Applications, FPL 2016, Lausanne, Switzerland, August 29 - September 2, 2016. pp. 1–4. IEEE (2016), https://doi.org/10.1109/FPL.2016.7577359

[60] Object Management Group: Unified Modeling Language (UML) Website. https://www.uml.org/, accessed: 2025-07-10

[61] Palma, G.D., Giallorenzo, S., Mauro, J., Trentin, M., Zavattaro, G.: Towards a function-as-a-service choreographic programming language: Examples and applications (2024), https://arxiv.org/abs/2406.09099

[62] Pnueli, A., Rosner, R.: On the synthesis of a reactive module. In: Conference Record of the Sixteenth Annual ACM Symposium on Principles of Programming

Languages, Austin, Texas, USA, January 11-13, 1989. pp. 179–190. ACM Press (1989), https://doi.org/10.1145/75277.75293

[63] Pnueli, A., Rosner, R.: On the synthesis of an asynchronous reactive module. In: Ausiello, G., Dezani-Ciancaglini, M., Rocca, S.R.D. (eds.) Automata, Languages and Programming, 16th International Colloquium, ICALP89, Stresa, Italy, July 11-15, 1989, Proceedings. pp. 652–671. Lecture Notes in Computer Science, Springer (1989), https://doi.org/10.1007/BFb0035790

[64] Roychoudhury, A., Goel, A., Sengupta, B.: Symbolic message sequence charts. ACM Trans. Softw. Eng. Methodol. **21**(2), 12:1–12:44 (2012), https://doi.org/10.1145/2089116.2089122

[65] Scalas, A., Dardha, O., Hu, R., Yoshida, N.: A linear decomposition of multiparty sessions for safe distributed programming. In: Müller, P. (ed.) 31st European Conference on Object-Oriented Programming, ECOOP 2017, June 19-23, 2017, Barcelona, Spain. LIPIcs, vol. 74, pp. 24:1–24:31. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2017), https://doi.org/10.4230/LIPIcs.ECOOP.2017.24

[66] Shen, G., Kashiwa, S., Kuper, L.: Haschor: Functional choreographic programming for all (functional pearl). CoRR **abs/2303.00924** (2023), https://doi.org/10.48550/arXiv.2303.00924

[67] Solar-Lezama, A.: Program Synthesis by Sketching. Ph.D. thesis, University of California, Berkeley (2008), http://www.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-177.html

[68] Toninho, B., Yoshida, N.: Certifying data in multiparty session types. J. Log. Algebraic Methods Program. **90**, 61–83 (2017), https://doi.org/10.1016/j.jlamp.2016.11.005

[69] Unno, H., Terauchi, T., Gu, Y., Koskinen, E.: Modular primal-dual fixpoint logic solving for temporal verification. Proc. ACM Program. Lang. **7**(POPL), 2111–2140 (2023), https://doi.org/10.1145/3571265

[70] Veanes, M., Bjørner, N.S.: Symbolic automata: The toolkit. In: Flanagan, C., König, B. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 18th International Conference, TACAS 2012, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2012, Tallinn, Estonia, March 24 - April 1, 2012. Proceedings. Lecture Notes in Computer Science, vol. 7214, pp. 472–477. Springer (2012), https://doi.org/10.1007/978-3-642-28756-5_33

[71] Veanes, M., de Halleux, P., Tillmann, N.: Rex: Symbolic regular expression explorer. In: Third International Conference on Software Testing, Verification and Validation, ICST 2010, Paris, France, April 7-9, 2010. pp. 498–507. IEEE Computer Society (2010), https://doi.org/10.1109/ICST.2010.15

[72] Web Services Choreography Working Group: Web services choreography description language version 1.0. W3c candidate recommendation, World Wide Web Consortium (W3C) (November 2005), available at http://www.w3.org/TR/2005/CR-ws-cdl-10-20051109/

[73] Yoshida, N.: Programming language implementations with multiparty session types. In: de Boer, F.S., Damiani, F., Hähnle, R., Johnsen, E.B., Kamburjan, E. (eds.) Active Object Languages: Current Research Trends, Lecture Notes in Computer Science, vol. 14360, pp. 147–165. Springer (2024), https://doi.org/10.1007/978-3-031-51060-1_6

[74] Yoshida, N., Hu, R., Neykova, R., Ng, N.: The scribble protocol language. In: Abadi, M., Lluch-Lafuente, A. (eds.) Trustworthy Global Computing - 8th International Symposium, TGC 2013, Buenos Aires, Argentina, August 30-31, 2013, Revised Selected Papers. Lecture Notes in Computer Science, vol. 8358, pp. 22–41. Springer (2013), https://doi.org/10.1007/978-3-319-05119-2_3

[75] Zhou, F., Ferreira, F., Hu, R., Neykova, R., Yoshida, N.: Statically verified refinements for multiparty protocols. Proc. ACM Program. Lang. **4**(OOPSLA), 148:1–148:30 (2020), https://doi.org/10.1145/3428216

Typing Scalas-Yoshida Benchmarks

Franco Barbanera
University of Catania (Italy)
franco.barbanera@unict.it

Mariangiola Dezani-Ciancaglini
University of Torino (Italy)
dezani@di.unito.it

Ugo de'Liguoro
University of Torino (Italy)
ugo.deliguoro@unito.it

Abstract

We show how the Scalas–Yoshida "Less Is More" benchmark examples for multiparty protocols can be represented and typed within the Simple Multiparty Sessions (SMPS) framework. We reconstruct suitable global types for the relevant protocols and provide typing derivations for them. The examples include service–client authentication, the recursive two-buyer protocol, recursive Map/Reduce, and independent multiparty workers. These derivations demonstrate that SMPS can type benchmark sessions that are problematic or unprojectable in standard MPST settings, while retaining the behavioural guarantees of the system, such as session fidelity and lock freedom.

1 Introduction

In these notes, we show how the relevant examples from [10], also known as the "Less Is More" benchmark [3], can be rephrased within the Simple MultiParty Sessions (SMPS) framework of [2] as typable sessions. There is a basic mismatch between the approaches in [10] and [2], since the former uses local-type contexts and does away with global types. In contrast, sessions are typed directly by global types in [2]. Nonetheless, it is possible to express the examples from [10] in the type system of [2] as follows. First, we observe that when there is a unique session, as in all the examples in Figure 4 of [10], local types in [10] are akin to processes

in [2]. In the setting of [2], the typing of messages in [10] is irrelevant, since only values of ground types can be exchanged in a unique session. Then, we reconstruct the global type of the resulting session in the terms of [2].

The typability of the "Less is More" benchmark examples has recently been shown in [3] within a framework that approaches projectability from a typing perspective. In that approach, global types are assigned to individual participants (which, in the process calculus of [3], can carry values in messages), rather than to entire multiparty sessions, as in [2].

The results of [10] have recently been rephrased in a setting with global types and projections [8], where the projection operator is defined inductively to enable feasible implementation. A coinductively defined projection would make the global types of the benchmark examples projectable.

The benchmark examples, along with their representation and typing in the SMPS formalism of [2], are presented in Section 3 after a brief review of the formalism in Section 2.

2 SMPS calculus and type system

We briefly recall the main definitions of SMPS from [1] and [2]. For the calculus, global types, and type system, the following base sets and notation are used: *labels*, ranged over by $\lambda, \lambda', \ldots$; *participant names*, ranged over by $\mathsf{p}, \mathsf{q}, \mathsf{r}, \mathsf{s}, \ldots$; *processes*, ranged over by $P, Q, R, S, \ldots$; *sessions*, ranged over by $\mathbb{M}, \mathbb{M}', \ldots$; *global types*, ranged over by $\mathsf{G}, \mathsf{G}', \ldots$; *integers*, ranged over by $i, j, l, h, k, \ldots$; *(finite) integer sets*, ranged over by $I, J, \ldots$

Definition 2.1 (Processes)**.** Processes *are defined by:*

$$P \quad ::=_{\rho} \quad \mathbf{0} \mid \mathsf{p}!\{\lambda_i.P_i\}_{i\in I} \mid \mathsf{p}?\{\lambda_i.P_i\}_{i\in I}$$

where $I \neq \emptyset$ is finite and $\lambda_h \neq \lambda_k$ for $h, k \in I$ and $h \neq k$. We restrict the set of processes to the regular ones, i.e., terms having finitely many distinct subterms.

The productions in the above definition are interpreted *coinductively*, as indicated by the symbol $::=_{\rho}$. This means that the set of processes is the greatest fixed point of the (monotonic) functor over sets defined by the grammar, restricted, however, to regular processes. Thus, processes may be infinite.

Definition 2.2 (Multiparty Sessions)**.** Multiparty sessions *(*sessions *for short) are defined by:*

$$\mathbb{M} = \mathsf{p}_1[\, P_1 \,] \parallel \cdots \parallel \mathsf{p}_n[\, P_n \,]$$

with $n \geq 1$ and $\mathsf{p}_h \neq \mathsf{p}_k$ for any $h \neq k$. The set of participants *of a session $\mathbb{M}$,* $\mathsf{prt}(\mathbb{M})$*, is defined as* $\mathsf{prt}(\mathsf{p}_1[\, P_1 \,] \parallel \cdots \parallel \mathsf{p}_n[\, P_n \,]) = \{\mathsf{p}_i \mid P_i \neq \mathbf{0} \ \ \& \ \ 1 \leq i \leq n\}$

Because of the condition $\mathsf{p}_h \neq \mathsf{p}_k$ for any $h \neq k$, a session is essentially a finite set of (not necessarily distinct) processes P_i located at distinct participants p_i. To make this formal, over sessions we define the *structural congruence relation* $\mathbb{M} \equiv \mathbb{M}'$, which is the least congruence under which the parallel composition is commutative and associative, and such that, for all $\mathbb{M}$ and fresh p, we have $\mathsf{p}[\,\mathbf{0}\,] \parallel \mathbb{M} \equiv \mathbb{M}$. This implies that $\mathsf{p}[\,\mathbf{0}\,] \equiv \mathsf{p}[\,\mathbf{0}\,] \parallel \mathsf{q}[\,\mathbf{0}\,] \equiv \mathsf{q}[\,\mathbf{0}\,]$ for all distinct p, q. In a sense, any session of the shape $\mathsf{p}[\,\mathbf{0}\,]$ represents the empty session. We omit writing trailing $\mathbf{0}$'s in processes and denote $\mathsf{p}!\{\lambda.P\}$ and $\mathsf{p}?\{\lambda.P\}$ by $\mathsf{p}!\lambda.P$ and $\mathsf{p}?\lambda.P$, respectively.

The (synchronous) operational semantics of sessions is formally defined by the following labelled transition system.

Definition 2.3 (Session LTS)**.** *A* communication action *is a triple* $\mathsf{p}\lambda\mathsf{q}$*, where* $\mathsf{p} \neq \mathsf{q}$*. The* labelled transition system (LTS) for multiparty sessions*, with communication actions as labels, is the closure under structural congruence of the reduction specified by the following axiom:*

$$\mathsf{p}[\,P\,] \parallel \mathsf{q}[\,Q\,] \parallel \mathbb{M} \xrightarrow{\mathsf{p}\lambda_k\mathsf{q}} \mathsf{p}[\,P_k\,] \parallel \mathsf{q}[\,Q_k\,] \parallel \mathbb{M} \quad (k \in I \subseteq J) \quad \text{[Comm]}$$

where $P = \mathsf{q}!\{\lambda_i.P_i\}_{i\in I}$ *and* $Q = \mathsf{p}?\{\lambda_j.Q_j\}_{j\in J}$*.*

Axiom [Comm] above is non-deterministic in the choice of messages and makes the communication possible: participant p sends message λ_k to participant q. The sender can freely choose the message because the condition $I \subseteq J$ ensures that the receiver must offer all sender messages and possibly more. This allows distinguishing between internal and external choices in the operational semantics. This condition is always satisfied in well-typed sessions.
When $\sigma = \Lambda_1 \cdot \ldots \cdot \Lambda_n$ $(n \geq 0)$, we write $\mathbb{M} \xrightarrow{\sigma} \mathbb{M}'$ as shorthand for

$$\mathbb{M} \xrightarrow{\Lambda_1} \mathbb{M}_1 \cdots \xrightarrow{\Lambda_n} \mathbb{M}_n = \mathbb{M}'$$

where the Λ_is range over communication actions.

We write $\mathbb{M} \xrightarrow{\sigma}$ to mean that $\mathbb{M} \xrightarrow{\sigma} \mathbb{M}'$ for some $\mathbb{M}'$. Moreover, we define $\mathsf{prt}(\mathsf{p}\lambda\mathsf{q}) = \{\mathsf{p}, \mathsf{q}\}$ and $\mathsf{prt}(\sigma)$ as its obvious extension to sequences of communication actions (traces).

Global types are usually represented by μ-expressions, as in [6, 7] and in almost all papers on "classical" MultiParty Session Types (MPST). Instead, we define global types coinductively as potentially infinite regular terms.

Definition 2.4 (Global types)**.** Global types *are defined by:*

$$\mathsf{G} ::=_{\rho} \mathsf{End} \mid \mathsf{p} \to \mathsf{q} : \{\lambda_i.\mathsf{G}_i\}_{i\in I}$$

where $I \neq \emptyset$ *and* $\lambda_i \neq \lambda_j$ *for* $i, j \in I$ *and* $i \neq j$*. We restrict the set of global types to the regular ones. Given a global type* G*, the set of its* participants*,* $\mathsf{prt}(\mathsf{G})$*, is defined as the smallest set satisfying the following equations:*

$$\mathsf{prt}(\mathsf{End}) = \emptyset \qquad \mathsf{prt}(\mathsf{p} \to \mathsf{q} : \{\lambda_i.\mathsf{G}_i\}_{i\in I}) = \{\mathsf{p},\mathsf{q}\} \cup \bigcup_{i\in I} \mathsf{prt}(\mathsf{G}_i)$$

Since the definition of G is coinductive, so is the definition of prt(G). Because the syntactic tree of G is regular, prt(G) is well-defined and finite. In the following, trailing End's will be omitted, and $\mathsf{p} \to \mathsf{q} : \{\lambda.\mathsf{G}\}$ will be written as $\mathsf{p} \to \mathsf{q} : \lambda.\mathsf{G}$.

Now we recall the SMPS type assignment system, as defined in [1]. The (synchronous) calculus of SMPS, because of its simplicity, enables us to devise a type system in which global types are directly inferred for sessions. Double lines in the rules indicate that they are interpreted coinductively, following [9].

Definition 2.5 (Type System)**.** *Judgements of the form* $\mathsf{G} \vdash \mathbb{M}$ *are coinductively derived by the type system below, by considering sessions up to structural congruence:*

$$\mathsf{End} \vdash \mathsf{p}[\,\mathbf{0}\,] \quad \text{[T-End]}$$

$$\frac{\begin{array}{c}\mathsf{G}_i \vdash \mathsf{p}[\,P_i\,] \parallel \mathsf{q}[\,Q_i\,] \parallel \mathbb{M} \\ \mathsf{prt}(\mathsf{G}_i) \setminus \{\mathsf{p},\mathsf{q}\} = \mathsf{prt}(\mathbb{M}) \quad \forall i \in I \subseteq J\end{array}}{\mathsf{p} \to \mathsf{q} : \{\lambda_i.\mathsf{G}_i\}_{i\in I} \vdash \mathsf{p}[\,\mathsf{q}!\{\lambda_i.P_i\}_{i\in I}\,] \parallel \mathsf{q}[\,\mathsf{p}?\{\lambda_j.Q_j\}_{j\in J}\,] \parallel \mathbb{M}} \text{ [T-Comm]}$$

Rule [T-Comm] adds simultaneous communications to global types and to the corresponding processes within sessions. This rule allows more inputs than corresponding outputs, in agreement with the condition in Rule [Comm] (Definition 2.3). It also allows more branches in the input process than in the global type, partially mimicking local type subtyping [4].

It is convenient to associate with a global type the set of communication actions that could label its transitions. We call these the *capabilities* of the global type.

Definition 2.6 (Capabilities)**.** *Let* G *be a type. The set* cap(G) *of the* capabilities *of* G *is the smallest set satisfying the following equations:*

$$\mathsf{cap}(\mathsf{End}) = \emptyset$$
$$\mathsf{cap}(\mathsf{p} \to \mathsf{q} : \{\lambda_i.\mathsf{G}_i\}_{i\in I}) = \{\mathsf{p}\lambda_i\mathsf{q} \mid i \in I\} \cup \bigcup_{i\in I} \mathsf{cap}(\mathsf{G}_i)$$

By regularity, the set cap(G) is finite for all G.

The properties of the type system use the following LTS for global types.

Definition 2.7 (Coinductive LTS for global types)**.**

$$\mathsf{p} \to \mathsf{q} : \{\lambda_i.\mathsf{G}_i\}_{i\in I} \xrightarrow{\mathsf{p}\lambda_j\mathsf{q}} \mathsf{G}_j \ \ (j \in I) \quad \text{[E-Comm]}$$

$$\frac{\begin{array}{c}\mathsf{G}_i \xrightarrow{\mathsf{p}\lambda\mathsf{q}} \mathsf{G}_i' \qquad \forall i \in I \\ \{\mathsf{p},\mathsf{q}\} \cap \{\mathsf{r},\mathsf{s}\} = \emptyset \qquad \mathsf{p}\lambda\mathsf{q} \in \bigcap_{i\in I} \mathsf{cap}(\mathsf{G}_i)\end{array}}{\mathsf{r} \to \mathsf{s} : \{\lambda_i.\mathsf{G}_i\}_{i\in I} \xrightarrow{\mathsf{p}\lambda\mathsf{q}} \mathsf{r} \to \mathsf{s} : \{\lambda_i.\mathsf{G}_i'\}_{i\in I}} \text{ [I-Comm]}$$

Theorem 2.8 (Properties of typable sessions [2])**.** *Let* G, $\mathbb{M}$ *be such that* $\mathsf{G} \vdash \mathbb{M}$.

1. **Subject Reduction:** *If* $\mathbb{M} \xrightarrow{\mathsf{p}\lambda\mathsf{q}} \mathbb{M}'$, *then* $\mathsf{G} \xrightarrow{\mathsf{p}\lambda\mathsf{q}} \mathsf{G}'$ *and* $\mathsf{G}' \vdash \mathbb{M}'$ *for some* G'.
2. **Session Fidelity:** *If* $\mathsf{G} \xrightarrow{\mathsf{p}\lambda\mathsf{q}} \mathsf{G}'$, *then* $\mathbb{M} \xrightarrow{\mathsf{p}\lambda\mathsf{q}} \mathbb{M}'$ *and* $\mathsf{G}' \vdash \mathbb{M}'$ *for some* $\mathbb{M}'$.
3. **Lock Freedom:** $\mathbb{M} \xrightarrow{\sigma} \mathbb{M}'$ *and* $\mathsf{p} \in \mathsf{prt}(\mathbb{M}')$ *imply* $\mathbb{M}' \xrightarrow{\sigma' \cdot \Lambda}$ *for some* σ' *and* Λ *such that* $\mathsf{p} \in \mathsf{prt}(\Lambda)$.

3 Typing the Scalas-Yoshida benchmark examples

The typings of the first two examples were already presented in [2] and are reported here for completeness. To improve readability, the conditions on participants will be written only in the derivation for the first example.

Service, client and authentication protocol. This is the example in the Introduction of [10], further detailed in Figure 4(1) of the same paper.

> The service s sends to the client c either a request to login or cancel. If login is issued, then c sends a password pwd to the authorisation server a and then s receives from a the authorisation auth. In case c receives cancel from s the interaction with a is aborted by c sending quit to a.

This session is encoded by

$$\mathbb{M}_{\mathsf{sca}} = \mathsf{s}[\, S \,] \parallel \mathsf{c}[\, C \,] \parallel \mathsf{a}[\, A \,]$$

where the processes are

$S = \mathsf{c}!\{\mathtt{login}.\mathsf{a}?\mathtt{auth}, \mathtt{cancel}\}$
$C = \mathsf{s}?\{\mathtt{login}.\mathsf{a}!\mathtt{pwd},\ \mathtt{cancel}.\mathsf{a}!\mathtt{quit}\}$
$A = \mathsf{c}?\{\mathtt{pwd}.\mathsf{s}!\mathtt{auth},\ \mathtt{quit}\}$

Then we derive $\mathsf{G}_{\mathsf{sca}} \vdash \mathbb{M}_{\mathsf{sca}}$ where

$$\begin{array}{rcl} \mathsf{G}_{\mathsf{sca}} & = & \mathsf{s} \to \mathsf{c} : \{\mathtt{login}.\mathsf{G}'_{\mathsf{sca}},\ \mathtt{cancel}.\mathsf{c} \to \mathsf{a} : \mathtt{quit}\} \\ \mathsf{G}'_{\mathsf{sca}} & = & \mathsf{c} \to \mathsf{a} : \mathtt{pwd}.\ \mathsf{a} \to \mathsf{s} : \mathtt{auth} \end{array}$$

as shown on the right of Figure 1, where $\mathrm{Q} = \mathtt{quit}$, $\mathrm{P} = \mathtt{pwd}$, and $\mathrm{A} = \mathtt{auth}$.

We observe that $\mathsf{G}_{\mathsf{sca}}$ is exactly the global type derived for this session in [10]. In that paper, it is shown that $\mathsf{G}_{\mathsf{sca}}$ is projectable, but S and A do not have dual input/output communications. The problem is that the input/output behaviours between s and a depend on their inputs/outputs with c. More precisely, s and a exchange the message auth only when c sends login to s. Hence, this example is ruled out by the classical MPST system in [6, 7, 5]. In this case, it seems that our system addresses this inconsistency by directly embodying in Rule [T-Comm] the

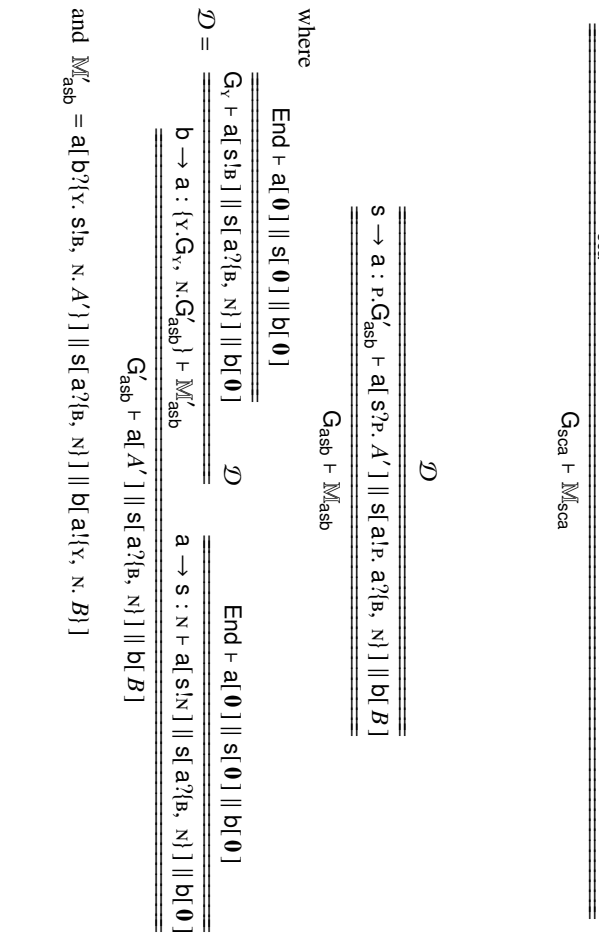


Figure 1: Derivations for $\mathsf{G}_{\mathsf{sca}} \vdash \mathbb{M}_{\mathsf{sca}}$ (right) and $\mathsf{G}_{\mathsf{asb}} \vdash \mathbb{M}_{\mathsf{asb}}$ (left).

guarantee that any implementation of the protocol $\mathbb{M}_{\mathsf{sca}}$ will be well behaved and lock-free.

Recursive two-buyer protocol. This is the example in Figure 4(2) of [10].

> After `Query`ing the store s for the `Price` of some good, Alice, represented by role a, asks Bob, represented by b, to `Split` the price. If Bob replies by issuing `Yes`, then Alice sends `Buy` to s; otherwise, she insists by asking Bob for a different splitting (the fact that the subsequent

proposals by Alice are actually different is not explicitly represented in the protocol below nor in the type of a in [10]). At any time, Alice might quit the protocol, sending esc to Bob and no to the store.

The protocol is encoded by the session

$$\mathbb{M}_{\mathsf{asb}} = \mathsf{a}[\, A \,] \parallel \mathsf{s}[\, S \,] \parallel \mathsf{b}[\, B \,]$$

where the processes are

$$A = \mathsf{s}!\mathtt{que}.\, \mathsf{s}?\mathtt{pri}.\, A'$$
$$A' = \mathsf{b}! \begin{cases} \mathtt{spl}.\, \mathsf{b}?\{\mathtt{yes}.\, \mathsf{s}!\mathtt{buy},\ \mathtt{no}.\, A'\} \\ \mathtt{esc}.\, \mathsf{s}!\mathtt{no} \end{cases}$$
$$S = \mathsf{a}?\mathtt{que}.\, \mathsf{a}!\mathtt{pri}.\, \mathsf{a}?\{\mathtt{buy},\ \mathtt{no}\}$$
$$B = \mathsf{a}? \begin{cases} \mathtt{spl}.\, \mathsf{a}!\{\mathtt{yes},\ \mathtt{no}.\, B\} \\ \mathtt{esc} \end{cases}$$

The session $\mathbb{M}_{\mathsf{asb}}$ can be typed by

$$\mathsf{G}_{\mathsf{asb}} = \mathsf{a} \to \mathsf{s} : \mathtt{que}.\mathsf{s} \to \mathsf{a} : \mathtt{pri}.\mathsf{G}'_{\mathsf{asb}}$$

where

$$\mathsf{G}'_{\mathsf{asb}} = \mathsf{a} \to \mathsf{b} : \begin{cases} \mathtt{split}.\mathsf{b} \to \mathsf{a} : \{\mathtt{yes}.\mathsf{G}_{\mathtt{yes}}, \mathtt{no}.\mathsf{G}'_{\mathsf{asb}}\}, \\ \mathtt{esc}.\mathsf{a} \to \mathsf{s} : \mathtt{no} \end{cases}$$
$$\mathsf{G}_{\mathtt{yes}} = \mathsf{a} \to \mathsf{s} : \mathtt{buy}$$

as shown on the left of Figure 1, where p = pri, b = buy, y = yes, and n = no.

This example is interesting for two reasons. In [10] it is argued that the typing context corresponding to $\mathbb{M}_{\mathsf{asb}}$ cannot be obtained by projecting any global type, no matter whether one uses plain or full merging (see Definition 3.3 in Figure 3 of [10]). Instead in our type system we can type $\mathbb{M}_{\mathsf{asb}}$ with the un-projectable global type $\mathsf{G}_{\mathsf{asb}}$. The other reason is that $\mathsf{G}_{\mathsf{asb}}$ is a good example of an unbounded global type [2] that is inhabited.

Recursive Map/Reduce (n = 2). This is the example in Figure 4(3) of [10]. We consider only the case of two workers for the sake of readability.

The mapper (m) assigns one datum to both Worker1 (w_1) and Worker2 (w_2) for processing. Once completed, both w_1 and w_2 send their results to the reducer (r). Then r informs m whether to continue or to stop. If termination is chosen, the m also instructs w_1 and w_2 to stop.

This protocol is implemented by the multiparty session

$$\mathbb{M}_{\text{MR}} = \mathsf{m}[\, M \,] \parallel \mathsf{r}[\, R \,] \parallel \mathsf{w}_1[\, W_1 \,] \parallel \mathsf{w}_2[\, W_2 \,]$$

$$\mathcal{D} = \dfrac{\dfrac{\dfrac{\

$$
\begin{array}{rcl}
\mathsf{G}^1_{\mathrm{MW}} & = & \mathsf{a}_1 \to \mathsf{b}_1 : \left\{ \begin{array}{l} \mathtt{Datum}.\mathsf{b}_1 \to \mathsf{c}_1 : \mathtt{Datum}.\mathsf{c}_1 \to \mathsf{a}_1 : \mathtt{Result}.\mathsf{G}^2_{\mathrm{MW}} \\ \mathtt{stop}.\mathsf{b}_1 \to \mathsf{c}_1 : \mathtt{stop}.\mathsf{G}^3_{\mathrm{MW}} \end{array} \right. \\
\mathsf{G}^2_{\mathrm{MW}} & = & \mathsf{a}_2 \to \mathsf{b}_2 : \left\{ \begin{array}{l} \mathtt{Datum}.\mathsf{b}_2 \to \mathsf{c}_2 : \mathtt{Datum}.\mathsf{c}_2 \to \mathsf{a}_2 : \mathtt{Result}.\mathsf{G}^1_{\mathrm{MW}} \\ \mathtt{stop}.\mathsf{b}_2 \to \mathsf{c}_2 : \mathtt{stop}.\mathsf{G}^4_{\mathrm{MW}} \end{array} \right. \\
\mathsf{G}^3_{\mathrm{MW}} & = & \mathsf{a}_2 \to \mathsf{b}_2 : \left\{ \begin{array}{l} \mathtt{Datum}.\mathsf{b}_2 \to \mathsf{c}_2 : \mathtt{Datum}.\mathsf{c}_2 \to \mathsf{a}_2 : \mathtt{Result}.\mathsf{G}^3_{\mathrm{MW}} \\ \mathtt{stop}.\mathsf{b}_2 \to \mathsf{c}_2 : \mathtt{stop} \end{array} \right. \\
\mathsf{G}^4_{\mathrm{MW}} & = & \mathsf{a}_1 \to \mathsf{b}_1 : \left\{ \begin{array}{l} \mathtt{Datum}.\mathsf{b}_1 \to \mathsf{c}_1 : \mathtt{Datum}.\mathsf{c}_1 \to \mathsf{a}_1 : \mathtt{Result}.\mathsf{G}^4_{\mathrm{MW}} \\ \mathtt{stop}.\mathsf{b}_1 \to \mathsf{c}_1 : \mathtt{stop} \end{array} \right.
\end{array}
$$

Figure 3: Definition of $\mathsf{G}^1_{\mathrm{MW}}$.

where

$$M = \mathsf{w}_1!\mathtt{Datum}.\mathsf{w}_2!\mathtt{Datum}.\mathsf{r}? \left\{ \begin{array}{l} \mathtt{continue}.M \\ \mathtt{stop}.\mathsf{w}_1!\mathtt{stop}.\mathsf{w}_2!\mathtt{stop} \end{array} \right.$$

$$R = \mathsf{w}_1?\mathtt{Result}.\mathsf{w}_2?\mathtt{Result}.\mathsf{m}! \left\{ \begin{array}{l} \mathtt{continue}.R \\ \mathtt{stop} \end{array} \right.$$

$$W_i = \mathsf{m}? \left\{ \begin{array}{l} \mathtt{Datum}.\mathsf{r}!\mathtt{Result}.W_i \\ \mathtt{stop} \end{array} \right. \quad \text{for } i = 1, 2.$$

The above multiparty session can be typed by the following global type:
$\mathsf{G}_{\mathrm{MR}} = \mathsf{m} \to \mathsf{w}_1 : \mathtt{Datum}.\mathsf{m} \to \mathsf{w}_2 : \mathtt{Datum}.\mathsf{w}_1 \to \mathsf{r} : \mathtt{Result}.\mathsf{w}_2 \to \mathsf{r} : \mathtt{Result}.\mathsf{G}'_{\mathrm{MR}}$
where

$$\mathsf{G}'_{\mathrm{MR}} = \mathsf{r} \to \mathsf{m} : \left\{ \begin{array}{l} \mathtt{continue}.\mathsf{G}_{\mathrm{MR}} \\ \mathtt{stop}.\mathsf{m} \to \mathsf{w}_1 : \mathtt{stop}.\mathsf{m} \to \mathsf{w}_2 : \mathtt{stop} \end{array} \right.$$

as shown in Figure 2, where $\mathtt{D} = \mathtt{Datum}$, $\mathtt{R} = \mathtt{Result}$, $\mathtt{C} = \mathtt{continue}$, and $\mathtt{S} = \mathtt{stop}$. Notice that the implicit use of subtyping in the typing rule allows us to get simpler processes for the workers than the corresponding local types in [10] and processes in [3].

Independent Multiparty Workers. (n = 2)

This is the example in Figure 4(4) of [10]. For readability, we consider the case of two workers per class (A, B, C).

> A starter (s) participant tells to both Worker A1 (a_1) and Worker A2 (a_2) to independently process a Datum. The protocol then proceeds as follows:
>
> - a_1 informs Worker B1 (b_1) to either process the Datum or to stop. In the former case, b_1 tells Worker C1 (c_1) to process the Datum, after which c_1 communicate the Result to a_1. Upon receiving that, a_1 can either repeat what was described above and again tell

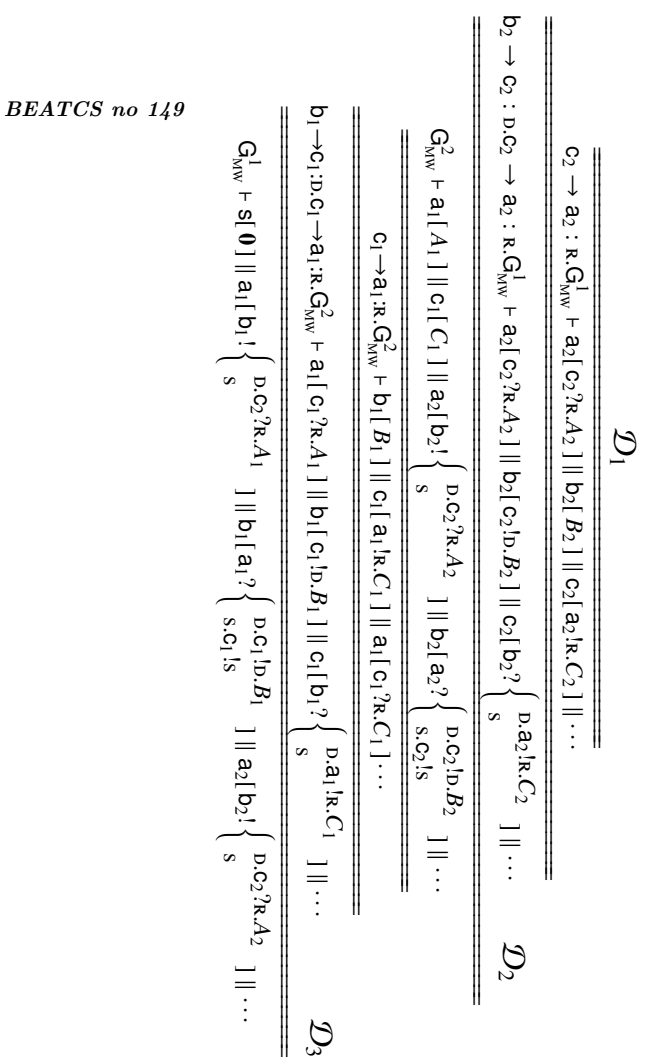


Figure 4: Definition of $\mathcal{D}_1$ for deriving $\mathsf{G}_{\mathrm{MW}} \vdash \mathbb{M}_{\mathrm{MW}}$.

b_1 to process the `datum` or to `stop`. In the latter case, b_1 instructs c_1 to `stop`, too.

- Workers A2, B2, C2 (a_2, b_2, c_2) follow the same sub-protocol as, respectively, a_1, b_1, c_1, independently.

The above protocol is encoded by the following multiparty session:

$$\mathbb{M}_{\mathrm{MW}} = \mathsf{s}[\, S \,] \parallel \mathsf{a}_1[\, A_1 \,] \parallel \mathsf{b}_1[\, B_1 \,] \parallel \mathsf{c}_1[\, C_1 \,] \parallel \mathsf{a}_2[\, A_2 \,] \parallel \mathsf{b}_2[\, B_2 \,] \parallel \mathsf{c}_2[\, C_2 \,]$$

$$\mathsf{G}^4_{\mathrm{MW}} \vdash \mathsf{a}_1[\,\mathsf{b}_1!\{\,\mathsf{s}\ \ {}^{\mathrm{D}}.\mathsf{c}_1?^{\mathrm{R}}.A_1\,\}\,]\,\|\,\mathsf{b}_1[\,\mathsf{a}_1?\{\,\mathsf{s}.\mathsf{c}_1!^{\mathrm{D}}.B_1\,\}\,]\,\|\cdots \qquad \mathcal{D}_{4,1}$$

$$\mathsf{b}_1 \to \mathsf{c}_1 : {}^{\mathrm{D}}.\mathsf{c}_1 \to \mathsf{a}_1 : {}^{\mathrm{R}}.\mathsf{G}^4_{\mathrm{MW}} \vdash \mathsf{a}_1[\,\mathsf{c}_1?^{\mathrm{R}}.A_1\,]\,\|\,\mathsf{b}_1[\,\mathsf{c}_1!^{\mathrm{D}}.B_1\,]\,\|\,\mathsf{c}_1[\,\mathsf{b}_1?\{\,\mathsf{s}\ \ {}^{\mathrm{D}}.\mathsf{a}_1!^{\mathrm{R}}.C_1\,\}\,]\,\|\cdots$$

$$\mathsf{c}_1 \to \mathsf{a}_1 : {}^{\mathrm{R}}.\mathsf{G}^4_{\mathrm{MW}} \vdash \mathsf{a}_1[\,\mathsf{c}_1?^{\mathrm{R}}.A_1\,]\,\|\,\mathsf{b}_1[\,B_1\,]\,\|\,\mathsf{c}_1[\,\mathsf{a}_1!^{\mathrm{R}}.C_1\,]\,\|\cdots \qquad \mathcal{D}_4$$

$$\mathsf{G}^3_{\mathrm{MW}} \vdash \mathsf{a}_2[\,\mathsf{b}_2!\{\,\mathsf{s}\ \ {}^{\mathrm{D}}.\mathsf{c}_2?^{\mathrm{R}}.A_2\,\}\,]\,\|\,\mathsf{b}_2[\,\mathsf{a}_2?\{\,\mathsf{s}.\mathsf{c}_2!^{\mathrm{s}}\,\}\,]\,\|\cdots \qquad \mathcal{D}_{3,2}$$

$$\mathsf{b}_2 \to \mathsf{c}_2 : {}^{\mathrm{D}}.\mathsf{c}_2 \to \mathsf{a}_2 : {}^{\mathrm{R}}.\mathsf{G}^3_{\mathrm{MW}} \vdash \mathsf{a}_2[\,\mathsf{c}_2?^{\mathrm{R}}.A_2\,]\,\|\,\mathsf{b}_2[\,\mathsf{c}_2!^{\mathrm{D}}.B_2\,]\,\|\,\mathsf{c}_2[\,\mathsf{b}_2?\{\,\mathsf{s}\ \ {}^{\mathrm{D}}.\mathsf{a}_2!^{\mathrm{R}}.C_2\,\}\,]\,\|\cdots \qquad \mathcal{D}_{3,1}$$

$$\mathsf{c}_2 \to \mathsf{a}_2 : {}^{\mathrm{R}}.\mathsf{G}^3_{\mathrm{MW}} \vdash \mathsf{a}_2[\,\mathsf{c}_2?^{\mathrm{R}}.A_2\,]\,\|\,\mathsf{b}_2[\,B_2\,]\,\|\,\mathsf{c}_2[\,\mathsf{a}_2!^{\mathrm{R}}.C_2\,]\,\|\cdots$$

Figure 5: Definitions of $\mathcal{D}_{3,1}$ (right) and $\mathcal{D}_4$ (left) for deriving $\mathsf{G}_{\mathrm{MW}} \vdash \mathbb{M}_{\mathrm{MW}}$.

where

$$S = \mathsf{a}_1!^{\mathrm{D}}\mathtt{atum}.\mathsf{a}_2!^{\mathrm{D}}\mathtt{atum}$$

$$A_i = \mathsf{s}?^{\mathrm{D}}\mathtt{atum}.\mathsf{b}_i!\left\{\begin{array}{l}{}^{\mathrm{D}}\mathtt{atum}.\mathsf{c}_i?^{\mathrm{R}}\mathtt{result}.A_i\\ \mathtt{stop}\end{array}\right.$$

$$B_i = \mathsf{a}_i?\left\{\begin{array}{l}{}^{\mathrm{D}}\mathtt{atum}.\mathsf{c}_i!^{\mathrm{D}}\mathtt{atum}.B_i\\ \mathtt{stop}.\mathsf{c}_i!\mathtt{stop}\end{array}\right.$$

$$C_i = \mathsf{b}_i?\left\{\begin{array}{l}{}^{\mathrm{D}}\mathtt{atum}.\mathsf{a}_i!^{\mathrm{R}}\mathtt{result}.C_i\\ \mathtt{stop}\end{array}\right.$$

$$\mathcal{D}_2 = \dfrac{\mathcal{D}_4}{\mathsf{b}_2 \to \mathsf{c}_2 : \mathsf{s}.\mathsf{G}^4_{\mathrm{MW}} \vdash \mathsf{a}_2[\,\mathbf{0}\,] \parallel \mathsf{b}_2[\,\mathsf{c}_2!\mathsf{s}\,] \parallel \mathsf{c}_2[\,\mathsf{b}_2?\begin{cases} \textsc{d}.\mathsf{a}_2!\textsc{r}.C_2 \\ \mathsf{s} \end{cases}\,] \parallel \cdots}$$

$$\mathcal{D}_3 = \dfrac{\mathcal{D}_{3,1}}{\mathsf{b}_1 \to \mathsf{c}_1 : \mathsf{s}.\mathsf{G}^3_{\mathrm{MW}} \vdash \mathsf{a}_1[\,\mathbf{0}\,] \parallel \mathsf{b}_1[\,\mathsf{c}_1!\mathsf{s}\,] \parallel \mathsf{c}_1[\,\mathsf{b}_1?\begin{cases} \textsc{d}.\mathsf{a}_1!\textsc{r}.C_1 \\ \mathsf{s} \end{cases}\,] \parallel \cdots}$$

$$\mathcal{D}_{3,2} = \dfrac{\mathsf{End} \vdash \mathsf{s}[\,\mathbf{0}\,] \parallel \mathsf{a}_1[\,\mathbf{0}\,] \parallel \mathsf{b}_1[\,\mathbf{0}\,] \parallel \mathsf{c}_1[\,\mathbf{0}\,] \parallel \mathsf{a}_2[\,\mathbf{0}\,] \parallel \mathsf{b}_2[\,\mathbf{0}\,] \parallel \mathsf{c}_2[\,\mathbf{0}\,]}{\mathsf{b}_2 \to \mathsf{c}_2 : \mathsf{s} \vdash \mathsf{a}_2[\,\mathbf{0}\,] \parallel \mathsf{b}_2[\,\mathsf{c}_2!\mathsf{s}\,] \parallel \mathsf{c}_2[\,\mathsf{b}_2?\begin{cases} \textsc{d}.\mathsf{a}_2!\textsc{r}.C_2 \\ \mathsf{s} \end{cases}\,] \parallel \cdots}$$

$$\mathcal{D}_{4,1} = \dfrac{\mathsf{End} \vdash \mathsf{s}[\,\mathbf{0}\,] \parallel \mathsf{a}_1[\,\mathbf{0}\,] \parallel \mathsf{b}_1[\,\mathbf{0}\,] \parallel \mathsf{c}_1[\,\mathbf{0}\,] \parallel \mathsf{a}_1[\,\mathbf{0}\,] \parallel \mathsf{b}_1[\,\mathbf{0}\,] \parallel \mathsf{c}_1[\,\mathbf{0}\,]}{\mathsf{b}_1 \to \mathsf{c}_1 : \mathsf{s} \vdash \mathsf{a}_1[\,\mathbf{0}\,] \parallel \mathsf{b}_1[\,\mathsf{c}_1!\mathsf{s}\,] \parallel \mathsf{c}_1[\,\mathsf{b}_1?\begin{cases} \textsc{d}.\mathsf{a}_1!\textsc{r}.C_1 \\ \mathsf{s} \end{cases}\,] \parallel \cdots}$$

Figure 6: Definitions of $\mathcal{D}_2$, $\mathcal{D}_3$, $\mathcal{D}_{3,2}$, $\mathcal{D}_{4,1}$ for deriving $\mathsf{G}_{\mathrm{MW}} \vdash \mathbb{M}_{\mathrm{MW}}$.

for $i = 1, 2$.

The above multiparty session can be typed by the following global type:

$$\mathsf{G}_{\mathrm{MW}} = \mathsf{s} \to \mathsf{a}_1 : \textsc{d}\mathtt{atum}.\mathsf{s} \to \mathsf{a}_2 : \textsc{d}\mathtt{atum}.\mathsf{G}^1_{\mathrm{MW}}$$

where $\mathsf{G}^1_{\mathrm{MW}}$ is defined in Figure 3.
The typing derivation for this example is

$$\dfrac{\mathcal{D}_1}{\mathsf{s} \to \mathsf{a}_2 : \textsc{d}.\mathsf{G}^1_{\mathrm{MW}} \vdash \mathsf{s}[\,\mathsf{a}_2!\textsc{d}\,] \parallel \mathsf{a}_1[\,\mathsf{b}_1!\begin{cases} \textsc{d}.\mathsf{c}_1?\textsc{r}.A_1 \\ \mathsf{s} \end{cases}\,] \parallel \mathsf{a}_2[\,\mathsf{s}?\textsc{d}.\mathsf{b}_2!\begin{cases} \textsc{d}.\mathsf{c}_2?\textsc{r}.A_2 \\ \mathsf{s} \end{cases}\,] \parallel \cdots}{\mathsf{G}_{\mathrm{MW}} \vdash \mathbb{M}_{\mathrm{MW}}}$$

where $\mathcal{D}_1$ is defined in Figure 4, $\mathcal{D}_{3,1}$ and $\mathcal{D}_4$ are defined in Figure 5, the other subderivations are defined in Figure 6. In these figures we abbreviate $\textsc{d} = \textsc{d}\mathtt{atum}$, $\textsc{r} = \textsc{r}\mathtt{esult}$, and $\mathsf{s} = \mathtt{stop}$.

We remark that by allowing a parallel composition operator between global types with different participants as in [6, 7] and the typing rule

$$\frac{\mathsf{G} \vdash \mathbb{M} \quad \mathsf{G}' \vdash \mathbb{M}'}{\mathsf{G} \parallel \mathsf{G}' \vdash \mathbb{M} \parallel \mathbb{M}'}$$

we could type this example with the simpler global type

$$\mathsf{s} \to \mathsf{a}_1 : \texttt{Datum}.\mathsf{s} \to \mathsf{a}_2 : \texttt{Datum}.(\mathsf{G}_1 \parallel \mathsf{G}_2)$$

where

$$\mathsf{G}_i = \mathsf{a}_i \to \mathsf{b}_i : \begin{cases} \texttt{Datum}.\mathsf{b}_i \to \mathsf{c}_1 : \texttt{Datum}.\mathsf{c}_i \to \mathsf{a}_i : \texttt{Result}.\mathsf{G}_i \\ \texttt{Stop}.\mathsf{b}_i \to \mathsf{c}_i : \texttt{Stop}. \end{cases}$$

for $i = 1, 2$.

References

[1] Franco Barbanera, Mariangiola Dezani-Ciancaglini & Ugo de'Liguoro (2022): *Open compliance in multiparty sessions*. In S. Lizeth Tapia Tarifa & José Proença, editors: *FACS*, *LNCS* 13712, Springer, pp. 222–243, doi:10.1007/978-3-031-20872-0_13.

[2] Franco Barbanera, Mariangiola Dezani-Ciancaglini & Ugo de'Liguoro (2024): *Unprojectable Global Types for Multiparty Sessions*. In Alessandro Bruni, Alberto Momigliano, Matteo Pradella, Matteo Rossi & James Cheney, editors: *PPDP*, ACM, pp. 15:1–15:13, doi:10.1145/3678232.3678245.

[3] David Castro-Perez, Francisco Ferreira & Sung-Shik Jongmans (2026): *A Synthetic Reconstruction of Multiparty Session Types*. *PACMPL* 10(POPL), pp. 1442–1470, doi:10.1145/3776692.

[4] Romain Demangeon & Kohei Honda (2011): *Full abstraction in a subtyped pi-calculus with linear types*. In J-P. Katoen & B. König, editors: *CONCUR*, *LNCS* 6901, Springer, Berlin, pp. 280–296, doi:10.1007/978-3-642-23217-6_19.

[5] Pierre-Malo Deniélou & Nobuko Yoshida (2011): *Dynamic multirole session types*. In Thomas Ball & Mooly Sagiv, editors: *POPL*, ACM Press, pp. 435–446, doi:10.1145/1926385.1926435.

[6] Kohei Honda, Nobuko Yoshida & Marco Carbone (2008): *Multiparty asynchronous session types*. In George C. Necula & Philip Wadler, editors: *POPL*, ACM Press, pp. 273–284, doi:10.1145/1328897.1328472.

[7] Kohei Honda, Nobuko Yoshida & Marco Carbone (2016): *Multiparty asynchronous session types*. *Journal of the ACM* 63(1), pp. 9:1–9:67, doi:10.1145/2827695.

[8] Ping Hou, Nobuko Yoshida & Iona Kuhn (2026): *Less is more revisited: Association with global protocols and multiparty sessions*. *Theoretical Computer Science* 1076, p. 115873, doi:10.1016/J.TCS.2026.115873.

[9] Xavier Leroy & Hervé Grall (2009): *Coinductive big-step operational semantics*. *Information and Computation* 207(2), pp. 284–304, doi:10.1016/J.IC.2007.12.004.

[10] Alceste Scalas & Nobuko Yoshida (2019): *Less is more: multiparty session types revisited*. *PACMPL* 3(POPL), pp. 30:1–30:29, doi:10.1145/3290343.

Conference Reports Column

by

Laurent Feuilloley

Laboratoire d'informatique
en image et systèmes d'information (LIRIS)
CNRS and Université Lyon 1
43 Bd du 11 novembre 1918, 69622 Villeurbanne, France
laurent.feuilloley@cnrs.fr

BEATCS no 149

Conference report on STACS 2026

Antoine Amarilli, Nicolas Bousquet, Justine Cauvi
and Ugo Giocanti

Abstract

This report is made of two parts: one general report about the conference by the three first authors, and one more personnal report by the last author.

1 General report

The 43rd International Symposium on Theoretical Aspects of Computer Science (STACS) took place in Grenoble, France, from 10 to 13 March 2026. The conference was preceded on March 9 by a tutorial given by Tatiana Starikovskaya and Gabriel Bathie on *Regular languages recognition in restricted models of computation.* STACS 2026 consisted of two tracks. Track A, chaired by Meena Mahajan and Nguyễn Kim Thắng, focused on algorithms, data structures and complexity. Track B, chaired by Annabelle McIver and Florin Manea focused on automata, logic, semantics and theory computing.

Three invited talks were given during the conference by Martin Grohe (Query Languages for Machine-Learning Models), Laura Kovacs (Moments in Time: Algebraic Analysis for Solvable Loops) and Ola Svensson (Advancements in Online Edge Coloring Algorithms) which gave an overview of deep recent results in their relative fields and showed the diversity of the research in TCS.

The social activity consisted in a snowshoe outing in the mountains, and the banquet took place in the restaurant "Le Père Gras", on top of the Bastille, which the participants could reach by a scenic cable car ride above the city.

Business meeting Interesting statistics on the conference were presented during the business meeting. A total of 206 papers were submitted to Track A, of which 50 were accepted, resulting in an acceptance rate of 27% while 60 papers were submitted to Track B, of which 20 were accepted, resulting in an acceptance rate of 33%. The committee highlighted the very high quality of the submissions, which led to the conference being extended

by half a day (and even so, papers with excellent reviews were rejected). Researchers from around the world submitted papers to STACS. The six countries with the most accepted papers were, in descending order of accepted papers, Germany, France, the United States, the United Kingdom, Poland and Netherlands.

The PC consisted of 26 members in track A and 26 members in track B, in addition to 529 external reviewers. The vast majority of papers received three reviews.

STACS is a member conference of the TheoretiCS journal, and a status update about TheoretiCS was presented at the end of the business meeting.

140 people registered to the conference and, in addition, several local participants from Grenoble attended the conference. Some presentations were given as online live talks from remote speakers.

The best papers accepted at the conference will be invited to journals: seven to *ACM Transactions on Computation Theory*, one to *TheoretiCS* and some papers will also be invited to *Logical Methods in Computer Science*.

During the business meeting, the next location of STACS was announced. STACS 2027 will be held in Göttingen, in the center of Germany but also online, with the possibility to have remote talks. Like in previous years, there will not be any limit on the number of accepted papers that will be presented remotely.

2 A personnal report

General feedback I (Ugo Giocanti) went to STACS 2026 with one accepted paper, that has been presented by one of my coauthors. I am a postdoctorate researcher, and I also attended to STACS 2023 in Hamburg, from which I keep some very good memories. I did my PhD in Grenoble, and thus was eager to attend to STACS 2026 there. Just like for the 2023 edition, I liked most of the talks (both in invited and parallel sessions), and found the overall quality to be high. As a graph theorist, also interested in a couple of questions from other areas, I really enjoyed the opportunity to attend talks from other branches of Theoretical CS, and to meet and exchange with people who do not necessarily belong to my close area of research.

Some talk highlights Here is a small list of talks I particularly liked. This is of course biased by my research interests, and I also enjoyed all the other talks I attended.

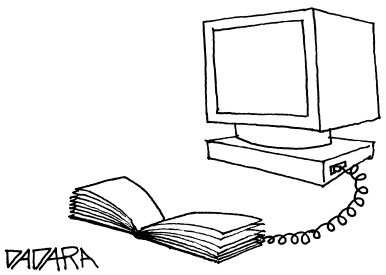
- *Testing H-freeness on sparse graphs, the case of bounded expansion*, by Samuel Humeau, Mamadou Kanté, Daniel Mock, Timothé Picavet and

Alexandre Vigny (Track A).
I am biased on this one as I know the person presenting well. I really liked the talk, first, because the result seems of great interest, second, because I believe that one of the main proof ideas (a reduction to the bounded treedepth case) was exposed very clearly, even for an audience not necessarily familiar with the graph notions involved, and third, because the style of the slides was quite unusual.

- *Optimal Average Disk-Inspection via Fermat's Principle*, by Konstantinos Georgiou (Track A).
  This talk was very original, both in its content and in the very personnal way the author presented his result. He answered a quite old question about the "Disk-Inspection" problem, which he has been working on for some time. The most surprising thing was that the key idea to solve the general case came after he watched a vulgarization video on Fermat's Principle (of least time) and Snell's law on light refraction! I really enjoyed that such very old and simple principle from physics could find some application in 2026.

- *Efficient Compression in Semigroups*, by Alexander Thumm and Armin Weiß (Track B).
  The presentation was on straight-line programs and compression of semigroups, a topic that I did not know at all before (but still not unrelated to some questions I work on), and I really enjoyed that the presenter gave a clear presentation of this nice area. This is clearly a topic on which I'm now eager to know more once I'll find time for it, thanks to this presentation.

News and Conference Reports



Report from EATCS Japan Chapter

Yuto Nakashima (Kyushu University)

EATCS-JP/LA Workshop on TCS and Presentation Awards

The 24th *EATCS-JP/LA Workshop on Theoretical Computer Science* was held at Research Institute of Mathematical Sciences, Kyoto University, January 26th to 28th, 2026. (The details can also be found, although this website is written in Japanese, at `https://la-symposium.github.io/2025/winter.html`.)

Every year, we choose the best presenter and the best student presenter. This year, we celebrated the following presentation as the 24th LA/EATCS-Japan Presentation Award:

> "*Courcelle's Theorem for Lipschitz Continuity*"
> presented by Tatsuya Gima (Hokkaido University), **Soh Kumabe** (CyberAgent), and Yuichi Yoshida (National Institute of Informatics).

We celebrated the following presentation as the 15th LA/EATCS-Japan Student Presentation Award:

> "*Faster Approximate Linear Matroid Intersection*"
> presented by **Tatsuya Terao** (Kyoto University).

The awards were recognized publicly on the last day, January 28th, 2026.

Congratulations!

This workshop is jointly organized by *LA symposium*, Japanese association of theoretical computer scientists. Its purpose is to give a place to discuss topics on all aspects of theoretical computer science. This workshop is an unrefereed meeting. All submissions are accepted for the presentation. There should be no problem of presenting these papers at refereed conferences and/or journals. This meeting is familiar and widely open for everyone who is interested in theoretical computer science. It is held twice a year (January/February and July/August). If you have a chance, I recommend that you attend it. Check the website `http://www.dais.is.tohoku.ac.jp/eatcs_japan/` for further details. The list of the presentations is as below; you can see the activity of the Japanese society of theoretical computer science.

Program of the 24th EATCS-JP/LA workshop on TCS

In the following program, "*" indicates speakers. The number [*xx*S] means it is given by a student speaker.

[1S] An Improved Approximation Algorithm for Maximum 2+-Path Packing
**Riki Uchida, Zhi-Zhong Chen, Ryo Yaguchi (Tokyo Denki University), Lusheng Wang (City University of Hong Kong)*

[2S] One-Sided Local Crossing Minimization
*Grzegorz Gutowski (Jagiellonian University), Maarten Löffler (Utrecht University), *Yuto Okada (Nagoya University), Alexander Wolff (Universität Würzburg)*

[3] Evaluating Monadic Second-Order Queries with Optimal-Set Quantifiers on Bounded Clique-Width
**Tatsuya Gima (Hokkaido University)*

[4] The PPP-Completeness of the Ward-Sabó Theorem
**Takashi Ishizuka (Technical Institute of Technology, Kochi College)*

[5] Computing Parameterized Squares and Related Repetitions
**Yuto Nakashima (Kyushu University), Jakub Radoszewski, Tomasz Waleń (University of Warsaw)*

[6S] Compression Sensitivity of LZW
**Mashyu Mikami, Dominik Köppl (University of Yamanashi)*

[7S] Correspondence between Subsumption and Language Inclusion Relations for Starlike Term Tree Patterns
**Yuta Horii, Yusuke Suzuki, Tomoyuki Uchida (Hiroshima City University), Takayoshi Shoudai (Fukuoka Institute of Technology), Satoshi Matsumoto (Tokai University), Tetsuhiro Miyahara (Hiroshima City University)*

[8S] Improvement Hitting Sets on Range Avoidance for NC^0_2 circuits
**Masato Baba, Kei Uchizawa (Yamagata University)*

[9] Sublinear-Time Collision Detection with a Polynomial Number of States in Population Protocols
*Takumi Araya, *Yuichi Sudo (Hosei University)*

[10S] Complexity of Computing Equilibrium of Generalized Threshold Game
**Sota Kawase, Nobutaka Shimizu (Institute of Science Tokyo)*

[11S] Minimization of a Jump M-Convex Function with L1-Distance Constraint and Its Application to Matching Problem
**Taiyo Jitsumoto, Akiyoshi Shioura (Institute of Science Tokyo)*

[12S] Reconfiguration of Graph Colorings Under Recolorability Constraints
**Yuta Ito, Rin Saito, Takehiro Ito (Tohoku University)*

[13S] Isomorphism between the Reconfiguration Graph of Independent Sets under Token Jumping and the Underlying Graph
**Naoki Shimada (Shinshu University), Masaya Sano (INTAGE TECHNOSPHERE), Hiroshi Fujiwara, Katsuhisa Ouchi (Shinshu University)*

[14S] On the Biclique Reconfiguration Problem
*Yota Otachi, *Emi Toyoda (Nagoya University)*

[15S] On Connectedness between Graph Colorings on Interval Graphs
**Asato Tsuchiya, Toshinori Yamada (Saitama University)*

[16S] Treewidth of $n \times n$ Toroidal Grids

*Tatsuya Gima (Hokkaido University), *Hiraku Morimoto, Yuto Okada, Yota Otachi (Nagoya University)*

[17S] Towards a Conjecture on Deadlock in Sports Scheduling
**Keita Ishijima, Hiroshi Fujiwara (Shinshu University)*

[18S] A Novel Approximation Algorithm for the Break Minimization Problem
**Koichi Fujii, Tomomi Matsui (Institute of Science Tokyo)*

[19S] Quantum Annealing for Solving the Compatible with the Separation Prohibited and Its Application to Sukoro
**Chiho Fukunaga, Seiya Okubo (University of Shizuoka)*

[20S] On the Group Structure of Planar Rubik's Cube
**Ryuki Kondo, Koichi Yamazaki (Tokyo Denki University)*

[21S] Card-Based Zero-Knowledge Proof for Nuritwin
**Kosuke Ohara, Chuzo Iwamoto (Hiroshima University)*

[22S] Hitting Geodesic Intervals in Structurally Restricted Graphs
*Tatsuya Gima, Yasuaki Kobayashi (Hokkaido University), Yuto Okada, Yota Otachi, *Hayato Takaike (Nagoya University)*

[23S] The (Parameterized) Complexity of Minimizing the Number of Colored Vertices in 2-Clique-Coloring
**Shunsuke Hamada, Yuto Okada, Hirotaka Ono (Nagoya University)*

[24] Max-Min and 1-Bounded Space Algorithms for the Bin Packing Problem
**Hiroshi Fujiwara (Shinshu University), Rina Atsumi (Mitsubishi Electric Digital Innovation Corporation), Hiroaki Yamamoto (Shinshu University)*

[25S] Improved Approximation Guarantees for the Bin Packing Algorithm MM
**Hiromu Hashimoto, Hiroshi Fujiwara, Katsuhisa Ouchi (Shinshu University)*

[26S] An Approximation Algorithm for Average Network Flow Maximization Problems
**Yutaro Fuse, Nobutaka Shimizu, Akiyoshi Shioura (Institute of Science Tokyo)*

[27] An All-LCS Graph for Run-Length Encoded Strings
**Yoshifumi Sakai (Tohoku University)*

[28] Solving the Sparse Matrix Problem for Logarithmic Widths
**Dominik Köppl (University of Yamanashi), Vincent Limouzy (University Clermont Auvergne), Andrea Marino (University of Florence), Jannik Olblich (University of Ulm), Giulia Punzi (University of Pisa), Takeaki Uno (National Institute of Informatics)*

[29S] Improved Data Structures for Uniform Sampling from Subset Families
**Sakuya Kaneko, Daiki Shigematu (Kyushu University), Kohei Hatano (Kyushu University/RIKEN AIP), Eiji Takimoto (Kyushu University)*

[30S] Fixed-Parameter Algorithm for Pre-Assignment Uniquifying Optimal Solution Feedback Vertex Set
**Raimu Nagakura, Hirotaka Ono (Nagoya University), Toshimasa Ishii (Hokkaido University)*

[31S] On Orientable Domination Number
*Hans L. Bodlaender (Utrecht University), Tatsuya Gima, Yasuaki Kobayashi (Hokkaido University), *Futaba Mizuno, Yuto Okada, Yota Otachi (Nagoya University)*

[32S] Enumeration Algorithms for Maximal Matchings in Chain Graphs
**Kanata Maruyama, Naoto Kaneko, Hiroshi Fujiwara (Shinshu University)*

[33] Courcelle's Theorem for Lipschitz Continuity

*Tatsuya Gima (Hokkaido University), *Soh Kumabe (CyberAgent), Yuichi Yoshida (National Institute of Informatics)*

[34S] On the (Meta-)Complexity of Locally Dense Lattice
*Shuichi Hirahara (National Institute of Informatics), *Kazuki Ogitsuka (The Graduate University for Advanced Studies)*

[35S] Faster Approximate Linear Matroid Intersection
**Tatsuya Terao (Kyoto University)*

Past/Forthcoming Events

WALCOM 2026 & 2027
International Conference and Workshops on Algorithms and Computation (WALCOM) conference has been established to encourage the researchers of theoretical computer science in Asia, especially, India and Bangladesh. Nowadays, there are many participants from all around the world. The 20th WALCOM (WALCOM 2026) was held in Perugia, Italy, from March 4th to 6th, 2026. This was the first WALCOM in Europe. See `https://mozart.diei.unipg.it/walcom2026/` for more information on WALCOM 2026. The next WALCOM will be held in Taipei, Taiwan.

AAAC 2026
Annual Meeting of the Asian Association for Algorithms and Computation (AAAC) aims at promoting collaborations in theoretical computer science in Asia (but not restricted in the region). The 17th AAAC was held in Qingdao, China, on May 23rd and 24th, 2026. See `https://math.ouc.edu.cn/aaac2026/` for more information on AAAC 2026.

WAAC 2026
The 26th Korea–Japan Joint Workshop on Algorithms and Computation (WAAC 2026) will be held in Busan, Korea, on August 19th and 20th, 2026. The aim of this workshop is to provide a forum for researchers working on algorithms and the theory of computation, to promote the exchange of recent results, to foster new collaborations among researchers. Historically, the workshop has established for the purpose of collaboration of researchers of Korea and Japan; however, participation from any country is welcome. See `https://algo.postech.ac.kr/workshops/waac26/` for more information of WAAC 2026.

ISAAC 2026
International Symposium on Algorithms and Computation (ISAAC) is intended to provide a forum for researchers working on algorithms and computation. The 37th edition of this symposium will be held in Hangzhou, China, from December 6th to 9th, 2026. See `https://www.algo-door.com/isaac2026/` for more information on ISAAC 2026.

Submission Deadline: June 26, 2026 (Anywhere on Earth)

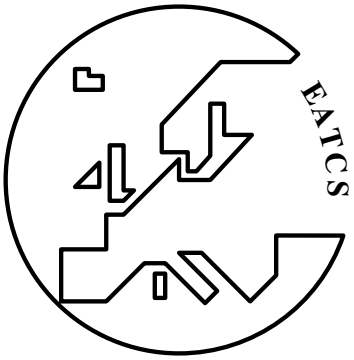
Notification of Acceptance: Early September, 2026

EATCS Japan Chapter

Chair:	Ryuhei Uehara
Vice Chair:	Takehiro Ito
Secretary:	Yuto Nakashima
email:	eatcs-jp@grp.tohoku.ac.jp
URL:	http://www.dais.is.tohoku.ac.jp/eatcs_japan/

BEATCS no 149

European Association for Theoretical Computer Science



E A T C S

164

EATCS

HISTORY AND ORGANIZATION

EATCS is an international organization founded in 1972. Its aim is to facilitate the exchange of ideas and results among theoretical computer scientists as well as to stimulate cooperation between the theoretical and the practical community in computer science.

Its activities are coordinated by the Council of EATCS, which elects a President, Vice Presidents, and a Treasurer. Policy guidelines are determined by the Council and the General Assembly of EATCS. This assembly is scheduled to take place during the annual **I**nternational **C**olloquium on **A**utomata, **L**anguages and **P**rogramming (ICALP), the conference of EATCS.

MAJOR ACTIVITIES OF EATCS

- Organization of ICALP;
- Publication of the "Bulletin of the EATCS;"
- Award of research and academic career prizes, including the EATCS Award, the Gödel Prize (with SIGACT), the Presburger Award, the EATCS Distinguished Dissertation Award, the Nerode Prize (joint with IPEC) and best papers awards at several top conferences;
- Active involvement in publications generally within theoretical computer science.

Other activities of EATCS include the sponsorship or the cooperation in the organization of various more specialized meetings in theoretical computer science. Among such meetings are: CIAC (Conference of Algorithms and Complexity), CiE (Conference of Computer Science Models of Computation in Context), DISC (International Symposium on Distributed Computing), DLT (International Conference on Developments in Language Theory), ESA (European Symposium on Algorithms), ETAPS (The European Joint Conferences on Theory and Practice of Software), LICS (Logic in Computer Science), MFCS (Mathematical Foundations of Computer Science), WADS (Algorithms and Data Structures Symposium), WoLLIC (Workshop on Logic, Language, Information and Computation), WORDS (International Conference on Words).

Benefits offered by EATCS include:

- Subscription to the "Bulletin of the EATCS;"
- Access to the Springer Reading Room;
- Reduced registration fees at various conferences;
- Reciprocity agreements with other organizations;
- 25% discount when purchasing ICALP proceedings;
- 25% discount in purchasing books from "EATCS Monographs" and "EATCS Texts;"
- Discount (about 70%) per individual annual subscription to "Theoretical Computer Science;"
- Discount (about 70%) per individual annual subscription to "Fundamenta Informaticae."

Benefits offered by EATCS to Young Researchers also include:

- Database for Phd/MSc thesis
- Job search/announcements at Young Researchers area

(1) THE ICALP CONFERENCE

ICALP is an international conference covering all aspects of theoretical computer science and now customarily taking place during the second or third week of July. Typical topics discussed during recent ICALP conferences are: computability, automata theory, formal language theory, analysis of algorithms, computational complexity, mathematical aspects of programming language definition, logic and semantics of programming languages, foundations of logic programming, theorem proving, software specification, computational geometry, data types and data structures, theory of data bases and knowledge based systems, data security, cryptography, VLSI structures, parallel and distributed computing, models of concurrency and robotics.

SITES OF ICALP MEETINGS:

- Paris, France 1972
- Saarbrücken, Germany 1974
- Edinburgh, UK 1976
- Turku, Finland 1977
- Udine, Italy 1978
- Graz, Austria 1979
- Noordwijkerhout, The Netherlands 1980
- Haifa, Israel 1981
- Aarhus, Denmark 1982
- Barcelona, Spain 1983
- Antwerp, Belgium 1984
- Nafplion, Greece 1985
- Rennes, France 1986
- Karlsruhe, Germany 1987
- Tampere, Finland 1988
- Stresa, Italy 1989
- Warwick, UK 1990
- Madrid, Spain 1991
- Wien, Austria 1992
- Lund, Sweden 1993
- Jerusalem, Israel 1994
- Szeged, Hungary 1995
- Paderborn, Germany 1996
- Bologne, Italy 1997
- Aalborg, Denmark 1998
- Prague, Czech Republic 1999
- Genève, Switzerland 2000
- Heraklion, Greece 2001
- Malaga, Spain 2002
- Eindhoven, The Netherlands 2003
- Turku, Finland 2004
- Lisabon, Portugal 2005
- Venezia, Italy 2006
- Wrocław, Poland 2007
- Reykjavik, Iceland 2008
- Rhodes, Greece 2009
- Bordeaux, France 2010
- Zürich, Switzerland 2011
- Warwick, UK 2012
- Riga, Latvia 2013
- Copenhagen, Denmark 2014
- Kyoto, Japan 2015
- Rome, Italy 2016
- Warsaw, Poland 2017
- Prague, Czech Republic 2018
- Patras, Greece 2019
- Saarbrücken, Germany (virtual conference) 2020
- Glasgow, UK (virtual conference) 2021
- Paris, France 2022
- Paderborn, Germany 2023
- Tallinn, Estonia 2024
- Aarhus, Denmark 2025

(2) THE BULLETIN OF THE EATCS

Three issues of the Bulletin are published annually, in February, June and October respectively. The Bulletin is a medium for *rapid* publication and wide distribution of material such as:

- EATCS matters;
- Technical contributions;
- Columns;
- Surveys and tutorials;
- Reports on conferences;
- Information about the current ICALP;
- Reports on computer science departments and institutes;
- Open problems and solutions;
- Abstracts of Ph.D. theses;
- Entertainments and pictures related to computer science.

Contributions to any of the above areas are solicited, in electronic form only according to formats, deadlines and submissions procedures illustrated at `http://www.eatcs.org/bulletin`. Questions and proposals can be addressed to the Editor by email at `bulletin@eatcs.org`.

(3) OTHER PUBLICATIONS

EATCS has played a major role in establishing what today are some of the most prestigious publication within theoretical computer science.

These include the *EATCS Texts* and the *EATCS Monographs* published by Springer-Verlag and launched during ICALP in 1984. The Springer series include *monographs* covering all areas of theoretical computer science, and aimed at the research community and graduate students, as well as *texts* intended mostly for the graduate level, where an undergraduate background in computer science is typically assumed.

Updated information about the series can be obtained from the publisher.

The editors of the EATCS Monographs and Texts are now M. Henzinger (Vienna), J. Hromkovič (Zürich), M. Nielsen (Aarhus), G. Rozenberg (Leiden), A. Salomaa (Turku). Potential authors should contact one of the editors.

EATCS members can purchase books from the series with 25% discount. Order should be sent to:

Prof.Dr. G. Rozenberg, LIACS, University of Leiden,
P.O. Box 9512, 2300 RA Leiden, The Netherlands

who acknowledges EATCS membership and forwards the order to Springer-Verlag.

The journal *Theoretical Computer Science*, founded in 1975 on the initiative of EATCS, is published by Elsevier Science Publishers. Its contents are mathematical and abstract in spirit, but it derives its motivation from practical and everyday computation. Its aim is to understand the nature of computation and, as a consequence of this understanding, provide more efficient methodologies.

The Editor-in-Chief of the journal currently are D. Sannella (Edinburgh), L. Kari and P.G. Spirakis (Patras).

ADDITIONAL EATCS INFORMATION

For further information please visit `http://www.eatcs.org`, or contact the President of EATCS:

Prof. Giuseppe F. Italiano,
Email: `president@eatcs.org`

EATCS MEMBERSHIP

DUES

The dues are € 40 for a period of one year (two years for students / Young Researchers). Young Researchers, after paying, have to contact `secretary@eatcs.org`, in order to get additional years. A new membership starts upon registration of the payment. Memberships can always be prolonged for one or more years.

In order to encourage double registration, we are offering a discount for SIGACT members, who can join EATCS for € 35 per year. We also offer a five-euro discount on the EATCS membership fee to those who register both to the EATCS and to one of its chapters. Additional € 35 fee is required for ensuring the *air mail* delivery of the EATCS Bulletin outside Europe.

HOW TO JOIN EATCS

You are strongly encouraged to join (or prolong your membership) directly from the EATCS website `www.eatcs.org`, where you will find an online registration form and the possibility of secure online payment. Alternatively, contact the Secretary Office of EATCS:

Mrs. Efi Chita,
Computer Technology Institute & Press (CTI)
1 N. Kazantzaki Str., University of Patras campus,
26504, Rio, Greece
Email: `secretary@eatcs.org`,
Tel: +30 2610 960333, Fax: +30 2610 960490

If you are an EATCS member and you wish to prolong your membership or renew the subscription you have to use the Renew Subscription form. The dues can be paid via paypal and all major credit cards are accepted.

For adittional information please contact the Secretary of EATCS:

Dmitry Chistikov
Computer Science
University of Warwick
Coventry
CV4 7AL
United Kingdom
Email: `secretary@eatcs.org`,